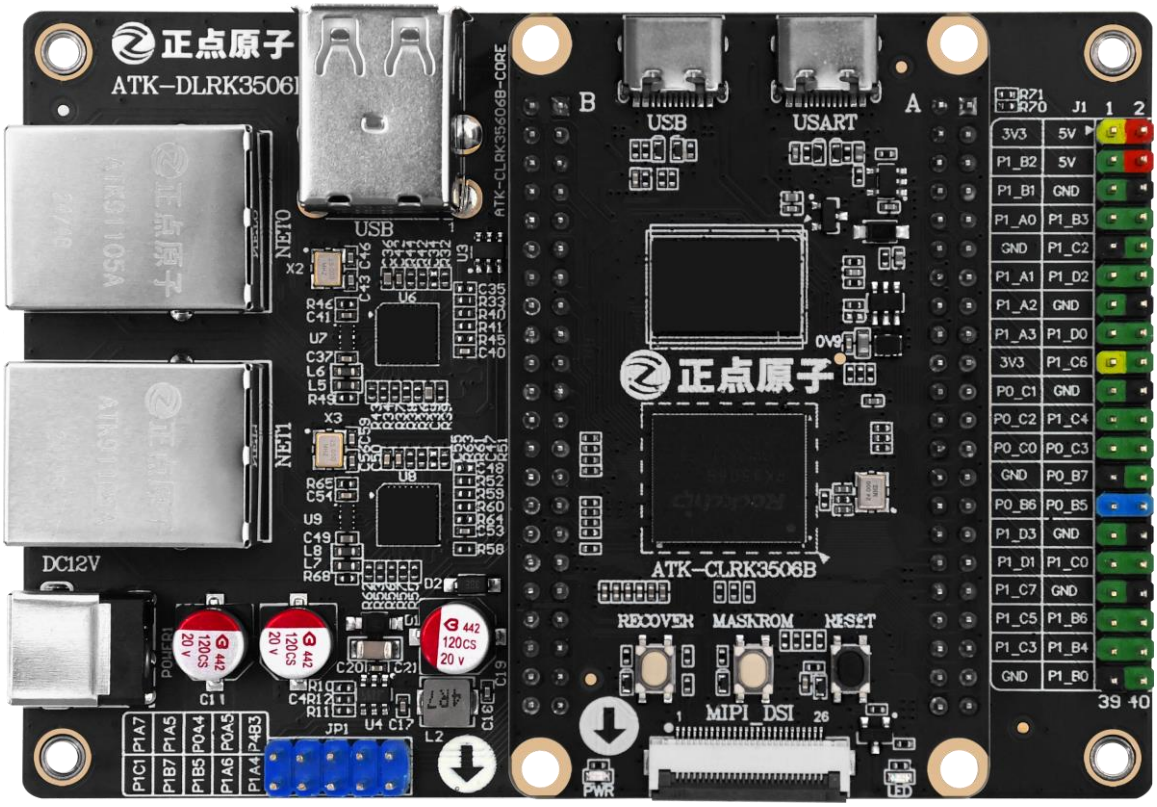
ATK-DLRK3506 嵌入式 Linux 驱动开发指南



正点原子

正点原子公司名称 ：广州市星翼电子科技有限公司

原子哥在线教学平台：www.yuanzige.com

开源电子网 / 论坛 ：http://www.openedv.com/forum.php

正点原子淘宝店铺 ：https://openedv.taobao.com

正点原子官方网站 ：www.alientek.com

正点原子 B 站视频 ：https://space.bilibili.com/394620890

电话：020-38271790 传真：020-36773971

请关注正点原子公众号，资料发布更新我们会通知。

扫码关注正点原子公众号

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	初稿：	正点原子 linux 团队	正点原子 linux 团队	2024.03.20
V1.1	修改一些笔误的地方			
V1.2	修改交叉编译器使用方式			

目录

前言 8
第一章 U-Boot 使用 10
1.1 U-Boot 简介 11
1.1.1 U-Boot 启动过程 12
1.2 U-Boot 命令使用 14
1.2.1 查询命令 14
1.2.2 内存操作命令 15
1.2.3 SD 卡操作命令 18
第二章 Linux 驱动开发准备工作 21
2.1 Linux 内核编译 21
2.1.1 编译 Linux 内核 21
2.2 根文件系统确认 21
2.2.1 /lib/modules/6.1.99+目录介绍 21
2.2.2 检查相关命令是否存在 21
2.3 Ubuntu 下使用 ADB 向开发板发送文件 21
2.4 安装驱动开发所使用的交叉编译器 22
2.5 驱动程序环境验证 22
第三章 内核源码基础使用说明 26
3.1 内核源码目录介绍 26
3.2 内核源码编译 27
3.3 内核模块说明 28
3.4 添加内核配置及菜单选项 29
3.4.1 内核菜单 menuconfig 29
3.4.2 配置文件的使用 32
3.4.3 修改并保存 menuconfig 32
第四章 嵌入式 Linux 内核驱动基础 35
4.1 嵌入式 Linux 内核思路 35
4.1.1 分层思想 35
4.1.2 sysfs 接口 36
4.2 驱动程序框架 38
4.3 设备模型 39
4.4 Platform 平台驱动 40
第五章 GPIO 子系统 42
5.1 GPIO 基础知识 42
5.2 GPIO 子系统简介 43
5.2.1 主要特点 43
5.2.2 组成部分 43
5.2.3 应用场景 43
5.3 开发板 GPIO 接口 44
5.3.1 原理图分析 44
5.3.2 实物硬件 44

5.4 内核 GPIO 子系统使能 45
5.5 设备树文件分析 46
5.6 GPIO 接口配置实验 48
5.6.1 确认目标 GPIO 信息 48
5.6.2 源码编译及固件更新 48
5.6.3 GPIO 输出和输入测试 49
第六章 PWM 子系统 51
6.1 PWM 基础知识 51
6.2 PWM 子系统简介 51
6.2.1 主要特点 51
6.2.2 组成部分 52
6.2.3 应用场景 52
6.3 开发板 PWM 接口 53
6.3.1 pwm 节点介绍 53
6.3.2 pwm 引脚复用 54
6.4 PWM 测试方法 55
6.4.1 使用 sysfs 方式操控 PWM 55
6.4.2 目标管脚 PWM 测试 56
第七章 LED 子系统 58
7.1 LED 基础知识 58
7.2 LED 子系统简介 58
7.2.1 主要特点 58
7.2.2 组成部分 59
7.2.3 应用场景 60
7.3 开发板 LED 设备 62
7.3.1 原理图分析 62
7.3.2 实物硬件 62
7.4 设备树文件分析 63
7.5 LED 测试方法 63
7.5.1 使用 sysfs 方式操控 LED 63
第八章 I2C 子系统 65
8.1 I2C 基础知识 65
8.2 I2C 子系统简介 66
8.3 主要特点 66
8.4 组成部分 66
8.5 开发板 I2C 设备 68
8.5.1 原理图分析 68
8.5.2 实物硬件 70
8.6 内核 I2C 子系统使能 70
8.7 设备树文件分析 70
8.8 屏幕触摸的使用 71
8.8.1 触摸测试 72
第九章 SPI 子系统 74

9.1 SPI 基础知识 74
9.2 SPI 子系统简介 75
9.3 主要特点 75
9.4 组成部分 76
9.5 开发板 SPI 设备 77
9.5.1 原理图分析 77
9.6 内核 SPI 子系统使能 77
9.7 设备树文件分析 77
9.8 SPI 调试工具 79
9.8.1 spi-config 79
9.8.2 spi-pipe 79
第十章 UART 子系统 81
10.1 UART 子系统简介 81
10.2 主要特点 81
10.3 组成部分 81
10.4 原理图分析 82
10.5 内核 UART 子系统使能 83
10.6 设备树文件分析 83
10.7 系统接口 84
10.7.1 /dev/ttyS* 84
10.7.2 /sys/class/tty/ttyS* 84
10.8 UART 测试 85
第十一章 Display 子系统 87
11.1 Display 子系统简介 87
11.2 主要特点 87
11.3 组成部分 87
11.4 开发板 MIPI 接口 88
11.4.1 原理图分析 88
11.4.2 实物硬件 89
11.5 设备树分析 89
第十二章 Ethernet 子系统 92
12.1 Ethernet 子系统简介 92
12.2 组成部分 92
12.3 主要功能 92
12.4 开发板 Ethernet 接口 93
12.4.1 原理图分析 93
12.4.2 硬件实物 94
12.5 内核 Ethernet 使能 94
12.6 设备树分析 94
12.7 ETH 工具 96
12.7.1 dmesg 96
12.7.2 ifconfig 97
12.7.3 udhcpc 98

12.7.4 ping 99
第十三章 Audio 子系统 100
13.1 Audio 子系统简介 100
13.2 组成部分 100
13.3 开发板 Audio 接口 101
13.3.1 原理图分析 101
13.3.2 硬件实物 102
13.4 内核 Audio 相关使能 103
13.5 设备树分析 103
13.6 ALSA 工具 104
13.6.1 alsamixer 104
13.6.2 amixer 106
13.7 音频测试 106
13.7.1 播放音频测试 106
13.7.2 录音测试 107
第十四章 WiFi 子系统 108
14.1 WiFi 子系统简介 108
14.2 组成部分 108
14.3 开发板 WiFi 说明 109
14.3.1 原理图分析 109
14.3.2 硬件实物 110
14.4 内核 WiFi 相关使能 110
14.5 WiFi 工具 110
14.5.1 connmanctl 110
第十五章 Bluetooth 子系统 116
15.1 Bluetooth 子系统简介 116
15.2 组成部分 116
15.3 开发板 Bluetooth 说明 117
15.3.1 原理图分析 117
15.3.2 硬件实物 118
15.4 内核 Bluetooth 相关使能 118
15.5 板载蓝牙测试 118

前言

本教程针对 ATK-DLRK3506 编写的 Linux 驱动开发教程，如果你没有 Linux 基础，请看我们的网盘资料中的另一份文档《【正点原子】Linux 驱动开发基础》，如果你已经有 Linux 开发基础，那么恭喜你，让我们一起来学习 ATK-DLRK3506 的 Linux 驱动开发吧!

免责声明

本文档所提及的产品规格和使用说明仅供参考，如有内容更新，恕不另行通知；除非有特殊约定，本文档仅作为产品指导，所作陈述均不构成任何形式的担保。本文档版权归广州市星翼电子科技有限公司所有，未经公司的书面许可，任何单位和个人不得以营利为目的进行任何方式的传播。

为了得到最新版本的产品信息，请用户定时访问正点原子资料下载中心或者与淘宝正点原子旗舰店客服联系索取。感谢您的包容与支持。

第一章 U-Boot 使用

U-Boot 用于引导 Linux 系统，所以要先使用一下 U-Boot，体验一下 U-Boot 是个什么东西。正点原子所有的 Linux 开发板出厂系统已经烧写了 U-Boot，所以可以直接拿开发板体验 U-Boot。当然了，提供资料里面也有 U-Boot 源码，大家也可以自行编译 U-Boot 源码烧写测试。本章我们主要学习使用 U-Boot 的相关命令。

1.1 U-Boot 简介

Linux 系统要启动需要通过 bootloader 程序引导，也就说芯片上电以后先运行一段 bootloader 程序。这段 bootloader 程序会先初始化 DDR 等外设，然后将 Linux 内核从 flash(NAND，NOR FLASH，SD，EMMC 等)拷贝到 DDR 中，最后启动 Linux 内核。当然了，bootloader 的实际工作要复杂的多，但是它最主要的工作就是启动 Linux 内核，bootloader 和 Linux 内核的关系就跟 PC 上的 BIOS 和 Windows 的关系一样，bootloader 就相当于 BIOS。所以我们要先搞定 bootloader，很庆幸，有很多现成的 bootloader 软件可以使用，比如 U-Boot、vivi、RedBoot 等等，其中以 U-Boot 使用最为广泛，为了方便书写，本教程会将 U-Boot 写为 uboot。

uboot 的全称是 Universal Boot Loader，uboot 是一个遵循 GPL 协议的开源软件，uboot 是一个裸机代码，可以看作是一个裸机综合例程。现在的 uboot 已经支持液晶屏、网络、USB 等高级功能。uboot 官网为 https://www.denx.de/project/u-boot/，如图 1.1.1 所示：

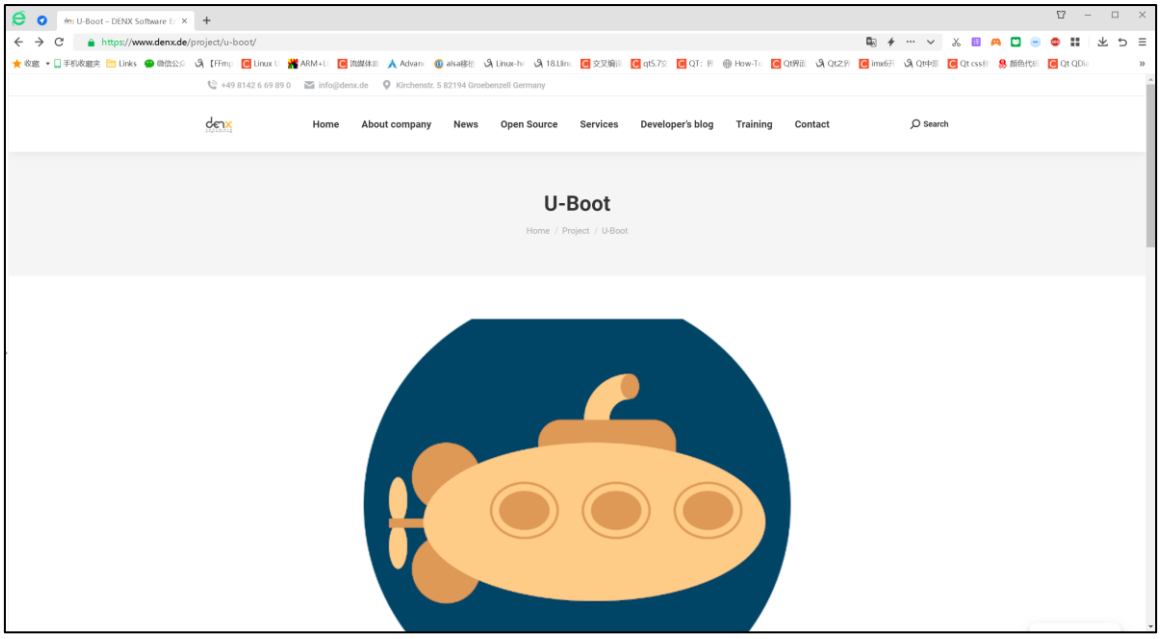


图 1.1.1 uboot 官网

我们可以在 uboot 官方 FTP 下载服务器地址为 https://ftp.denx.de/pub/u-boot/，进入此 FTP 服务器即可看到 uboot 源码，如图 1.1.2 所示：

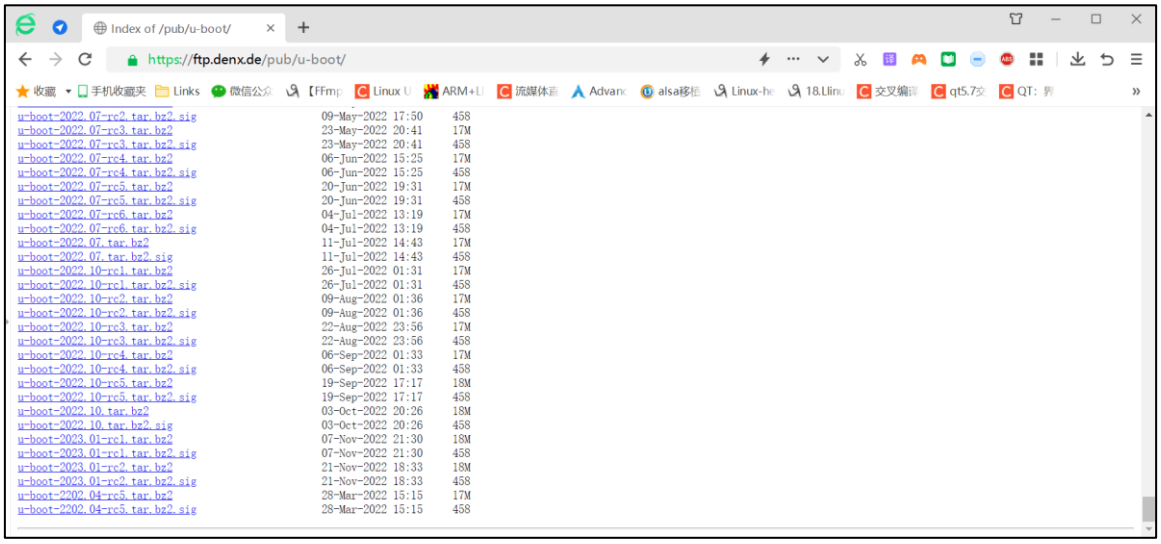


图 1.1.2 uboot 源码

图 1.1.2 中就是 uboot 原汁原味的源码文件，目前最新的版本是 2023.01-rc2。但是我们一般不会直接用 uboot 官方的 U-Boot 源码的。uboot 官方的 uboot 源码是给半导体厂商准备的，半导体厂商会下载 uboot 官方的 uboot 源码，然后将自家相应的芯片移植进去。也就是说半导体厂商会自己维护一个版本的 uboot，这个版本的 uboot 相当于是他们定制的。既然是定制的，那么肯定对自家的芯片支持会很全，虽然 uboot 官网的源码中一般也会支持他们的芯片，但是绝对是没有半导体厂商自己维护的 uboot 全面。比如瑞芯微的 RK3506 6.1.99 内核版本的 SDK 包提供了 2017.09 版本的 uboot，由瑞芯微开发维护。

1.1.1 U-Boot 启动过程

烧写完成以后用 USB Type-C 线将开发板上的 UART 接口与电脑连接起来，因为我们要在串口终端里面输入命令来操作 uboot。

打开 MobaXterm，设置好串口参数瑞芯微系列芯片波特率一般为 1500000，最后复位开发板。在 MobaXterm 上出现“Hit key to stop autoboot('CTRL+C'):”倒计时的时候按下键盘上“CTRL+C”按键。为了加快系统启动速度，开发板默认是 0 秒倒计时，所以大家一般很难卡准时间按下按键。所以建议大家按下开发板复位按键的时候，就一直按 CTRL+C，这样就能很轻松的进入到 uboot 的命令行。

注意！一般 uboot 在倒计时的时候随便按下键盘上哪个按键都可以进入 uboot 命令行，但是如瑞芯微的 uboot 应该是被瑞芯微改过，只能按“CTRL+C”组合键！按其他按键是不能打断启动过程计入命令行的。

以瑞芯微的 RK3506 开发板为例，进入 uboot 的命令行模式以后如图所示：

```
U-Boot 2017.09-g0e48f04 #alientek (Mar 18 2025 - 16:53:50 +0800)

Model: Rockchip RK3506 EVB Board
MPIDR: 0xf00
PreSerial: 0, raw, 0xff0a0000
DRAM:  512 MiB
Sysmem: init
Relocation Offset: 1d4f2000
Relocation fdt: 1c4fb3d0 - 1c4fede0
CR: M/C/I
Using default environment

Hotkey: ctrl+c
DM: v2
SPI Nand ID ef aa 23
Bootdev(atags): mtd 1
PartType: EFI
boot mode: None
RESC: 'boot', blk@0x0000dd91
resource: sha256+
FIT: no signed, no conf required
DTB: rk-kernel.dtb
HASH(c): OK
vdd_arm init 1011000 uV
Could not find baseparameter partition
Model: ATK-DLRK3506B Board
Rockchip UBOOT DRM driver version: v1.0.1
Using display timing dts
dsi@ff640000:  detailed mode clock 65000 kHz, flags[a]
    H: 0720 0768 0776 0828
    V: 1280 1296 1302 1317
bus_format: 100e
VOP:0xff600000 update mode to: 720x1280p60, type: MIPI0
VOP:0xff600000 set crtc_clock to 65000KHz, get 62500000Hz
Unsupported bt709f at 10bit csc depth, use bt601f instead
VOP:0xff600000 set plane [400x400->400x400@160x440] fmt[0] addr[0x1d900000]
final DSI-Link bandwidth: 849 Mbps x 2
CLK: (arm clk use pvtpll, rate = 1200M)
  gpll 1500000 KHz
  v0pll 1179648 KHz
  v1pll 903168 KHz
  clk_gpll_div 187500 KHz
  clk_gpll_div_100m 93750 KHz
  clk_v0pll_div 147456 KHz
  clk_v1pll_div 129024 KHz
  aclk_bus_root 187500 KHz
  hclk_bus_root 147456 KHz
  pclk_bus_root 93750 KHz
  aclk_hsperi_root 147456 KHz
  hclk_ksperi_root 93750 KHz
Net:   eth0: ethernet@ff4c8000, eth1: ethernet@ff4d0000
Hit key to stop autoboot('CTRL+C'):  0
=> <INTERRUPT>
```

图：启动 log 信息

可以看出，当进入到 uboot 的命令行模式以后，左侧会出现一个“=>”

简单介绍一下 uboot 启动过程中都打印出了哪些信息：

1、uboot 版本号，比如上图中当前 uboot 版本号是 2017.09。

2、板子信息，当前板子是瑞芯微的 Rockchip RK3506 EVB 开发板，这个信息是可以改的，因为正点原子 ATK-DLRK3506B 开发板是直接参考瑞芯微官方的 RK3506 开发板移植的 uboot，所以这部分信息也就没改。

3、DDR 大小为 512MiB（请参考根据个人开发板的配置，有可能为 256MiB）

4、……

最后是“Hit key to stop autoboot('CTRL+C'):”倒计时提示，倒计时结束之前按下“CTRL+C”组合键就会进入 Uboot 命令行模式。如果在倒计时结束以后没有按下“CTRL+C”组合键，那么 Linux 内核就会启动，Linux 内核一旦启动，uboot 就会寿终正寝。

uboot 的主要作用是引导 kernel，我们现在已经进入 uboot 的命令行模式了，进入命令行模式以后就可以给 uboot 发号施令了。当然了，不能随便发号施令，得看看 uboot 支持哪些命令，然后使用这些 uboot 所支持的命令来做一些工作，下一节就讲解 uboot 命令的使用。

1.2 U-Boot 命令使用

进入 uboot 的命令行模式以后输入“help”或者“？”，然后按下回车即可查看当前 uboot 所支持的命令，如下图所示：

```
=> help
?       - alias for 'help'
android_print_hdr- print android image header
atags   - Dump all atags
base    - print or set address offset
bidram_dump- Dump bidram layout
blk     - Block device sub-system
boot    - boot default, i.e., run 'bootcmd'
boot_android- Execute the Android Bootloader flow.
boot_fit- Boot FIT Image from memory or boot/recovery partition
bootavb - Execute the Android avb a/b boot flow.
bootd   - boot default, i.e., run 'bootcmd'
bootm   - boot application image from memory
bootp   - boot image via network using BOOTP/TFTP protocol
bootz   - boot Linux zImage image from memory
cmp     - memory compare
cp      - memory copy
crypto_sum- crypto checksum engine
dhcp    - boot image via network using DHCP/TFTP protocol
dm      - Driver model low level access
download- enter rockusb/bootrom download mode
dump_irqs- Dump IRQs
dump_resource- dump resource files
echo    - echo args to console
editenv - edit environment variable
env     - environment handling commands
exit    - exit script
ext2load- load binary file from a Ext2 filesystem
ext2ls  - list files in a directory (default /)
ext4load- load binary file from a Ext4 filesystem
ext4ls  - list files in a directory (default /)
ext4size- determine a file's size
false   - do nothing, unsuccessfully
```

图：uboot 的命令列表(部分)

图中只是 uboot 的一部分命令，具体的命令列表以实际为准。图中的命令并不是 uboot 所支持的所有命令，uboot 是可配置的，需要什么命令就使能什么命令。所以图中的命令是正点原子提供的 uboot 中使能的命令，uboot 支持的命令还有很多，而且也可以在 uboot 中自定义命令。这些命令后面都跟有命令说明，用于描述此命令的作用，但是命令具体怎么用呢？我们输入“help(或?) 命令名”就可以查看命令的详细用法，以“boot_fit”这个命令为例，我们输入如下命令即可查看“boot_fit”这个命令的用法：

```
? boot_fit 或 help boot_fit
```

结果如下图所示：

```
=> ? boot_fit
boot_fit - Boot FIT Image from memory or boot/recovery partition

Usage:
boot_fit boot_fit [addr]
```

图：boot_fit 命令使用说明

图中列出了“boot_fit”这个命令的详细说明，其它的命令也可以使用此方法查询具体的使用方法。接下来我们学习一下一些常用的 uboot 命令。

1.2.1 查询命令

常用的和信息查询有关的命令有 2 个： printenv 和 version。

命令“printenv”用于输出环境变量信息，uboot 也支持 TAB 键自动补全功能，输入“print”然后按下 TAB 键就会自动补全命令。直接输入“print”也可以，因为整个 uboot 命令中只有 printenv 的前缀是“print”，所以当输入 print 以后就只有 printenv 命令了。输入“print”，然后按下回车键，环境变量如下图所示：

```
=> printenv
arch=arm
autoload=no
baudrate=115200
board=evb_rk3506
board_name=evb_rk3506
boot_a_script=load ${devtype} ${devnum}:${distro_bootpart} ${scriptaddr} ${prefix}${script}; source ${scriptaddr}
boot_extlinux=sysboot ${devtype} ${devnum}:${distro_bootpart} any ${scriptaddr} ${prefix}extlinux/extlinux.conf
boot_prefixes=/ /boot/
boot_script_dhcp=boot.scr.uimg
boot_scripts=boot.scr.uimg boot.scr
boot_targets=mmc1 mmc0 mtd2 mtd1 mtd0 pxe dhcp
bootargs=storagemedia=mtd androidboot.storagemedia=mtd androidboot.mode=normal
bootcmd=boot_fit;boot_android ${devtype} ${devnum};
bootcmd_dhcp=if dhcp ${scriptaddr} ${boot_script_dhcp}; then source ${scriptaddr}; fi;
bootcmd_mmc0=setenv devnum 0; run mmc_boot
bootcmd_mmc1=setenv devnum 1; run mmc_boot
bootcmd_mtd0=setenv devnum 0; run mtd_boot
bootcmd_mtd1=setenv devnum 1; run mtd_boot
bootcmd_mtd2=setenv devnum 2; run mtd_boot
bootcmd_pxe=dhcp; if pxe get; then pxe boot; fi
bootdelay=0
cpu=armv7
devnum=1
devtype=mtd
distro_bootcmd=for target in ${boot_targets}; do run bootcmd_${target}; done
eth1addr=fe:a3:84:57:8a:01
ethaddr=fa:a3:84:57:8a:01
fdt_addr_r=0x00063000
kernel_addr_c=0x01100000
kernel_addr_r=0x00108000
mem=518144k
```

图：printenv 命令部分结果

图中只是 printenv 命令的部分内容，RK3506 的 uboot 环境变量有很多。uboot 中的环境变量都是字符串，既然叫做环境变量，那么它的作用就和“变量”一样。比如 bootdelay 这个环境变量就表示 uboot 启动延时时间，如果将 bootdelay 改为 5 的话就会倒计时 5s 了。uboot 中的环境变量是可以修改的，有专门的命令来修改环境变量的值，稍后我们会讲解。

命令 version 用于查看 uboot 的版本号，输入“version”，uboot 版本号如下图所示：

```
=> version
U-Boot 2017.09-g0e48f04 #alientek (Mar 18 2025 - 16:53:50 +0800)

arm-none-linux-gnueabihf-gcc (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29)) 10.3.1 20210621
GNU ld (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29)) 2.36.1.20210621
```

图：version 命令结果

1.2.2 内存操作命令

内存操作命令就是用于直接对 DRAM 进行读写操作的，常用的内存操作命令有 md、nm、mm、mw、cp 和 cmp，我们依次来看一下这些命令都是做什么的。

1、md 命令

md 命令用于显示内存值，格式如下：

```
md[.b, .w, .l] address [# of objects]
```

命令中的[.b .w .l]对应 byte、word 和 long，也就是分别以 1 个字节、2 个字节、4 个字节来显示内存值。address 就是要查看的内存起始地址，[# of objects]表示要查看的数据长度，这个数据长度单位不是字节，而是跟你所选择的显示格式有关。比如你设置要查看的内存长度为 20(十六进制为 0x14)，如果显示格式为.b 的话那就表示 20 个字节；如果显示格式为.w 的话就表示 20 个 word，也就是 20*2=40 个字节；如果显示格式为.l 的话就表示 20 个 long，也就是 20*4=80 个字节，另外要注意：uboot 命令中的数字都是十六进制的！不是十进制的！

比如你想查看 0X6EA00 开始的 20 个字节的内存值，显示格式为.b 的话，应该使用如下所示命令：

```
md.b 6EA00 14
```

而不是：

```
md.b 6EA00 20
```

上面说了，uboot 命令里面的数字都是十六进制的，所以可以不用写“0x”前缀，十进制的 20 对应的十六进制为 0x14，所以命令 md 后面的个数应该是 14，如果写成 20 的话就表示查看 32(十六进制为 0x20)个字节的数据。分析下面三个命令的区别：

```
md.b 6EA00 10
md.w 6EA00 10
md.l 6EA00 10
```

上面这三个命令都是查看以 0X6EA00 为起始地址的内存数据，第一个命令以.b 格式显示，长度为 0x10，也就是 16 个字节；第二个命令以.w 格式显示，长度为 0x10，也就是 16*2=32 个字节；最后一个命令以.l 格式显示，长度也是 0x10，也就是 16*4=64 个字节。这三个命令的执行结果如下图所示：

```
=> md.b 7EA00 14
0007ea00: 05 07 00 00 bc 07 00 00 bd 08 00 00 f4 09 00 00    ................
0007ea10: 43 02 00 00                                       C...
=> md.b 6EA00 14
0006ea00: 34 30 a0 e3 f0 60 c8 e1 06 40 a0 e1 93 67 27 e0    40...`...@...g'.
0006ea10: 1e 00 00 ea                                       ....
=> md.w 6EA00 14
0006ea00: 3034 e3a0 60f0 e1c8 4006 e1a0 6793 e027    40...`...@...g'.
0006ea10: 001e ea00 0036 0a00 3000 e594 002b e353    ....6....0..+.S.
0006ea20: 3001 13a0 3000 03e0                        .0...0..
=> md.l 6EA00 14
0006ea00: e3a03034 e1c860f0 e1a04006 e0276793    40...`...@...g'.
0006ea10: ea00001e 0a000036 e5943000 e353002b    ....6....0..+.S.
0006ea20: 13a03001 03e03000 e5843030 e2856020    .0...0..00.. `..
0006ea30: e5846020 e1a00006 e2844034 faff862f     `......4@../...
0006ea40: e2800001 e0866000 e5046010 e1a00006    .....`...`......
```

图：md 命令使用示例

2、nm 命令

nm 命令用于修改指定地址的内存值，命令格式如下：

```
nm [.b, .w, .l] address
```

nm 命令同样可以以.b、.w 和.l 来指定操作格式，比如现在以.l 格式修改 0X6EA00 地址的数据为 0x12345678。输入命令：

```
nm.l 6EA00
```

输入上述命令以后如下图所示：

```
=> nm.l 6EA00
0006ea00: e3a03034 ?
```

图：nm 命令

6EA00 表示现在要修改的内存地址，e3a03034 表示地址 0X6EA00 现在的数据，‘？’后面就可以输入要修改后的数据 0x12345678，输入完成以后按下回车，然后再输入‘q’即可退出(大小写不敏感)，如下图所示：

```
=> nm.l 6EA00
0006ea00: e3a03034 ? 0x12345678
0006ea00: 12345678 ? q
```

图：修改内存数据

修改完成以后再使用 md 命令来查看一下有没有修改成功，如下图所示：

```
=> md.l 6EA00 1
0006ea00: 12345678                                  xV4.
```

图：查看修改后的值

可以看出，此时地址 0X6EA00 的值变为了 0X12345678。

3、nm 命令

mm 命令也是修改指定地址内存值的，使用 mm 修改内存值的时候地址会自增，而使用 nm 命令的话地址不会自增。比如以.l 格式修改从地址 0X6EA00 开始的连续 3 个内存块(3*4=12 个字节)的数据为 0X05050505，操作如下图所示：

```
=> mm.l 6EA00
0006ea00: 12345678 ? 05050505
0006ea04: e1c860f0 ? 05050505
0006ea08: e1a04006 ? 05050505
0006ea0c: e0276793 ? q
```

图：命令 mm

从图可以看出，修改了地址 0X6EA00、0X6EA04 和 0X6EA08 的内容为 0x05050505。使用命令 md 查看修改后的值，结果如下图所示：

```
=> md.l 6EA00 3
0006ea00: 05050505 05050505 05050505                ............
```

图：查看修改后的内存数据

可以看出内存数据修改成功。

4、mw 命令

命令 mw 用于使用一个指定的数据填充一段内存，命令格式如下：

```
mw [.b, .w, .l] address value [count]
```

mw 命令同样以.b、.w 和.l 来指定操作格式，address 表示要填充的内存起始地址，value 为要填充的数据，count 是填充的长度。比如使用.l 格式将以 0X6EA00 为起始地址的 0x08 个内存块(0x08 * 4=32 字节)填充为 0X0A0A0A0A，命令如下：

```
mw.l 6EA00 0A0A0A0A 8
```

然后使用命令 md 来查看，如下图所示：

```
=> md.l 6EA00 8
0006ea00: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
0006ea10: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
```

图 1.2-1 查看修改后的内存数据

看出内存数据修改成功。

5、cp 命令

cp 是数据拷贝命令，用于将 DRAM 中的数据从一段内存拷贝到另一段内存中，或者把 NorFlash 中的数据拷贝到 DRAM 中。命令格式如下：

```
cp [.b, .w, .l] source target count
```

cp 命令同样以.b、.w 和.l 来指定操作格式，source 为源地址，target 为目的地址，count 为拷贝的长度。我们使用.l 格式将 0X6EA00 处的地址拷贝到 0x68A00 处，长度为 0x10 个内存块(0x10 * 4=64 个字节)，命令如下所示：

```
cp.l 6EA00 68A00 10
```

结果如下图所示：

```
=> cp.l 6EA00 68A00 10
=> md.l 6EA00 10
0006ea00: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
0006ea10: 0a0a0a0a 00000533 000002b3 000000dc    ....3...........
0006ea20: 00000179 000006a8 0000014d 000007c5    y.......M.......
0006ea30: 00000628 0000061e 00000000 0000018d    (...............
=> md.l 68A00 10
00068a00: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a    ................
00068a10: 0a0a0a0a 00000533 000002b3 000000dc    ....3...........
00068a20: 00000179 000006a8 0000014d 000007c5    y.......M.......
00068a30: 00000628 0000061e 00000000 0000018d    (...............
```

图：cp 命令操作结果

先使用 md.l 命令打印出地址 0X6EA00 和 0x68A00 处的数据，然后使用命令 cp.l 将 0X6EA00 处的数据拷贝到 0x68A00 处。最后使用 md.l 查看 0x68A00 处的数据有没有变化，检查拷贝是否成功。

6、cmp 命令

cmp 是比较命令，用于比较两段内存的数据是否相等，命令格式如下：

```
cmp [.b, .w, .l] addr1 addr2 count
```

cmp 命令同样以.b、.w 和.l 来指定操作格式，addr1 为第一段内存首地址，addr2 为第二段内存首地址，count 为要比较的长度。我们使用.l 格式来比较 0X6EA00 和 0x08300100 这两个地址数据是否相等，比较长度为 0x10 个内存块(16 * 4=64 个字节)，命令如下所示：

```
cmp.l 6EA00 68A00 10
```

结果如下图所示：

```
=> cmp.l 6EA00 68A00 10
Total of 16 word(s) were the same
```

图：cmp 命令比较结果

1.2.3 SD 卡操作命令

uboot 支持 SD 卡，因此也要提供 SD 卡的操作命令。本教程统一使用 MMC 来代指 SD 卡。uboot 中常用于操作 MMC 设备的命令为“mmc”。

mmc 是一系列的命令，其后可以跟不同的参数，输入“？mmc”即可查看 mmc 有关的命令，如下图所示：

```
=> ? mmc
mmc - MMC sub system

Usage:
mmc info - display info of the current MMC device
mmc read addr blk# cnt
mmc write addr blk# cnt
mmc erase blk# cnt
mmc rescan
mmc part - lists available partition on current mmc device
mmc dev [dev] [part] - show or set current mmc device [partition]
mmc list - lists available devices
mmc hwpartition [args...] - does hardware partitioning
  arguments (sizes in 512-byte blocks):
    [user [enh start cnt] [wrrel {on|off}]] - sets user data area attributes
    [gp1|gp2|gp3|gp4 cnt [enh] [wrrel {on|off}]] - general purpose partition
    [check|set|complete] - mode, complete set partitioning completed
  WARNING: Partitioning is a write-once setting once it is set to complete.
  Power cycling is required to initialize partitions after set to complete.
mmc setdsr <value> - set DSR register value
```

图：mmc 命令

从图中可以看出，mmc 后面跟不同的参数可以实现不同的功能，如下表所示：

命令	描述
mmc info	输出 MMC 设备信息
mmc read	读取 MMC 中的数据。
mmc wirte	向 MMC 设备写入数据。
mmc rescan	扫描 MMC 设备。
mmc part	列出 MMC 设备的分区。
mmc dev	切换 MMC 设备。
mmc list	列出当前有效的所有 MMC 设备。
mmc hwpartition	设置 MMC 设备的分区。
……	……

表：mmc 命令

1、mmc info 命令

mmc info 命令用于输出当前选中的 mmc info 设备的信息，先插入 sd 卡，输入 mmc dev 0 确保切换到 sd 卡再输入命令“mmc info”即可，如下图所示：

```
=> mmc dev 0
mmc@ff480000: 0, mmc@ff480000: 1
switch to partitions #0, OK
mmc0 is current device
=> mmc info
Device: mmc@ff480000
Manufacturer ID: 9f
OEM: 5449
Name: SD32G
Timing Interface: Legacy
Tran Speed: 52000000
Rd Block Len: 512
SD version 3.0
High Capacity: Yes
Capacity: 29.1 GiB
Bus Width: 4-bit
Erase Group Size: 512 Bytes
```

图：mmc info 命令

可以看出，当前选的 SD 版本为 3.0，容量为 32GiB，速度为 52000000Hz=52MHz，4 位宽的总线。还有一个与 mmc info 命令相同功能的命令：mmcinfo，“mmc”和“info”之间没有空格。

2、mmc list 命令

mmc list 命令用于来查看当前开发板一共有几个 MMC 设备，输入“mmc list”，结果如图所示：

```
=> mmc list
mmc@ff480000: 0 (SD)
mmc@ff480000: 1
```

图：扫描 MMC 设备

可以看出当前开发板有一个 MMC 设备：mmc@ff480000: 0 (SD)

3、mmc dev 命令

mmc dev 命令用于切换当前 MMC 设备，命令格式如下：

```
mmc dev [dev] [part]
```

[dev]用来设置要切换的 MMC 设备号，[part]是分区号，如果不写分区号的话默认为分区 0。使用如下命令切换到 SD 卡（请插上你的 SD 卡，如果你有的话）：

```
mmc dev 0          //切换到 SD 卡，0 为 SD 卡
```

结果如下图所示：

```
=> mmc dev 0
switch to partitions #0, OK
mmc0 is current device
```

图 1.2-2 切换到 SD 卡

从图中可以看出，切换到 SD 卡成功，mmc0 为当前的 MMC 设备。

4、mmc part 命令

有时候 SD 卡会有多个分区，可以使用命令“mmc part”来查看其分区，输入如下命令：

```
mmc dev 0  //切换到 CD
mmc part   //查看 CD 分区
```

结果如下图所示：

```
=> mmc dev 0
mmc@ff480000: 0, mmc@ff480000: 1
switch to partitions #0, OK
mmc0 is current device
=> mmc part

Partition Map for MMC device 0  --   Partition Type: DOS

Part    Start Sector    Num Sectors     UUID            Type
  1     8192            1048576         0becb39d-01     0c
  2     1056768         59383808        0becb39d-02     83
```

图：查看 EMMC 分区

可以看出，此时 SD 是 DOS 类型并且有 2 个分区。

5、mmc read 命令

mmc read 命令用于读取 mmc 设备的数据，命令格式如下：

```
mmc read addr blk# cnt
```

addr 是数据读取到 DRAM 中的地址，blk 是要读取的块起始地址(十六进制)，一个块是 512 字节，这里的块和扇区是一个意思，在 MMC 设备中我们通常说扇区，cnt 是要读取的块数量(十六进制)。比如从 SD 的第 1024(0x400)个块开始，读取 16(0x10)个块的数据，命令如下：

```
mmc dev 0                  //切换到 SD
mmc read 68A00 400 10      //读取数据
```

结果如下图所示：

```
=> mmc dev 0
switch to partitions #0, OK
mmc0 is current device
=> mmc read 68A00 400 10

MMC read: dev # 0, block # 1024, count 16 ... 16 blocks read: OK
```

图：mmc read 命令

可以通过 md 命令来查看 0X68A00 处的数据，也就是读取到的 MMC 设备里面的数据，在这里，这些数据是随便读的，没什么意义，就不看了。

第二章 Linux 驱动开发准备工作

在正式开始学习 Linux 驱动开发之前，有一些准备工作要先处理好，比如交叉编译器安装，Linux 内核与设备树的确定，ADB 使用等。

2.1 Linux 内核编译

2.1.1 编译 Linux 内核

本书 Linux 驱动开发全部基于正点原子**出厂 Linux 系统，包括 Buildroot 根文件系统**。但是在 Linux 驱动开发过程中肯定要涉及到修改或编译 Linux 内核源码以及设备树。关于正点原子出厂 SDK 包中的 Linux 内核编译方法请参考开发板光盘→10、用户手册→01、开发文档→【正点原子】ATK-DLRK3506 _SDK 开发手册.pdf。

2.2 根文件系统确认

2.2.1 /lib/modules/6.1.99+目录介绍

系统有/lib/modules/6.1.99+目录，因为后面驱动开发都是将驱动编译成模块，然后放到/lib/modules/6.1.99+目录中。其中 6.1.99+是所使用的 Linux 内核版本号，RK3506 所使用的 linux 内核版本号为 6.1.99+，所以就是/lib/modules/6.1.19+目录，目录如下图所示：

```
root@ATK-DLRK3506:/lib/modules/6.1.99+# pwd
/lib/modules/6.1.99+
root@ATK-DLRK3506:/lib/modules/6.1.99+#
```

图：/lib/modules/6.1.99+目录

2.2.2 检查相关命令是否存在

我们需要使 modprobe 或 insmod 命令来加载驱动模块，所以根文件系统要存在这两个命令。卸载驱动模块的时候需要用到 rmmod 命令，所以需要确保 modprobe、insmod 和 rmmod 这三个命令都存在。

2.3 Ubuntu 下使用 ADB 向开发板发送文件

我们一般都是在 Ubuntu 下进行驱动和应用的编写与编译，最终肯定要将编译出来的应用文件或驱动文件发送到开发板中运行测试。

关于开发板的 adb 安装与使用请参考开发板光盘→10、用户手册→02、辅助文档→【正点原子】adb 工具使用说明.pdf。

在驱动开发中，我们需要使用 adb 将开发板连接到 Ubuntu 下，然后使用如下“adb push”命令将文件通过 adb 发送到开发板。

我们在 Ubuntu 下随便新建一个名为“adbtest.txt”的空文件，然后将其通过 adb 发送到开发板的/lib/modules/6.1.99+目录下。

1、开发板通过 ADB 连接到 Ubuntu

首先使用 adb 命令将开发板连接到 Ubuntu，先用“adb devices”命令查看一下能不能找到开发板，如果能找到的话就会例举出当前 Ubuntu 下所有连接的 adb 设备，如下图所示：

```
alientek@alientek-virtual-machine:~/rk3588_linux_sdk$ adb devices
List of devices attached
fc15b03e160271b0        device
```

图：当前 adb 设备

图中的 fc15b03e160271b0 就是我的开发板，如果找不到对应的设备，请检查 Ubuntu 下相关 adb 命令有没有安装成功，或者 USB OTG 线有没有连接好。

使用“adb push”将前面创建的 adbtest.txt 文件发动到开发板中，输入如下命令：

```
adb push adbtest.txt /lib/modules/6.1.99+/
```

过程如下图所示：

```
alientek@alientek-virtual-machine:~$ adb push adbtest.txt /lib/modules/6.1.99+/
adbtest.txt: 1 file pushed. 0.0 MB/s (2 bytes in 0.003s)
alientek@alientek-virtual-machine:~$
```

图：adb push 运行过程

从图中看出，adbtest.txt 文件已经 push 成功。到开发板中查看一下/lib/modules/5.10.160 目录下是否有 adbtest.txt 这个文件，如下图所示：

```
root@ATK-DLRK3506:/lib/modules/6.1.99+# ls adbtest.txt -l
-rw-rw-rw-    1 root     root             2 Apr 14 12:23 adbtest.txt
root@ATK-DLRK3506:/lib/modules/6.1.99+#
```

图：发送到开发板里面的 adbtest.txt 文件

2.4 安装驱动开发所使用的交叉编译器

编译驱动肯定要用到交叉编译器，这个交叉编译工具链在 SDK 里默认就已经内置的，路径为<SDK>/ prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin，如下图，后续我们也将使用 SDK 内置的交叉编译工具链来编译驱动。

```
alientek@alientek-virtual-machine:~/ATK-DLRK3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin$ ls
arm-none-linux-gnueabihf-addr2line  arm-none-linux-gnueabihf-gcc         arm-none-linux-gnueabihf-gdb-add-index  arm-none-linux-gnueabihf-objdump
arm-none-linux-gnueabihf-ar         arm-none-linux-gnueabihf-gcc-10.3.1  arm-none-linux-gnueabihf-gfortran       arm-none-linux-gnueabihf-ranlib
arm-none-linux-gnueabihf-as         arm-none-linux-gnueabihf-gcc-ar      arm-none-linux-gnueabihf-gprof          arm-none-linux-gnueabihf-readelf
arm-none-linux-gnueabihf-c++        arm-none-linux-gnueabihf-gcc-nm      arm-none-linux-gnueabihf-ld             arm-none-linux-gnueabihf-size
arm-none-linux-gnueabihf-c++filt    arm-none-linux-gnueabihf-gcc-ranlib  arm-none-linux-gnueabihf-ld.bfd         arm-none-linux-gnueabihf-strings
arm-none-linux-gnueabihf-cpp        arm-none-linux-gnueabihf-gcov        arm-none-linux-gnueabihf-ld.gold        arm-none-linux-gnueabihf-strip
arm-none-linux-gnueabihf-dwp        arm-none-linux-gnueabihf-gcov-dump   arm-none-linux-gnueabihf-lto-dump
arm-none-linux-gnueabihf-elfedit    arm-none-linux-gnueabihf-gcov-tool   arm-none-linux-gnueabihf-nm
arm-none-linux-gnueabihf-g++        arm-none-linux-gnueabihf-gdb         arm-none-linux-gnueabihf-objcopy
alientek@alientek-virtual-machine:~/ATK-DLRK3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin$
```

图 sdk 内置的交叉编译器

2.5 驱动程序环境验证

上一个章节讲解了交叉编译器在 sdk 哪个路径下，这个章节我们通过一个简单的例程来验证一下我们的开发环境。

首先是驱动程序，先创建一个 test_hello.c。内容如下：

```
#include <linux/module.h>
#include <linux/miscdevice.h>
#include <linux/fs.h>

static ssize_t hello_read(struct file *f, char __user *u, size_t l, loff_t *o)
{
   printk(KERN_INFO "hello world\n");
   return 0;
}
```

```
static const struct file_operations fops = {
    .owner = THIS_MODULE,
    .read  = hello_read,
};

static struct miscdevice hello_dev = {
    .minor = MISC_DYNAMIC_MINOR,
    .name  = "hello",
    .fops  = &fops,
};

static int __init hello_init(void)
{
    misc_register(&hello_dev);
    printk(KERN_INFO "hello driver loaded\n");
    return 0;
}

static void __exit hello_exit(void)
{
    misc_deregister(&hello_dev);
    printk(KERN_INFO "hello driver unloaded\n");
}

module_init(hello_init);
module_exit(hello_exit);

MODULE_LICENSE("GPL");
```

然后再创建一个应用层测试代码 test.c。内容如下：

```
#include <stdio.h>
#include <fcntl.h>

int main()
{
    int fd = open("/dev/hello", O_RDONLY);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    read(fd, NULL, 0);
    close(fd);
    return 0;
}
```

接着我们来编译代码。首先是编译驱动程序，这里需要我们创建一个 Makefile 文件。内容如下：

```
export CROSS_COMPILE=/home/alientek/ATK-DLRK3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf-
obj-m += test_hello.o

KDIR := /home/alientek/ATK-DLRK3506/kernel

PWD := $(shell pwd)

all:
	$(MAKE) -C $(KDIR) M=$(PWD) modules

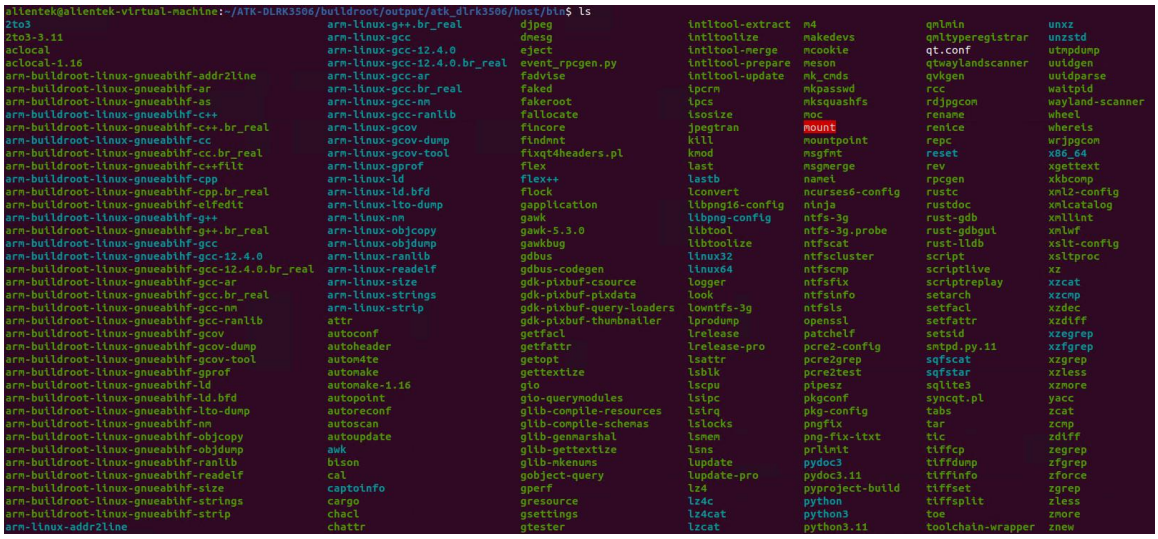
clean:
	$(MAKE) -C $(KDIR) M=$(PWD) clean
```

注意红色部分修改为读者电脑中 sdk 的实际路径。接着执行

```
make ARCH=arm  //编译驱动代码
```

```
alientek@alientek-virtual-machine:~/test$ make ARCH=arm
make -C /home/alientek/ATK-DLRK3506/kernel M=/home/alientek/test modules
make[1]: 进入目录“/home/alientek/ATK-DLRK3506/kernel-6.1”
  CC [M]  /home/alientek/test/test_hello.o
  MODPOST /home/alientek/test/Module.symvers
  CC [M]  /home/alientek/test/test_hello.mod.o
  LD [M]  /home/alientek/test/test_hello.ko
make[1]: 离开目录“/home/alientek/ATK-DLRK3506/kernel-6.1”
```

接着我们编译应用层代码 test.c。需要注意的是，应用层的代码用到的是 buildroot 的编译器，这个编译器需要编译 SDK 后，在 SDK/buildroot/output/atk_dlrk3506/host/bin 下，如图所示：



我们执行：

```
/home/alientek/ATK-DLRK3506/buildroot/output/atk_dlrk3506/host/bin/arm-buildroot-linux-gnueabihf-gcc -o test test.c  //红色部分修改为电脑上实际路径
```

```
alientek@alientek-virtual-machine:~/test$ /home/alientek/ATK-DLRK3506/buildroot/output/atk_dlrk3506/host/bin/arm-buildroot-linux-gnueabihf-gcc -o test test.c
test.c: 在函数‘main’中:
test.c:11:5: 警告: implicit declaration of function ‘read’; did you mean ‘fread’? [-Wimplicit-function-declaration]
   11 |     read(fd, NULL, 0);
      |     ^~~~
      |     fread
test.c:12:5: 警告: implicit declaration of function ‘close’; did you mean ‘pclose’? [-Wimplicit-function-declaration]
   12 |     close(fd);
      |     ^~~~~
      |     pclose
alientek@alientek-virtual-machine:~/test$
```

将生成的 test 和 test_hello.ko 文件拷贝到开发板 /lib/modules/6.1.99+ 目录下。执行：

```
modprobe test_hello.ko        //挂载模块
```

然后执行 ./test 运行代码：

```
root@ATK-DLRK3506:/lib/modules/6.1.99+# modprobe test_hello.ko
[  199.261908] test_hello: loading out-of-tree module taints kernel.
[  199.264099] hello driver loaded
root@ATK-DLRK3506:/lib/modules/6.1.99+# ./test
[  202.662073] hello world
```

第三章 内核源码基础使用说明

3.1 内核源码目录介绍

进入到 ATK-DLRK3506B 的 SDK 源码的内核目录下，执行 ls 指令可以查看内核源码子目录。

```
alientek@alientek-virtual-machine:~/ATK-DLRK3506/kernel$ ls
arch        built-in.a  crypto         include  Kbuild  LICENSES          Makefile              modules.order   resource.img  security  usr       vmlinux.o
block       certs       Documentation  init     Kconfig  logo.bmp          mm                    Module.symvers  rust          sound     virt      vmlinux.symvers
boot.img    COPYING     drivers        io_uring  kernel  logo_kernel.bmp   modules.builtin       net             samples       System.map  vmlinux   zboot.img
boot.its    CREDITS     fs             ipc      lib     MAINTAINERS       modules.builtin.modinfo  README       scripts       tools     vmlinux.a
alientek@alientek-virtual-machine:~/ATK-DLRK3506/kernel$
```

图：内核源码文件

子目录的介绍如下：

文件/路径	说明
arch	包含不同处理器架构的特定实现代码和配置。
block	主要包含了块设备相关的驱动程序和核心子系统代码。
certs	包含了一些安全相关的证书文件。
COPYING	包含 Linux 内核的开源许可证文件。
LICENSES	
CREDITS	对 Linux 内核贡献过的开发者名单。
crypto	包含了加密算法和密码学相关的内核组件。
Documentation	存放内核的文档和说明文件。
drivers	包含各种硬件设备驱动程序的源码。
fs	文件系统相关代码，包含多种 Linux 支持的文件系统的实现。
include	定义了内核的各种数据结构、常量、宏、全局变量声明及函数原型等。
init	初始化相关代码，例如内核启动时的一些基本设置。
ipc	包含了实现内核级进程间通信机制的相关代码。
Kbuild	一种高度自定义的 Makefile 系统，用于指导内核及模块的编译过程。
Kconfig	内核配置文件，用户可以通过配置工具选择内核编译选项。
kernel	内核核心部分的源码，包括调度器、内存管理等。
lib	内核公用函数库和数据结构的源码。
LICENSES	包含了一系列与内核源码相关的许可协议文件。
MAINTAINERS	各个内核子系统、驱动程序和其他组件的主要维护者及其联系信息。
Makefile	控制整个内核编译过程的核心脚本。
mm	包含了内核内存管理子系统的源代码。
Module.symvers	提供了模块与内核交互所需要的符号信息。
net	网络子系统相关代码。
README	包含该版本 Linux 内核的基本信息、内核特性等内容。
samples	包含一些示例代码或内核模块样本。
scripts	包含用于构建、配置、测试、转换以及其他辅助内核编译的脚本文件。
security	安全相关的内核子系统代码。
sound	音频子系统相关代码。
tools	辅助工具集，用于内核开发和分析。
usr	包含了一些与用户空间相关的链接或配置文件。
virt	包含与虚拟化相关的代码和子系统。

3.2 内核源码编译

编译出厂源码时，用户不必修改任何内容。适配于 ATK-DLRK3506B 开发板硬件资源的 Linux 内核源码设备树路径为 kernel/arch/arm/boot/dts。设备树文件为 rk3506b-atk-dlrk3506b*.dts。

```
alientek@alientek-virtual-machine:~/ATK-DLRK3506/kernel/arch/arm/boot/dts$ ls rk3506b-atk-dlrk3506b*.dts
rk3506b-atk-dlrk3506b-nand.dts  rk3506b-atk-dlrk3506b-nand-rwfs.dts  rk3506b-atk-dlrk3506b-sd.dts
alientek@alientek-virtual-machine:~/ATK-DLRK3506/kernel/arch/arm/boot/dts$
```

一些版本的更替，可能名称会有些许区别，基本都是以带 atk 字眼的设备树。

直接在 sdk 顶层目录下运行脚本即可编译，代码如下所示：

```
./build.sh kernel
```

如果是新打开的终端，在执行编译脚本时，需要先执行./build.sh lunch 选择对应的配置文件再去编译。在单独编译内核前，需要完整的编译 sdk。

编译结果如下图所示：

```
alientek@alientek-virtual-machine:~/ATK-DLRK3506$ ./build.sh kernel

############### Rockchip Linux SDK ###############

Manifest: rk3506_linux6.1_release_v1.1.0_20241128.xml

Log colors: message notice warning error fatal

Log saved at /home/alientek/ATK-DLRK3506/output/sessions/2025-03-24_11-02-14
Using last kernel version(6.1)

=========================================
          Final configs
=========================================
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS=/home/alientek/ATK-DLRK3506/device/rockchip/.chip/zboot.its
RK_BOOT_IMG=zboot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=atk_dlrk3506
RK_BUILDROOT_CFG_FRAGMENTS=atk_dlrk3506_append_ubi_rwfs_defconfig
RK_CHIP=rk3506
RK_CHIP_ARM32=y
RK_CHIP_FAMILY=rk3506
RK_DEFCONFIG=atk_dlrk3506b_nand_ubi_rwfs_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ubi:defaults:rk3506_oem:max:@auto:userdata:auto:ubi:defaults:normal:max:@@@
RK_FLASH_SIZE=2048
RK_KERNEL=y
```

图：linux 源码编译结果

3.3 内核模块说明

内核模块是操作系统内核的一部分，它可以在系统运行时动态加载和卸载，而不需要重新编译整个内核或重新启动系统。内核模块通常用于添加设备驱动程序、文件系统或其他系统级功能。这些模块使得开发人员能够根据需要扩展操作系统的功能，而无需对内核进行永久性修改。

内核模块的主要特点包括：

动态加载和卸载：内核模块可以在系统运行时被加载进内核或从内核中移除，提供了极大的灵活性和便利。

功能扩展：通过加载模块，可以根据需要临时添加新的功能，比如支持新的硬件设备或者新的网络协议。

独立开发：模块可以独立于主内核开发和编译，使得开发者可以专注于特定的功能，而不影响整个系统的稳定性。

节省资源：只有当需要特定功能时才加载相应的模块，可以节省系统资源，如内存和处理器时间。

在 Linux 操作系统中，内核模块通常以.ko（kernel object）文件形式存在。开发人员可以使用命令如 insmod、rmmod、modprobe 等来管理这些模块，分别用于加载、卸载和智能处理依赖关系和配置选项的加载。

lsmod 命令是最直接的方式来查看当前加载的内核模块。它实际上是/proc/modules 文件的格式化输出，这个文件包含了所有当前加载的内核模块的信息。开发板出厂系统执行 lsmod，示例如下：

```
root@ATK-DLRK3506:/# lsmod
Module                  Size  Used by    Not tainted
uvcvideo               61213  0
rtk_btusb              44337  0
sd_mod                 24918  0
usb_f_fs               17105  3
libcomposite           28654 10 usb_f_fs
usb_storage            36032  0
scsi_mod               74950  2 sd_mod,usb_storage
scsi_common             2495  2 usb_storage,scsi_mod
dwc2                   98674  0
udc_core               11099  3 usb_f_fs,libcomposite,dwc2
phy_rockchip_inno_usb2    19899  2
8733bu               1377864  0
mac80211              274774  0
cfg80211              177943  2 8733bu,mac80211
rfkill_rk              17654  0
libarc4                  863  1 mac80211
ntfs3                 121030  0
exfat                  45597  0
nls_iso8859_1           3358  1
nls_cp437               4894  1
vfat                    7092  1
fat                    39485  1 vfat
configfs               17392  3 usb_f_fs,libcomposite
root@ATK-DLRK3506:/#
```

图：出厂系统内核模块

决定是否将驱动程序编译进内核或者编译为模块，主要取决于几个关键因素，包括系统的需求、资源限制、安全考虑和可维护性。以下是一些常见情况，用以判断如何处理驱动程序：

将驱动程序编译进内核

必需的启动设备：如果一个驱动程序是用于启动设备（如系统盘的文件系统或者网络启动所需的网络接口），通常需要将其编译进内核，以确保系统启动时可以立即使用这些设备。

性能要求：当对性能有极高要求时，将驱动程序编译进内核可以减少模块加载所需的额外处理和可能的延迟，尽管这种影响通常非常小。

稳定性和安全性：对于一些关键的安全功能，如加密或安全硬件访问，直接集成到内核中可以降低被动态修改的风险。

将驱动程序编译为模块

硬件兼容性：如果系统需要支持多种硬件配置，或者需要能够在不同硬件之间迁移，使用模块可以提供更高的灵活性。模块可以根据需要加载和卸载，不需要为每种配置重新编译内核。

资源限制：在资源受限的环境（如嵌入式系统）中，使用模块可以在不需要的时候卸载它们，从而节省宝贵的内存和存储空间。

开发和测试：开发新的驱动程序或者对现有驱动进行测试时，使用模块可以避免重复编译整个内核。这样可以快速迭代和测试修改。

系统维护和更新：模块可以在不重新启动系统的情况下被更新或替换，这使得系统维护和更新变得更加灵活和方便。

3.4 添加内核配置及菜单选项

3.4.1 内核菜单 menuconfig

内核配置及菜单选项是指在编译操作系统内核之前，通过一套配置界面来选择和定制内核的功能和模块的过程。这种配置允许开发者按需启用或禁用特定的内核功能，从而构建一个满足特定需求的内核。这对于优化系统性能、增加新的支持设备或简化内核以减少资源消耗等目的非常有用。

在 Linux 系统中，内核配置通常是通过 make menuconfig，这是一个基于文本的图形界面，允许用户在类似菜单的结构中浏览不同的内核选项。它适用于大多数终端环境，因为它不需要图形界面。

打开内核菜单 menuconfig 的步骤：

1. 安装必需的软件包

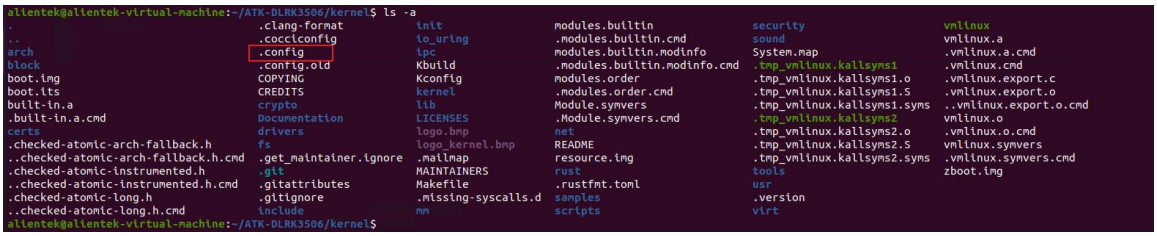
在开始配置内核之前，你需要确保已经安装了编译内核所需的软件包。

```
sudo apt install make build-essential libncurses-dev bison flex libssl-dev bc
```

2.设置内核配置

在准备 Linux 内核进行编译和配置时，需要使用特定的默认配置命令加载一个预设的配置文件，这个配置文件针对特定的硬件平台，对于 ATK-DLRK3506B 开发板出厂内核源码而言，内核配置文件是 arch/arm/configs/rk3506-atk-dlrk3506.config。此配置目的是提供一个良好的出发点，确保出厂系统所有必需的驱动和选项已经被启用。

确保内核源码已经全编译过一次，配置文件会被加载到 .config 文件中，此文件中包含了所有的配置选项和它们的设置。



图：内核配置

在 Linux 内核配置过程中，rk3506-atk-dlrk3506.config 和 .config 文件都扮演着重要的角色，但它们的功能和用途有所不同。

rk3506-atk-dlrk3506.config

定义：rk3506-atk-dlrk3506.config 是一个预设的配置文件，专为特定的硬件平台（如基于 RK3506 处理器，正点原子在这个文件基础上添加了 ATK-DLRK3506 开发板的配置）优化。这个文件通常包含了为该硬件平台推荐的内核选项和模块设置。

用途：使用 make ARCH=arm rk3506-atk-dlrk3506.config 命令，可以将这个预设配置加载到当前的内核构建环境中。这通常是编译适用于特定硬件的内核的第一步，确保所有必需的硬件支持都已启用，并作为基础配置存在。

.config

定义：.config 文件是当前内核构建环境的实际配置文件。它包含了所有内核选项的当前设置，这些设置定义了哪些功能将被编译进内核，哪些将作为模块编译，以及哪些被完全禁用。

用途：这个文件是内核编译过程中使用的直接输入，内核的编译工具（如 make）根据 .config 文件中的设置来编译内核和模块。在使用诸如 make ARCH=arm menuconfig 或其他配置工具修改内核设置时，更改将被保存在这个文件中。

生成和修改：.config 文件可以通过执行类似 make ARCH=arm defconfig、make ARCH=arm oldconfig 或特定的 make xxx_defconfig 命令来生成或更新。它也可以手动编辑，或者通过图形化或文本界面工具进行更新。

区别和联系

区别：rk3506-atk-dlrk3506.config 是一个为特定硬件预设的模板或基线配置文件，而 .config 是根据这些预设模板或通过其他方式生成的实际配置文件，用于控制具体的内核编译过程。

联系：执行 make ARCH=arm rk3506-atk-dlrk3506.config 会将配置文件的内容复制或导入到 .config 文件中。之后，.config 可以进一步修改以满足特定的需求或偏好。

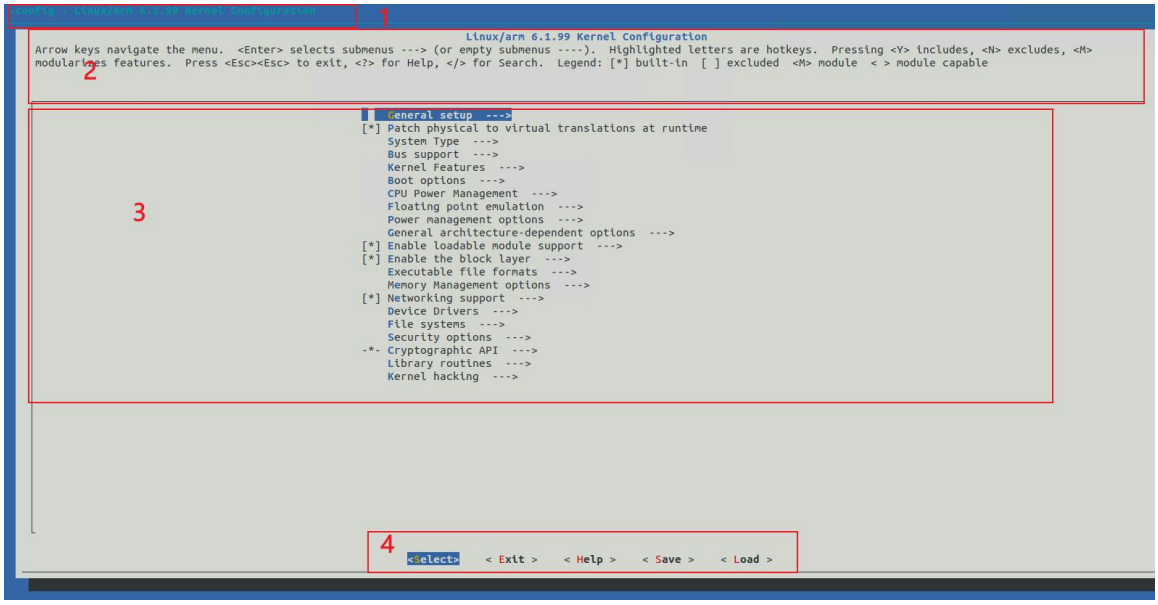
4. 运行 make ARCH=arm menuconfig

在内核源代码的根目录中，运行 make ARCH=arm menuconfig 命令启动配置菜单：

```
make ARCH=arm menuconfig
```

笔者已经在本终端下编译过一次内核源码了，正常情况下使能交叉编译器、设置内核配置文件后就可以启动配置菜单了。menuconfig 是一个菜单界面，对终端界面大小有最低限制要求 19 行和 80 列，否则会有 Your display is too small to run Menuconfig!It must be at least 19 lines by 80 columns.相关报错提示。

正常打开 menuconfig 界面后如下：



图：menuconfig 界面

对于界面的分析如下：

1、顶部栏 (.config - Linux/arm 6.1.99 Kernel Configuration)：

这部分显示了当前的配置文件名称（.config），目标的内核架构（arm），以及内核的版本号（6.1.99）。这提供了关于当前配置会话的基本信息，包括用于正在为哪个架构配置哪个版本的内核。

2、帮助说明区域：

这一区域提供了如何在菜单中导航和操作的基本指南。例如，箭头键用于在菜单中上下移动，回车键（Enter）用来选择子菜单或更改设置等。它还说明了界面底部的快捷键和图例的含义，比如 [ ] 表示可以切换的选项，<*> 表示内建到内核中的功能，<M> 表示编译为模块的功能。

3、主菜单区域：

这里显示了所有可配置的内核选项的主菜单。这些选项通常按功能类别进行分组，例如“General setup”、“Device Drivers”和“File systems”等。用户可以在这些类别中进一步导航以查看和配置更详细的选项。

4、底部按钮区域：

Select：选择当前高亮的菜单项或选项。

Exit：退出当前菜单，如果是在最顶层菜单将退出 menuconfig，会提示你是否保存更改。

Help：显示当前选中的选项或子菜单的帮助信息。

Save：保存当前的配置到 .config 文件中。

Load：加载先前保存的配置。

这些区域的组合为用户提供了一个强大且直观的接口，用于配置和构建 Linux 内核。用户可以通过这些工具精确控制内核的构建过程，确保内核配置与其特定的需求和硬件兼容。

在 make menuconfig 的内核配置界面中，有一系列的键盘操作用于导航和更改配置选项。以下是一些基本的操作：

键盘按键		说明
方向键	上/下箭头键	在菜单项之间上下移动。

	左/右箭头键	在某些上下文中移动，在有可选值的配置选项中进行切换。
回车键		选择菜单项或确认更改。
字母键	Y 键	将选项设置为内建（编译进内核）。
	M 键	将选项设置为模块（在需要时可动态加载）。
	N 键	禁用选项，不构建此模块。
空格键		切换选项的状态（未选中、选中）。
符号键	/键	打开搜索配置选项。
	?键	获取关于当前选中项的详细帮助信息。
Tab 键		在界面不同区域间切换，比如从配置选项列表到底部按钮区域。
Esc 键		退出当前视图或对话框。连续按两次 Esc（Esc Esc）通常会退出当前菜单或者 menuconfig 系统本身。

这些基本操作允许用户在 menuconfig 界面中有效地浏览和修改内核的配置选项。不同的内核版本和不同的配置工具可能会有一些变化，但大部分操作都保持一致。在界面的底部通常会有一行简短的说明，提醒用户可以使用哪些键。

3.4.2 配置文件的使用

当管理员或开发者需要编译内核时，这个.config 文件会被 make 工具读取，根据文件中定义的选项来编译内核。

如果需要更改这些设置，通常会通过运行 make menuconfig 或类似的工具来交互式地调整选项，而不是直接编辑.config 文件。这样可以确保所有依赖关系得到正确处理，并且所有选项都保持一致。

3.4.3 修改并保存 menuconfig

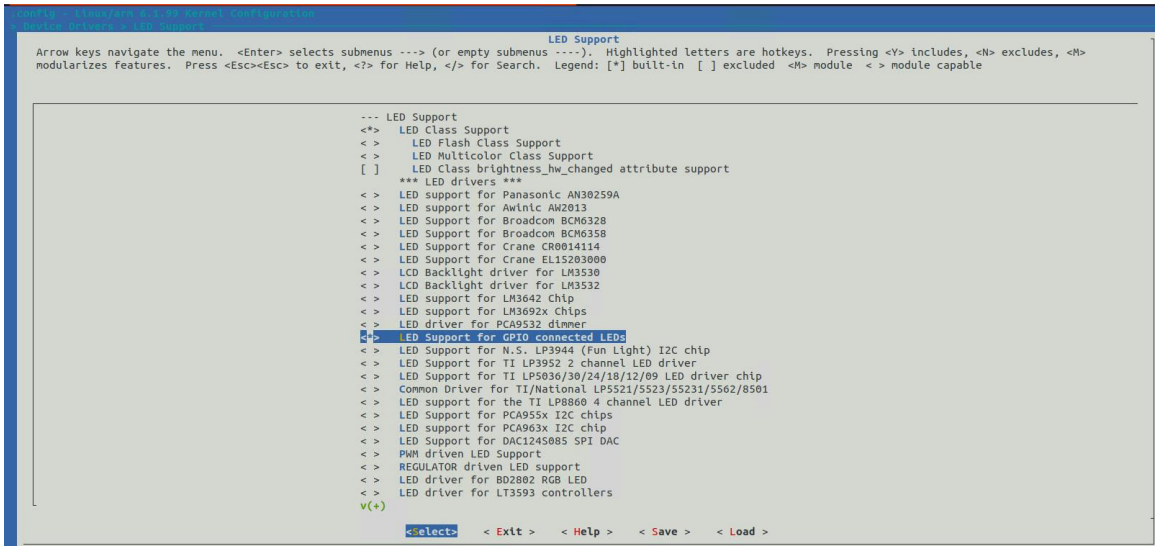
通过 menuconfig 修改内核配置并保存为 rk3506-atk-dlrk3506.config 文件或者自定义文件，这是内核裁剪的基础操作。

进入出厂内核源码目录下，打开 menuconfig 配置。

```
make ARCH=arm menuconfig
```

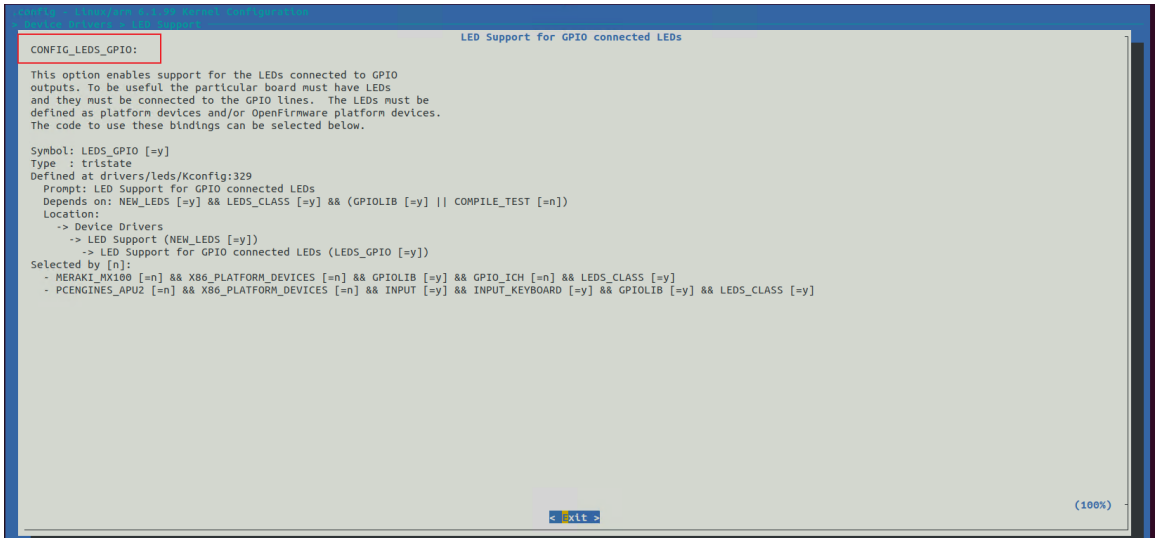
这里以关闭内核心跳灯为例，进到 menuconfig 界面，按如下路径选择

```
-> Device Drivers
  -> LED Support (NEW_LEDS [=y])
    -> LED Support for GPIO connected LEDs (LEDS_GPIO [=y])
```



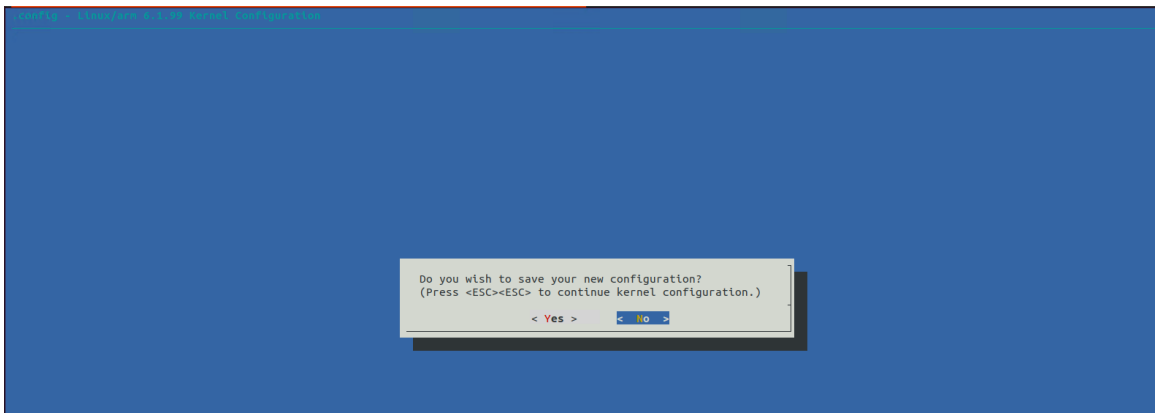
图：中搜索心跳灯

可以看到系统默认是有将 led 灯编译进内核的。选中这一行，按 h 得到心跳灯相关配置和路径信息。



图：心跳灯的信息

可以看到 led 灯的配置选项是 CONFIG_LEDS_GPIO，然后一直按 ESC 退出，最后会提示是否保持配置，我们选择< No >退出 menuconfig 界面。



图：退出 menuconfig

接着打开配置文件 rk3506-atk-dlrk3506.config：

```
vi arch/arm/configs/rk3506-atk-dlrk3506.config
```

```
alientek@alientek-virtual-machine:~/ATK-DLRK3506/kernel$ vi arch/arm/configs/rk3506-atk-dlrk3506.config
alientek@alientek-virtual-machine:~/ATK-DLRK3506/kernel$
```

图：打开配置文件

```
# CONFIG_TOUCHSCREEN_GT1X is not set
CONFIG_TOUCHSCREEN_GT9XX=y
CONFIG_NEW_LEDS=y
CONFIG_LEDS_CLASS=y
CONFIG_LEDS_GPIO=y
CONFIG_LEDS_TRIGGERS=y
CONFIG_LEDS_TRIGGER_HEARTBEAT=y
CONFIG_WLAN_VENDOR_REALTEK=y
CONFIG_RTL8733BU=m
CONFIG_EXT4_FS=y
CONFIG_EXT2_FS=y
CONFIG_EXT2_FS_XATTR=y
CONFIG_EXT2_FS_POSIX_ACL=y
CONFIG_EXT2_FS_SECURITY=y
CONFIG_EXT3_FS=y
CONFIG_USB=y
CONFIG_MOTORCOMM_PHY=y
# CONFIG_FB is not set
# CONFIG_FB_SIMPLE is not set
CONFIG_BT=y
CONFIG_RFKILL=y
CONFIG_BT_HCIUART=y
CONFIG_BT_RTK_HCIBTUSB=m
CONFIG_USB_VIDEO_CLASS=m
CONFIG_MEDIA_USB_SUPPORT=y
CONFIG_SPI_SPIDEV=y
CONFIG_SPI_ROCKCHIP=y
~
```

图：rk3506-atk-dlrk3506.config 配置文件

可以看到在配置文件中，默认将 CONFIG_LEDS_GPIO=y 表示编译进内核，如果要取消编译可以将如下指令去替换掉 CONFIG_LEDS_GPIO=y

```
#CONFIG_LEDS_GPIO is not set
```

保存配置后，接下来进到 sdk 顶层目录下编译内核。

```
./build.sh kernel
```

```
alientek@alientek-virtual-machine:~/ATK-DLRK3506$ ./build.sh kernel

############### Rockchip Linux SDK ###############

Manifest: rk3506_linux6.1_release_v1.1.0_20241128.xml

Log colors: message notice warning error fatal

Log saved at /home/alientek/ATK-DLRK3506/output/sessions/2025-03-24_15-45-07
Using last kernel version(6.1)

=========================================
          Final configs
=========================================
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS=/home/alientek/ATK-DLRK3506/device/rockchip/.chip/zboot.its
RK_BOOT_IMG=zboot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=atk_dlrk3506
RK_BUILDROOT_CFG_FRAGMENTS=atk_dlrk3506_append_ubi_rwfs_defconfig
```

图：重新编译内核

第四章 嵌入式 Linux 内核驱动基础

4.1 嵌入式 Linux 内核思路

4.1.1 分层思想

嵌入式 Linux 系统的设计通常采用分层思想，以便更好地组织和管理系统的不同组成部分。这种分层结构不仅有助于明确每层的功能和责任，还可以简化开发和维护工作，提高系统的模块化和可扩展性。以下是嵌入式 Linux 系统的典型分层架构：

1. 硬件层

这是系统的最底层，包括所有的物理设备，如处理器、内存、输入/输出端口和其他外围设备。硬件层是所有软件操作的基础，它直接与物理设备交互。

2. 驱动层

位于硬件层之上的是驱动层，它包括操作系统的所有硬件驱动程序。这些驱动程序负责管理硬件设备的具体操作，如数据读写、设备控制和状态监控。驱动层的作用是为上层提供一个抽象的、统一的接口，使上层软件不需要关心硬件的具体细节。

3. 操作系统内核层

内核是嵌入式 Linux 系统的核心，负责管理系统资源（如 CPU、内存和设备）、处理系统调用以及执行基本的任务调度和内存管理。内核层还包括各种内核模块和内核级服务，如网络协议栈、文件系统管理等。

4. 中间件层

中间件层位于内核层和应用层之间，提供了更高级别的服务和库，如数据库、多媒体处理、网络通信框架和设备协议。中间件层的目的是简化应用程序的开发，提供跨平台的服务和接口，以及封装复杂的功能。

5. 应用层

应用层包含运行在嵌入式设备上的所有用户级应用程序。这些应用程序利用下层提供的接口和服务来执行特定的任务，如用户界面展示、数据处理、设备控制等。应用层是与用户直接交互的层面，其性能和功能直接影响用户的体验。

6. 用户界面层

在某些复杂的嵌入式系统中，还可能有一个专门的用户界面层，用于管理图形用户界面（GUI）或其他形式的交互界面。这一层通常依赖于底层的图形库或 GUI 框架来实现，如 Qt 或其他专门的嵌入式 GUI 库。

这种分层架构使得嵌入式 Linux 系统的每个部分都可以独立开发和测试，提高了系统的可维护性和可扩展性。此外，分层也有助于在系统设计时进行资源和性能的优化，确保系统能够高效地运行在资源受限的嵌入式硬件上。

在操作系统中，尤其是在像 Linux 这样的现代操作系统中，内存被划分为两个主要区域：用户空间和内核空间。这种划分提高了系统的安全性和稳定性，确保了用户程序不能直接访问硬件资源或其他关键的系统资源。

用户空间（User Space）

用户空间是普通用户程序执行的区域。这些程序包括各种应用软件，如文本编辑器、游戏、网络浏览器等。用户空间的程序通常具有有限的权限，不能直接操作硬件或访问内核数据

结构。这意味着用户程序在执行时更安全，因为即使应用程序崩溃或执行恶意操作，也不会直接危害整个系统。

内核空间（Kernel Space）

内核空间是操作系统内核的运行区域。内核负责管理计算机的硬件资源，如 CPU、内存和 I/O 设备。它也负责进程管理、文件系统操作、网络通信等核心功能。内核空间具有完全的系统访问权限，可以执行硬件级操作。由于内核空间的代码可以直接访问硬件和控制系统操作，因此任何在内核空间中的错误都可能导致严重的系统问题，包括系统崩溃和安全漏洞。

用户空间与内核空间的交互

尽管用户空间和内核空间在权限和功能上有严格的区分，但它们之间仍然需要频繁的交互，以执行诸如读写文件、发送接收网络数据包或访问设备等操作。以下是几种用户空间和内核空间交互的机制：

系统调用：

系统调用是用户空间访问内核服务的主要方式。当用户程序需要执行诸如文件操作、创建进程或者进行网络通信等不能直接完成的任务时，它会通过系统调用请求内核来执行这些操作。

例如，open(), read(), write() 是文件操作相关的系统调用。

虚拟文件系统（如 /proc 和 /sys）：

Linux 提供了一些虚拟文件系统，例如 sysfs，它们以文件系统的形式向用户空间程序提供内核数据的访问。

用户可以通过读写这些特殊的文件来获取系统信息或修改系统行为。

信号：

内核可以向用户空间的进程发送信号，以通知发生了某些系统级事件（如除零错误、访问违规等）。

设备驱动程序：

设备驱动程序在内核空间运行，提供了一种机制，允许用户空间程序通过标准的文件操作接口（如 open、read、write）与硬件设备进行交互。

共享内存：

用户空间程序和内核可以通过共享内存进行通信，这是一种高效的数据交换方法，尤其是在处理大量数据时。

4.1.2 sysfs 接口

在嵌入式 Linux 系统中，sysfs 是一个虚拟文件系统，提供了一种机制，通过它可以从用户空间与内核空间的各种对象进行交互。sysfs 通过文件系统的方式暴露内核设备、驱动程序和其他内核结构的属性，使得用户和应用程序可以通过读写文件的简单操作来获取系统信息或修改系统设置。

功能和作用

1、设备和驱动程序的动态展示：

sysfs 展示了系统中所有设备和驱动程序的结构，每个设备和驱动程序都有一个对应的目录。

设备通常在 /sys/devices 中列出，按照物理、虚拟和总线分类。

驱动程序通常在 /sys/bus/、/sys/class/ 或 /sys/block/ 下列出。

2、属性文件：

在 sysfs 中，设备和驱动的特性通过属性文件表示。这些文件可以是可读的、可写的或两者都有，允许用户和应用程序查看和修改设备设置。

例如，可以读取一个设备的状态，修改设备的配置，或者触发设备执行特定操作。

3、动态性：

sysfs 是动态的，意味着设备被连接或断开时，相应的 sysfs 目录和文件会实时创建或删除。

使用场景

1、设备管理：

管理员和开发人员可以通过 sysfs 监控和配置硬件设备，例如查看网络设备的统计数据或设置 LED 的状态。

2、系统监控：

通过 sysfs 文件读取各种硬件传感器的数据（如温度传感器、电池电量监控等）。

3、调试：

开发人员可以使用 sysfs 来获取有关设备和驱动程序的详细信息，这对于驱动开发和系统调试非常有用。

编程与操作示例

在 Linux 环境中，操作 sysfs 可以通过简单的命令行工具（如 cat、echo）或在程序中使用文件操作 API 来完成。下面是一些示例：

读取属性：

通过 cat 命令读取属性：cat /sys/class/gpio/gpio53/value，可以读取 GPIO 端口 53 的状态。

写入属性：

通过 echo 命令写入属性。

编程访问：

在 C 程序中，可以使用标准的文件操作函数如 open(), read(), write(), 和 close() 来操作 sysfs 文件。例如，打开并读取一个 GPIO 设备的值可能包括打开文件 /sys/class/gpio/gpio53/value，读取值，然后关闭文件。具体可以参考开发板网盘 A 盘-基础资料\10、用户手册\01、开发文档\【正点原子】Linux_C 应用编程参考手册 V1.0.pdf 的 GPIO 应用编程章节。

4.2 驱动程序框架

在 Linux 系统中，驱动程序框架是一种结构化和标准化的方式，用来处理各种硬件设备。Linux 的驱动程序架构设计考虑到了模块性、可移植性和安全性，使得开发人员能够为各种硬件写出高效和稳定的驱动程序。下面是 Linux 驱动程序的一些关键组成部分和概念：

1. 设备驱动类型

Linux 中的设备驱动主要可以分类为以下几种：

字符设备驱动：操作以字符为单位进行数据传输的设备（如串口、文本显示设备）。

块设备驱动：操作以数据块为单位传输的设备（如硬盘驱动器）。

网络设备驱动：处理网络通信的设备（如以太网卡）。

2. 设备模型（Device Model）

Linux 的设备模型提供了一种描述和组织系统中所有设备的方式，包括那些由设备驱动程序控制的设备。这个模型包括：

设备（Devices）：代表系统中的物理或虚拟设备。

驱动（Drivers）：与设备交互的软件组件。

总线（Buses）：连接设备和驱动的通信通道，如 PCI、USB 等。

3. 驱动程序接口

Linux 内核为驱动开发者提供了一系列的核心接口和辅助函数，以简化硬件操作和与系统其他部分的交互。这些接口包括：

文件操作接口：如 open(), read(), write(), ioctl() 等。

设备注册与注销：使用 register_chrdev() 或 register_blkdev() 等函数来注册和注销设备。

中断处理：驱动程序可以注册中断处理函数，响应硬件中断。

内存管理：包括直接内存访问（DMA）和虚拟内存操作。

4. 设备树（Device Tree）

在嵌入式 Linux 系统中，设备树用于描述硬件设备的信息，它是一个平台独立的数据结构，用于在不修改内核代码的情况下提供有关硬件的详细信息。

5. 模块化

Linux 支持驱动程序的模块化，这意味着驱动可以编译为内核模块（*.ko 文件），并在系统运行时动态加载和卸载。这提供了极大的灵活性和便利：

insmod：加载模块。

rmmod：卸载模块。

modprobe：智能加载模块，自动处理依赖。

6. 驱动程序编写和调试

编写：驱动程序通常用 C 语言编写，遵循 Linux 内核的编码规范。

调试：驱动开发可以使用内核调试工具如 printk 等。

4.3 设备模型

Linux 设备模型是一个核心的组织架构，它为设备、驱动程序、和总线提供了一种结构化的管理方式。这个模型使得 Linux 内核能够高效地处理硬件设备，支持热插拔，并且能够自动管理设备与驱动程序之间的关系。这里详细解释 Linux 设备模型的核心组件和它们的作用。

核心组件

1、设备（Devices）

在设备模型中，设备是指代表系统中的一个硬件组件。每个设备由一个 struct device 结构表示，这个结构包含了描述设备的各种信息，如其在系统中的位置、状态、和与其关联的资源（如内存、I/O 端口等）。

设备对象通常与一个具体的驱动程序关联，驱动程序负责实现与该设备交互的具体逻辑。

2、驱动程序（Drivers）

驱动程序是管理一个或多个特定类型设备的软件模块。每个驱动程序由一个 struct device_driver 结构表示，其中包含了驱动程序的名称、总线类型、以及一系列回调函数，这些函数用于设备的探测、移除、挂起和恢复等。

驱动程序负责响应来自设备或内核的请求，执行如读写设备、处理中断、管理电源等操作。

3、总线（Buses）

总线是连接设备和驱动程序的通道。在设备模型中，每种总线类型（如 PCI、USB、I2C 等）都由一个 struct bus_type 结构表示。

总线类型负责定义和管理挂载在该总线上的设备和驱动程序，提供方法来匹配设备和它们的驱动程序，并处理总线上的热插拔事件。

4、类（Classes）

类是一种高级的抽象，用来组织提供相似功能的设备。例如，所有的声卡设备可能都属于一个名为 sound 的类。

类由 struct class 结构表示，用于创建表示设备功能的统一视图，而不考虑它们连接到哪种类型的总线。

设备模型的功能

1、自动设备注册与注销：当设备被连接到系统（例如通过 USB 接口插入）时，相应的设备对象会被自动创建并注册。当设备断开连接时，设备对象会被自动销毁。

2、驱动程序绑定：内核会自动将设备与合适的驱动程序绑定。这是通过在驱动程序中定义的匹配规则来完成的，匹配规则通常基于设备的 ID、总线类型等信息。

3、热插拔支持：设备模型支持热插拔，允许设备在系统运行时被添加或移除，系统会相应地加载或卸载驱动程序。

4、电源管理：设备模型通过与设备和驱动程序的交互，支持复杂的电源管理操作，如设备的挂起和恢复，这对于移动设备尤其重要。

实现机制

设备模型广泛使用了内核中的 kobject 机制，每个 struct device、struct device_driver 和 struct bus_type 都包含一个 kobject 结构。这使得设备模型能够利用 kobject 提供的功能，如引用计数、层次结构、热插拔事件和 sysfs 集成等。

通过 sysfs，设备模型的各个组成部分（设备、驱动、总线等）都在文件系统中有对应的表示，这使得用户和管理员可以通过文件系统接口来查询和控制硬件设备，例如读写设备属性或触发特定操作。这种设计不仅增强了 Linux 的设备管理能力，也极大地方便了系统管理和调试。

4.4 Platform 平台驱动

在嵌入式 Linux 开发中，平台驱动（Platform Driver）是一个处理没有专门硬件总线的设备的驱动类型。这类设备通常是系统的一部分，如处理器上的集成外设，它们不是通过标准的硬件总线（如 PCI 或 USB）来发现和管理的。平台驱动的概念允许开发者以一种标准的方式来处理这些设备，即使它们不符合标准总线架构。

平台设备和平台驱动

平台设备（Platform Device） 是内核中的一种设备，它代表那些直接集成在系统板上的硬件组件。这些设备通常包括时钟、中断控制器、GPIO 控制器等。

平台驱动（Platform Driver） 则是控制这类设备的驱动程序。这些驱动需要能够加载和初始化设备，处理数据传输，以及可能的中断处理等。

平台驱动的主要组件

1、设备注册：

平台设备在系统启动时通过硬编码或使用设备树（Device Tree）来注册。注册过程包括设备的名称、资源（如内存区域、IRQ 等）和其他配置参数。

2、驱动注册：

平台驱动使用 platform_driver_register()函数来注册自己。注册过程中会指定驱动可以处理的设备名称，以及设备的初始化和退出时调用的函数。

3、驱动和设备的匹配：

当平台设备注册到系统后，内核会尝试为其找到一个匹配的驱动。如果设备的名称与驱动支持的名称相匹配，那么该驱动的初始化函数会被调用。

平台驱动的实现示例

下面是一个简单的平台驱动的代码示例，展示了如何定义、注册和匹配一个平台驱动：

```
1  #include <linux/module.h>
2  #include <linux/platform_device.h>
3  #include <linux/init.h>
4  #include <linux/kernel.h>
5
6  // 设备初始化函数
7  static int my_device_probe(struct platform_device *pdev)
8  {
9      printk("My device is probed!\n");
10     return 0;
11 }
12
13 // 设备移除函数
14 static int my_device_remove(struct platform_device *pdev)
```

```
15 {
16     printk("My device is removed!\n");
17     return 0;
18 }
19
20 // 定义平台驱动结构体
21 static struct platform_driver my_driver = {
22     .probe = my_device_probe,
23     .remove = my_device_remove,
24     .driver = {
25         .name = "my_platform_device", // 与平台设备的名字匹配
26         .owner = THIS_MODULE,
27     },
28 };
29
30 // 驱动加载函数
31 static int __init my_driver_init(void)
32 {
33     return platform_driver_register(&my_driver);
34 }
35
36 // 驱动卸载函数
37 static void __exit my_driver_exit(void)
38 {
39     platform_driver_unregister(&my_driver);
40 }
41
42 module_init(my_driver_init);
43 module_exit(my_driver_exit);
44
45 MODULE_LICENSE("GPL");
46 MODULE_AUTHOR("ATK");
47 MODULE_DESCRIPTION("A simple platform driver");
```

第五章 GPIO 子系统

5.1 GPIO 基础知识

1、GPIO 的基本概念

GPIO（General Purpose Input/Output）引脚是一种通用的数字信号引脚，能够在输入和输出之间进行配置，用于与外部设备进行通信。

2、GPIO 引脚的配置模式

每个 GPIO 引脚可以配置为不同的模式，根据所需的功能进行设定。主要有以下几种模式：

- 输入模式（Input Mode）：用于读取外部设备的信号。引脚的状态由外部信号决定。
- 输出模式（Output Mode）：用于向外部设备发送信号。引脚的状态由程序控制。
- 输入上拉模式（Input Pull-Up Mode）：在输入模式的基础上，启用内部上拉电阻，以确保引脚在未连接外部信号时处于高电平状态。
- 输入下拉模式（Input Pull-Down Mode）：在输入模式的基础上，启用内部下拉电阻，以确保引脚在未连接外部信号时处于低电平状态。

3、GPIO 引脚的电平状态

GPIO 引脚的电平状态有两种：

- 高电平（High Level）：对应逻辑 1，通常表示高电压（如 3.3V 或 5V）
- 低电平（Low Level）：对应逻辑 0，通常表示低电压（如 0V）

4、中断（Interrupt）

中断是一种重要的机制，可以在特定事件发生时打断正常的程序执行，立即执行预先定义的中断服务程序（ISR）。在 GPIO 中，中断常用于处理按钮按下、传感器触发等事件。

5、内部上拉和下拉电阻

内部上拉和下拉电阻用于确保在引脚未连接外部信号时，有一个确定的电平状态。通常用于输入引脚，以避免浮空引脚的噪声干扰。

- 上拉电阻（Pull-Up Resistor）：将未连接的引脚拉到高电平。
- 下拉电阻（Pull-Down Resistor）：将未连接的引脚拉到低电平。

6、GPIO 的高级应用

GPIO 引脚可以用于更多高级应用，如：

- PWM 输出：生成脉宽调制信号，用于控制电机速度、LED 亮度等。
- 模拟信号读取：通过 ADC 将模拟信号转换为数字信号进行处理。
- 串行通信：通过 I2C、SPI、UART 等接口实现与其他设备的通信。
-

7、GPIO 的实际应用

在实际项目中，GPIO 可以用于各种场景，如：

- 控制 LED：通过 GPIO 引脚控制 LED 的亮灭。
- 读取传感器数据：连接温度传感器、光敏传感器等，读取并处理传感器数据。

- 按钮输入：检测按钮按下状态，执行相应操作。
- 驱动继电器：通过 GPIO 引脚控制继电器，实现高电压设备的开关控制。

5.2 GPIO 子系统简介

Linux 内核的 GPIO（General Purpose Input/Output，通用输入输出）子系统是为了管理和控制硬件上的 GPIO 引脚而设计的。它允许开发者在内核空间和用户空间中控制引脚的状态，实现输入和输出功能。这个子系统在嵌入式系统和硬件交互中扮演了重要的角色，使得与外界环境的交互成为可能。

5.2.1 主要特点

1. 通用性：GPIO 子系统提供了一个统一的接口来操作不同硬件上的 GPIO 引脚，确保了在不同平台间的可移植性和兼容性。
2. 灵活性：支持将 GPIO 引脚配置为输入或输出模式，以及配置上拉/下拉电阻，满足不同的硬件需求。
3. 易用性：通过提供用户空间的访问接口（如 sysfs 和 GPIO character device API），简化了对 GPIO 引脚的操作，使得开发者可以不编写内核驱动程序即可控制 GPIO。
4. 中断支持：GPIO 子系统允许 GPIO 引脚被配置为中断源，可以在特定事件（如电平变化）发生时通知处理器，实现高效的事件响应。

5.2.2 组成部分

1. gpiolib：是 Linux 内核中的 GPIO 核心库，提供了一套标准的 API 来管理和控制 GPIO 引脚。它作为一个抽象层，屏蔽了不同硬件之间的差异。
2. GPIO 控制器驱动：为特定的硬件平台实现 GPIO 操作的细节。每个控制器驱动管理一组 GPIO 引脚，并注册到 gpiolib，以供上层使用。
3. 设备树（Device Tree）：在许多基于 ARM 的系统中，GPIO 的配置（如引脚编号、功能、默认状态等）在设备树中静态描述，内核通过解析设备树来初始化和配置 GPIO 系统。
4. 用户空间接口：

- sysfs：早期 Linux 版本通过 sysfs 文件系统提供对 GPIO 引脚的直接访问。虽然简单，但不推荐在新的开发中使用，因为它缺乏必要的同步机制和保护。
- GPIO character device：是一个较新的接口，提供了一种更加安全和高效的方式来从用户空间访问 GPIO 引脚，是目前推荐的方法。

5.2.3 应用场景

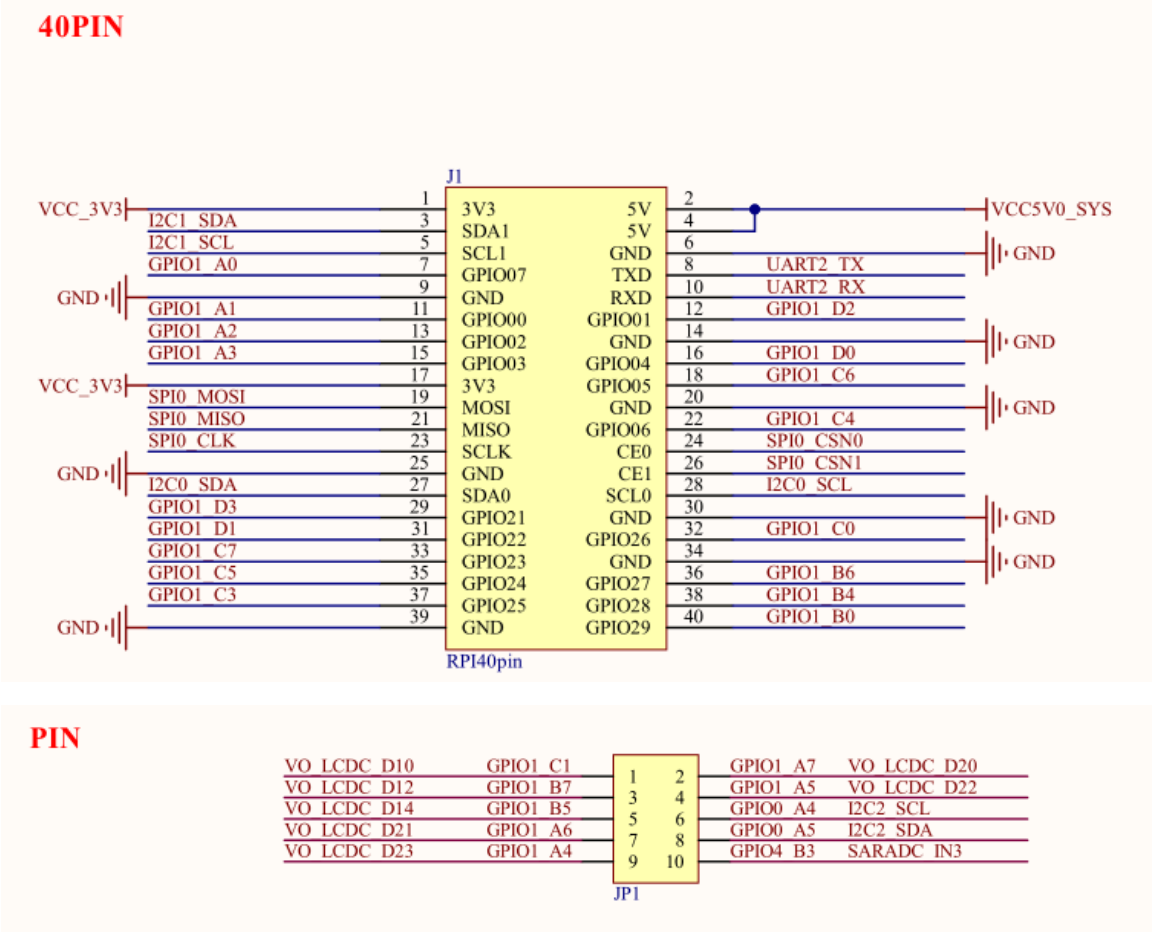
GPIO 子系统广泛应用于嵌入式系统中，用于控制 LED 灯、读取按钮状态、驱动继电器、读取传感器数据等。凭借其高度的灵活性和易用性，GPIO 子系统使得 Linux 内核能够有效地与各种硬件设备交互，满足多样化的应用需求。

总的来说，GPIO 子系统是 Linux 内核与硬件交互不可或缺的一部分，它通过提供一系列的抽象和接口，简化了 GPIO 的管理和控制。随着技术的发展，GPIO 子系统也在不断进化，以满足不断增长的性能和功能需求。

5.3 开发板 GPIO 接口

5.3.1 原理图分析

打开开发板上开发板网盘 A 盘-基础资料\02、原理图\01、底板原理图\ ATK-DLRK3506B V1.0.pdf。开发板上引出了 38 个 GPIO 管脚，这些引脚均可以作为 GPIO 使用，如下图所示：

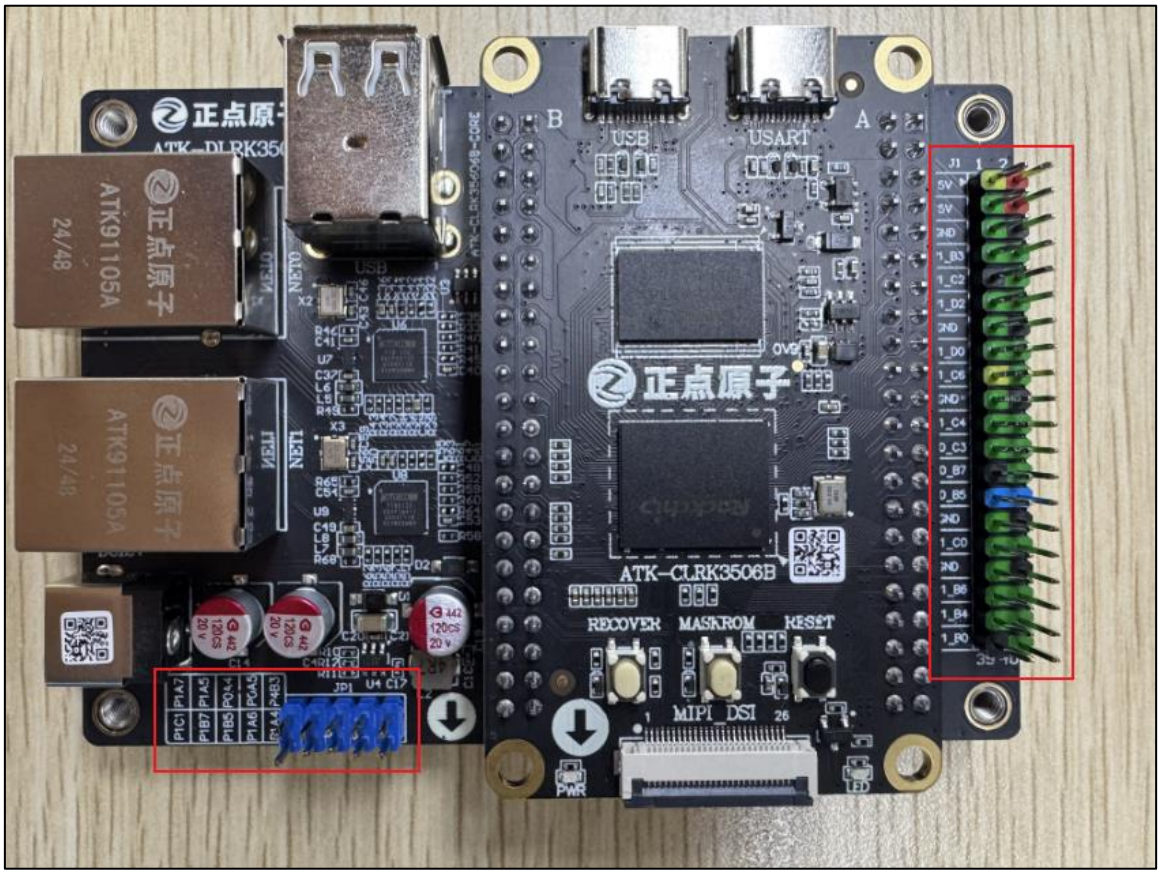


图：开发板原理图 GPIO 部分

对于给定的一个 GPIO 引脚，如何计算它在 sysfs 中对应的编号呢？其实非常简单，Rockchip Pin 的 ID 按照：控制器(bank)+端口(port)+索引序号(pin)组成。控制器和 GPIO 控制器数量一致；端口固定 A、B、C 和 D；索引序号固定 0、1、2、3、4、5、6、7。举例来说，例如：GPIO1_A4：拆解来看就是 GPIO1（控制器）+A（端口号）+4（索引序号）。GPIO1 控制器对应的最小编号为 32;端口号 A 的编号为 0；最后加上索引号 4；即 GPIO1_A4 的引脚号为 32+0+4=36。

5.3.2 实物硬件

开发板上的排针如下图所示：



图：开发板上的 GPIO 排针

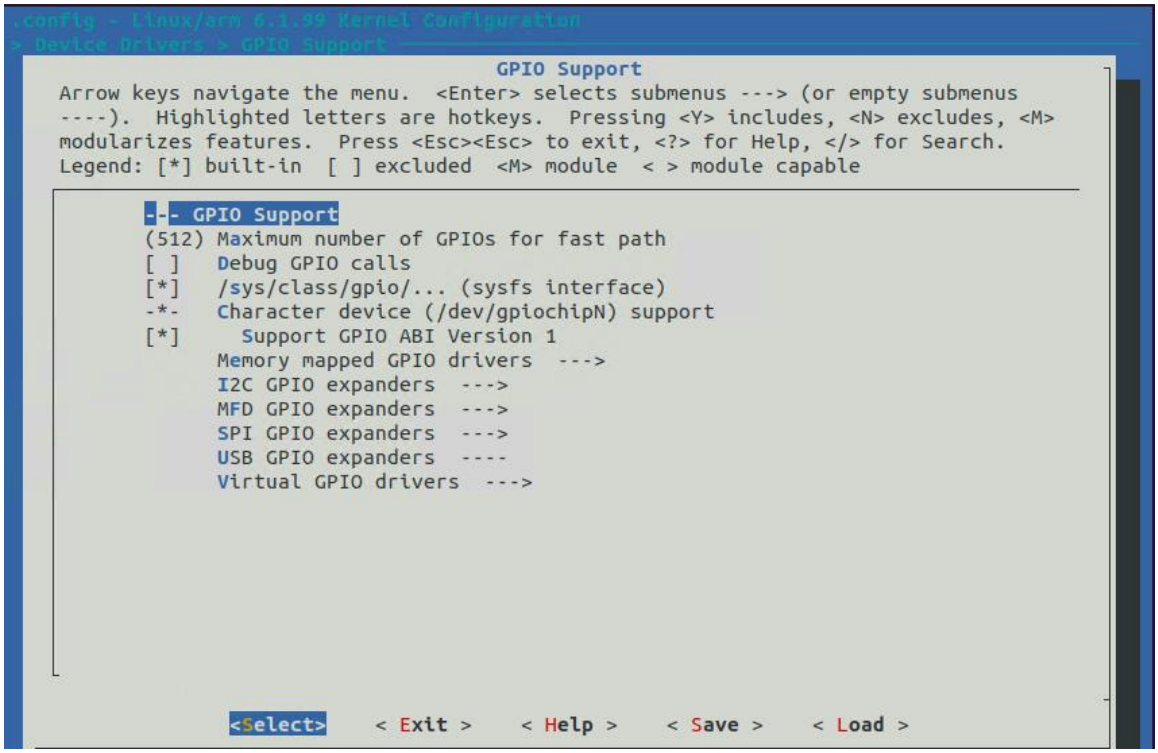
5.4 内核 GPIO 子系统使能

内核源码配置文件 arch/arm/configs/rk3506_defconfig，GPIO 系统相关配置如下(略有删减，只列举重点部分)：

```
CONFIG_KEYBOARD_GPIO=y    //允许 GPIO 引脚模拟键盘输入
CONFIG_GPIO_SYSFS=y       //提供 sysfs 接口用于从用户空间访问和控制 GPIO 引脚
```

这些配置反映了一个系统的 GPIO 支持情况，包括基础架构、访问接口和一些额外的功能。用户可以使用 menuconfig 进行配置，在 menuconfig 中搜索上面相关配置即可查看具体说明。

对应的 menuconfig 配置如下（篇幅有限，截图有所省略）：



图：GPIO 其他配置

5.5 设备树文件分析

打开 kernel/arch/arm/boot/dts/rk3502.dtsi，需要解释一下的是，谁然这儿设备树的名字是 rk3502.dtsi 但 rk3506 芯片也是引用的这个设备树，在此设备树文件下做的添加，gpio 这部分内容被定义在 rk3502.dtsi。在 soc 节点下面有 GPIO 控制器的具体描述。

```
1       gpio0: gpio@ff940000 {
2           compatible = "rockchip,gpio-bank";
3           reg = <0xff940000 0x200>;
4           interrupts = <GIC_SPI 0 IRQ_TYPE_LEVEL_HIGH>;
5           clocks = <&cru PCLK_GPIO0>, <&cru DBCLK_GPIO0>;
6
7           gpio-controller;
8           #gpio-cells = <2>;
9           gpio-ranges = <&pinctrl 0 0 32>;
10          interrupt-controller;
11          #interrupt-cells = <2>;
12      };
13
14      gpio1: gpio@ff870000 {
15          compatible = "rockchip,gpio-bank";
16          reg = <0xff870000 0x200>;
17          interrupts = <GIC_SPI 4 IRQ_TYPE_LEVEL_HIGH>;
18          clocks = <&cru PCLK_GPIO1>, <&cru DBCLK_GPIO1>;
19
```

```
20          gpio-controller;
21          #gpio-cells = <2>;
22          gpio-ranges = <&pinctrl 0 32 32>;
23          interrupt-controller;
24          #interrupt-cells = <2>;
25      };
26
27      gpio2: gpio@ff1c0000 {
28          compatible = "rockchip,gpio-bank";
29          reg = <0xff1c0000 0x200>;
30          interrupts = <GIC_SPI 8 IRQ_TYPE_LEVEL_HIGH>;
31          clocks = <&cru PCLK_GPIO2>, <&cru DBCLK_GPIO2>;
32
33          gpio-controller;
34          #gpio-cells = <2>;
35          gpio-ranges = <&pinctrl 0 64 32>;
36          interrupt-controller;
37          #interrupt-cells = <2>;
38      };
39
40      gpio3: gpio@ff1d0000 {
41          compatible = "rockchip,gpio-bank";
42          reg = <0xff1d0000 0x200>;
43          interrupts = <GIC_SPI 12 IRQ_TYPE_LEVEL_HIGH>;
44          clocks = <&cru PCLK_GPIO3>, <&cru DBCLK_GPIO3>;
45
46          gpio-controller;
47          #gpio-cells = <2>;
48          gpio-ranges = <&pinctrl 0 96 32>;
49          interrupt-controller;
50          #interrupt-cells = <2>;
51      };
52
53      gpio4: gpio@ff1e0000 {
54          compatible = "rockchip,gpio-bank";
55          reg = <0xff1e0000 0x200>;
56          interrupts = <GIC_SPI 16 IRQ_TYPE_LEVEL_HIGH>;
57          clocks = <&cru PCLK_GPIO4>, <&cru DBCLK_GPIO4>;
58
59          gpio-controller;
60          #gpio-cells = <2>;
61          gpio-ranges = <&pinctrl 0 128 32>;
62          interrupt-controller;
```

```
63          #interrupt-cells = <2>;
64      };
39      };
```

对上述配置进行简单说明：

- gpioX: gpio@ 0xXXX
  这部分是基础配置，由标签和地址组成，gpioX 是控制器的名称，RK3506 一共有 5 组 gpio，@ff940000 表明该控制器的基础内存地址。
- compatible = "rockchip,gpio-bank";
  这部分是设备兼容设置，该字段说明此 GPIO 控制器与特定硬件平台兼容。在 Linux 内核源码中搜索"rockchip,gpio-bank"，就可以找到对应的驱动文件，文件路径为：比如 drivers/gpio/gpio-rockchip.c。
- reg = <0xff940000 0x200>;
  这部分是寄存器配置，reg：这指定了控制器使用的内存寄存器区域。0xff940000 是寄存器的起始物理地址。0x200 表示寄存器块的大小。

这些配置共同确保了 GPIO 控制器在系统中的正确功能，以及其与其他系统组件（如中断控制器和时钟管理系统）的适当交互。

5.6 GPIO 接口配置实验

5.6.1 确认目标 GPIO 信息

这里以底板上 40Pin 座子中的 I2C1_SDA 引脚，即 GPIO1_B2 这个引脚，出厂系统下这个引脚被用作 I2C1_SDA，对应的设备树节点为：
kernel/arch/arm/boot/dts/rk3506-atk-dlrk3506b.dtsi，节点内容如下：

```
1   &i2c1 {
2       status = "okay";
3       pinctrl-names = "default";
4       pinctrl-0 = <&rm_io24_i2c1_scl
5                    &rm_io25_i2c1_sda>;
6   };
```

这里的追加的 rm_io25_i2c1_sda 节点就是将 GPIO1_B2 这个引脚配置成 I2C1_SDA。如果我们要将这个引脚复用成 GPIO，需要注释掉 i2c1 的这个节点，可以直接 status = "disable";

```
141 &i2c1 {
142         status = "disable";
143         pinctrl-names = "default";
144         pinctrl-0 = <&rm_io24_i2c1_scl
145                      &rm_io25_i2c1_sda>;
146 };
```

图：设备树 i2c1 节点

在提供的内核源码中，默认已使能 GPIO 系统，没有使用的 IO 默认就会作为 GPIO，不需要修改。

5.6.2 源码编译及固件更新

将上述 i2c1 节点关闭后，重新编译内核，烧录 boot.img 文件

```
./build.sh kernel
```

```
alientek@alientek-virtual-machine:~/ATK-DLRK3506$ ./build.sh kernel

############### Rockchip Linux SDK ###############

Manifest: rk3506_linux6.1_release_v1.1.0_20241128.xml

Log colors: message notice warning error fatal

Log saved at /home/alientek/ATK-DLRK3506/output/sessions/2025-03-25_15-33-27
Using last kernel version(6.1)

=========================================
          Final configs
=========================================
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS=/home/alientek/ATK-DLRK3506/device/rockchip/.chip/zboot.its
```

图：编译内核

编译完成后会生成 boot.img，烧录进板子即可。

```
  Hash value:   ba3fd9317cefe5b275e2dd29de0b7caf448736a893f4667c249c4bcecfc5bcca
 Default Configuration: 'conf'
 Configuration 0 (conf)
  Description:  unavailable
  Kernel:       kernel
  FDT:          fdt
+ ln -rsf kernel/zboot.img /home/alientek/ATK-DLRK3506/output/firmware/boot.img
Not Found io-domains in kernel/arch/arm/boot/dts/rk3506b-atk-dlrk3506b-nand-rwfs.dts
Running 10-kernel.sh - build_kernel succeeded.
```

图：编译完成

5.6.3 GPIO 输出和输入测试

更新完内核镜像重新启动，在系统/sys/class/gpio 下有如下内容：

```
root@ATK-DLRK3506:/sys/class/gpio# ls
export        gpiochip128  gpiochip64   unexport
gpiochip0     gpiochip32   gpiochip96
root@ATK-DLRK3506:/sys/class/gpio#
```

图：/sys/class/gpio 目录

gpiochipX：当前 SoC 所包含的 GPIO 控制器，RK3506 一共包含了 5 个 GPIO 控制器，分别为 GPIO1、GPIO2、GPIO3、GPIO4、GPIO5，在这里分别对应 gpiochip0、gpiochip32、gpiochip64、gpiochip96、gpiochip128 这 5 个文件夹，每一个 gpiochipX 文件夹用来管理一组 GPIO。

export：用于将指定编号的 GPIO 引脚导出。在使用 GPIO 引脚之前，需要将其导出，导出成功之后才能使用它。例如将前面 GPIO1_B2 对应引脚编号 42 导出

```
echo 42 > /sys/class/gpio/export
```

导出后可以看到在/sys/class/gpio 下会生成一个 gpio42 文件夹：

```
root@ATK-DLRK3506:/# ls /sys/class/gpio/
export        gpiochip0    gpiochip32   gpiochip96
gpio42        gpiochip128  gpiochip64   unexport
root@ATK-DLRK3506:/#
```

图：导出 gpio 节点

如果要删除这个 gpio 引脚可以向 unexport 下写入对应的引脚编号。

```
echo 42 > /sys/class/gpio/unexport
```

我们进到这个文件夹，可以看到如下内容：

```
root@ATK-DLRK3506:/sys/class/gpio/gpio42# ls
active_low  direction   power       uevent
device      edge        subsystem   value
root@ATK-DLRK3506:/sys/class/gpio/gpio42#
```

图：gpio42 文件

我们主要关心的文件是 active_low、direction、edge 以及 value 这四个属性文件，接下来分别介绍这四个属性文件的作用：

- direction ：配置 GPIO 引脚为输入或输出模式。该文件可读、可写，读表示查看 GPIO 当前是输入还是输出模式，写表示将 GPIO 配置为输入或输出模式；读取或写入操作可取的值为"out"（输出模 式）和"in"（输入模式），如下所示：

```
root@ATK-DLRK3506:/sys/class/gpio/gpio42# cat direction
in
root@ATK-DLRK3506:/sys/class/gpio/gpio42# echo out > direction
root@ATK-DLRK3506:/sys/class/gpio/gpio42# cat direction
out
root@ATK-DLRK3506:/sys/class/gpio/gpio42#
```

图：direction 属性文件

- value ：在 GPIO 配置为输出模式下，向 value 文件写入"0"控制 GPIO 引脚输出低电平，写入"1"则 控制 GPIO 引脚输出高电平。在输入模式下，读取 value 文件获取 GPIO 引脚当前的输入电平状态。
- active_low ：这个属性文件用于控制极性，可读可写，默认情况下为 0；
- edge ：控制中断的触发模式，该文件可读可写。在配置 GPIO 引脚的中断触发模式之前，需将其设 置为输入模式：非中断引脚：

  echo "none" > edge

  上升沿触发：echo "rising" > edge

  下降沿触发：echo "falling" > edge

  边沿触发：echo "both" > edge

GPIO 输出模式测试：

```
echo out > direction
echo 1 > value      //输出高电平
echo 0 > value      //输出低电平
```

```
root@ATK-DLRK3506:/sys/class/gpio/gpio42# echo out > direction
root@ATK-DLRK3506:/sys/class/gpio/gpio42# echo 1 > value
root@ATK-DLRK3506:/sys/class/gpio/gpio42# echo 0 > value
root@ATK-DLRK3506:/sys/class/gpio/gpio42#
```

图：gpio 输出测试

GPIO 输入模式测试：

```
echo in > direction
cat value
```

```
root@ATK-DLRK3506:/sys/class/gpio/gpio42# echo in > direction
root@ATK-DLRK3506:/sys/class/gpio/gpio42# cat value
1
root@ATK-DLRK3506:/sys/class/gpio/gpio42# cat value
0
root@ATK-DLRK3506:/sys/class/gpio/gpio42#
```

图：gpio 输入测试

value 值为 1 表示引脚输入高电平，value 值为 0 表示引脚输入低电平。

第六章 PWM 子系统

6.1 PWM 基础知识

PWM（脉宽调制，Pulse Width Modulation）是一种通过调节信号的占空比来控制模拟量的方法。它在嵌入式系统中有广泛的应用，例如控制电机速度、调节 LED 亮度、生成音频信号等。

1、PWM 的基本概念

PWM 通过改变信号的占空比（Duty Cycle）来控制功率传递。占空比是指信号在一个周期内高电平的时间占整个周期时间的比例。

- 占空比（Duty Cycle）：信号高电平持续时间与周期时间的比值。通常用百分比表示。
- 周期（Period）：信号重复的时间长度，通常以秒（s）或毫秒（ms）为单位。
- 频率（Frequency）：信号的重复频率，通常以赫兹（Hz）为单位。频率与周期的关系为：频率 = 1 / 周期。

2、PWM 的工作原理

PWM 信号在高电平和低电平之间快速切换，通过改变高电平的持续时间来控制平均输出电压。例如，50%占空比意味着高电平和低电平的时间相等，平均输出电压是电源电压的一半。

3、PWM 的应用

电机控制：通过改变占空比来调节电机的速度。

LED 调光：通过改变占空比来调节 LED 的亮度。

音频信号生成：通过 PWM 生成不同频率的音频信号。

电源调节：通过 PWM 控制开关电源的输出电压。

6.2 PWM 子系统简介

PWM（Pulse Width Modulation，脉宽调制）子系统是 Linux 内核中的一个重要组件，用于生成脉宽调制信号。脉宽调制信号是一种周期性的数字信号，其脉冲宽度可以通过调节占空比来控制。PWM 子系统通常与处理器上的定时器硬件模块相关联，用于生成高精度的脉宽调制信号。

6.2.1 主要特点

PWM 子系统在 Linux 内核中具有以下主要特点：

1、脉宽调制信号生成： PWM 子系统主要用于生成脉宽调制信号，这是一种周期性的数字信号，其脉冲宽度可以通过调节占空比来控制。这种信号通常用于控制电机速度、LED 亮度、音频合成等应用。

2、多通道支持： PWM 子系统通常支持多个独立的 PWM 通道，每个通道都可以独立配置和控制。这使得系统能够同时控制多个外部设备或执行多个不同的 PWM 任务。

3、高精度和稳定性： PWM 子系统通常与处理器上的定时器硬件模块相关联，能够提供高精度和稳定的脉宽调制信号。这种高精度的输出信号对于一些需要精确控制的应用非常重要。

4、灵活的配置和控制： PWM 子系统提供了一组 API 函数，允许设备驱动程序和用户空间应用程序对 PWM 通道进行灵活的配置和控制。这包括设置 PWM 频率、占空比、极性等参数，以及启用/禁用 PWM 输出。

5、设备树绑定支持： PWM 子系统通常通过设备树绑定来管理 PWM 通道的配置和控制。开发者可以在设备树中描述每个 PWM 通道的相关信息，如引脚连接、时钟源、频率范围等。

6、应用广泛： PWM 子系统在嵌入式系统中有着广泛的应用，包括电机控制、LED 亮度调节、音频合成、电源管理等领域。

总的来说，PWM 子系统提供了一种灵活、高精度的脉宽调制信号生成功能，能够满足各种不同应用的需求。它在嵌入式系统中扮演着重要的角色，为系统提供了精确的时间控制和信号调节能力。

6.2.2 组成部分

PWM 子系统在 Linux 内核中的组成部分通常包括以下几个方面：

1、PWM 核心代码： PWM 子系统的核心代码包括 PWM 框架及相关的数据结构、API 函数等。这些代码提供了对 PWM 硬件模块的抽象，允许设备驱动程序和用户空间应用程序通过统一的接口来配置和控制 PWM 通道。

2、设备驱动程序： 每个具体的 PWM 硬件模块通常都有相应的设备驱动程序，用于与硬件进行交互。这些驱动程序通过 PWM 框架提供的接口来注册 PWM 通道，并实现对 PWM 硬件模块的初始化、配置和控制等功能。

3、设备树绑定： 在 Linux 系统中，PWM 通道的配置信息通常会以设备树的形式进行描述。设备树中的节点包含了 PWM 通道的相关属性，如引脚连接、时钟源、频率范围等。PWM 子系统会根据设备树中的描述来初始化和配置 PWM 通道。

4、用户空间接口： PWM 子系统提供了一组 API 函数，允许用户空间应用程序对 PWM 通道进行配置和控制。这些接口通常包括打开/关闭 PWM 通道、设置 PWM 频率、占空比等操作。

5、时钟管理： PWM 通道通常需要一个时钟源来生成脉宽调制信号。因此，PWM 子系统可能会涉及到时钟管理模块，用于配置和控制时钟源的频率和分频等参数。

6、中断处理： 一些 PWM 硬件模块支持中断功能，用于在特定事件发生时通知处理器。因此，PWM 子系统可能会涉及到中断处理模块，用于注册和处理 PWM 通道相关的中断。

综上所述，PWM 子系统的组成部分包括核心代码、设备驱动程序、设备树绑定、用户空间接口、时钟管理和中断处理等模块，它们共同构成了对 PWM 硬件模块的抽象和管理，使得系统能够灵活地配置和控制 PWM 通道，满足不同应用的需求。

6.2.3 应用场景

PWM 子系统在嵌入式系统中有着广泛的应用场景，其中一些主要的应用包括：

1、电机控制： PWM 信号常用于控制电机的转速和方向。通过调节 PWM 信号的占空比，可以控制电机的平均电压和功率，从而实现精确的速度调节和方向控制。

2、LED 亮度调节： PWM 信号可以用于控制 LED 的亮度。通过调节 PWM 信号的占空比，可以控制 LED 的平均功率，从而实现 LED 的亮度调节。

3、音频合成： PWM 信号可以用于音频合成和数字音频处理。通过调节 PWM 信号的频率和占空比，可以产生不同频率和幅度的音频信号，从而实现音频合成和音频效果的调节。

4、电源管理： PWM 信号常用于电源管理和电压调节。通过调节 PWM 信号的占空比，可以控制电源输出的电压和电流，从而实现电源管理和电压调节。

5、通信接口： PWM 信号可以用于模拟和数字通信接口。在一些特定的通信协议中，如脉冲编码调制（PCM）、脉冲位置调制（PPM）等，PWM 信号可以作为通信信号进行数据传输。

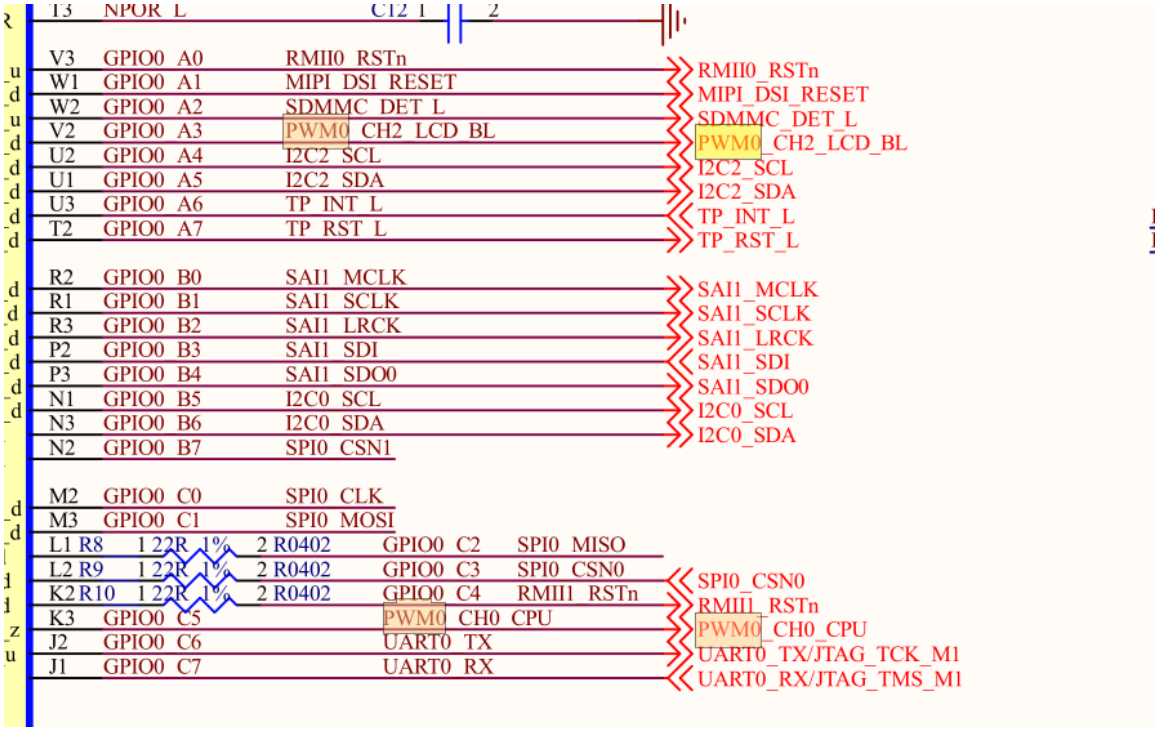
6、温度控制： PWM 信号可以用于温度控制和温度调节。通过调节 PWM 信号的占空比，可以控制加热元件的功率和温度，从而实现温度控制和温度调节。

综上所述，PWM 子系统在嵌入式系统中具有多种应用场景，包括电机控制、LED 亮度调节、音频合成、电源管理、通信接口和温度控制等领域，它为系统提供了一种灵活、高效的脉宽调制信号生成功能，能够满足各种不同应用的需求。

6.3 开发板 PWM 接口

6.3.1 pwm 节点介绍

打开开发板上开发板网盘 A 盘-基础资料\02、原理图\02、核心板原理图\ ATK-CLRK3506B V1.0.pdf 在核心板原理图上搜索可以看到如下内容：



图：PWM 使用管脚

如上图所示在出厂系统下 PWM0_CH0 给到 CPU 使用，PWM0_CH2 用于屏幕背光。在开发板的/sys/class/pwm 下能看到 pwmchip0 和 pwmchip1 两个文件夹，对应的就是这两路 pwm。

```
root@ATK-DLRK3506:/sys/class/pwm# ls -l
total 0
lrwxrwxrwx    1 root     root             0 Jan  1 08:00 pwmchip0 -> ../../devices/platform/ff930000.pwm/pwm/pwmchip0
lrwxrwxrwx    1 root     root             0 Jan  1 08:00 pwmchip1 -> ../../devices/platform/ff932000.pwm/pwm/pwmchip1
root@ATK-DLRK3506:/sys/class/pwm#
```

图：PWM 管脚节点

可以看到这两个 pwmchipX 都对应各自的映射地址，在 SDK 内核设备树中，有对应的节点信息 kernel/arch/arm/boot/dts/rk3502.dtsi

```
kernel-6.1 > arch > arm > boot > dts > rk3502.dtsi
/ {
    grf_pmu: syscon@ff910000 {
    };

    pwm0_4ch_0: pwm@ff930000 {
        compatible = "rockchip,rk3506-pwm", "rockchip,rk3576-pwm";
        reg = <0xff930000 0x200>;
        interrupts = <GIC_SPI 22 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&cru CLK_PWM0>, <&cru PCLK_PWM0>, <&cru CLK_OSC_PWM0>;
        clock-names = "pwm", "pclk", "osc";
        #pwm-cells = <3>;
        status = "disabled";
    };

    pwm0_4ch_1: pwm@ff931000 {
        compatible = "rockchip,rk3506-pwm", "rockchip,rk3576-pwm";
        reg = <0xff931000 0x200>;
        interrupts = <GIC_SPI 23 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&cru CLK_PWM0>, <&cru PCLK_PWM0>, <&cru CLK_OSC_PWM0>;
        clock-names = "pwm", "pclk", "osc";
        #pwm-cells = <3>;
        status = "disabled";
    };

    pwm0_4ch_2: pwm@ff932000 {
        compatible = "rockchip,rk3506-pwm", "rockchip,rk3576-pwm";
        reg = <0xff932000 0x200>;
        interrupts = <GIC_SPI 24 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&cru CLK_PWM0>, <&cru PCLK_PWM0>, <&cru CLK_OSC_PWM0>;
        clock-names = "pwm", "pclk", "osc";
        #pwm-cells = <3>;
        status = "disabled";
    };
```

图：PWM 设备树节点

RK3506 有提供 PWM0 和 PWM1 两个模块，其中 PWM0 有四个通道 CH0-3，PWM1 有八个通道 CH0-7；接着在这个设备树上搜索 pwm1 就能看到 pwm1 对应的 8 路通道。

```
kernel-6.1 > arch > arm > boot > dts > rk3502.dtsi
/ {
    spi1: spi@ff130000 {
    };

    pwm1_8ch_0: pwm@ff170000 {
        compatible = "rockchip,rk3506-pwm", "rockchip,rk3576-pwm";
        reg = <0xff170000 0x200>;
        interrupts = <GIC_SPI 26 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&cru CLK_PWM1>, <&cru PCLK_PWM1>;
        clock-names = "pwm", "pclk";
        #pwm-cells = <3>;
        status = "disabled";
    };

    pwm1_8ch_1: pwm@ff171000 {
        compatible = "rockchip,rk3506-pwm", "rockchip,rk3576-pwm";
        reg = <0xff171000 0x200>;
        interrupts = <GIC_SPI 27 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&cru CLK_PWM1>, <&cru PCLK_PWM1>;
        clock-names = "pwm", "pclk";
        #pwm-cells = <3>;
        status = "disabled";
    };
```

图：PWM1 设备树节点

6.3.2 pwm 引脚复用

前面一个章节我们了解了 RK3506 中 pwm 的配置节点，现在我们以 GPIO1_D2 引脚为例，教大家如何复用为 pwm。

因为在出厂系统下 pwm0 中 ch0 和 ch2 通道已经被使用，所以将 GPIO1_D2 复用到 ch1 通道。在 kernel/arch/arm/boot/dts/rk3506-atk-dlrk3506b.dtsi 下追加如下内容：

```
&pwm0_4ch_1 {
        pinctrl-names = "active";
        pinctrl-0 = <&rm_io30_pwm0_ch1>;
    status = "okay";
};
```

其中&rm_io30_pwm0_ch1 是追加 GPIO1_D2 引脚的一个复用（RM_IO）即 Rochchip Matrix IO 这个节点定义在 kernel/arch/arm/boot/dts/rk3506-pinctrl-rmio.dtsi 下：

```
kernel-6.1 > arch > arm > boot > dts > rk3506-pinctrl-rmio.dtsi
    10  &pinctrl {
14801       rm_io30 {
            };
14952       /omit-if-no-ref/
14953       rm_io30_pwm0_ch1: rm-io30-pwm0-ch1 {
14954           rockchip,pins =
14955               <1 RK_PD2 46 &pcfg_pull_none_drv_level_1>;
14956       };
```

图：rm_io 设备树节点

在这个节点下有<1 RK_PD2 46 &pcfg_pull_none_drv_level_1>;其中 1 RK_PD2 表示 GPIO1_D2。46 表示复用成 pwm0_ch1。&pcfg_pull_none_drv_level_1 为引脚配置。

配置好设备树后，编译内核烧录内核到开发板。

6.4 PWM 测试方法

6.4.1 使用 sysfs 方式操控 PWM

PWM 通过 sysfs 方式进行操控，进入到/sys/class/pwm 目录下，如下所示：

```
root@ATK-DLRK3506:/sys/class/pwm# ls -l
total 0
lrwxrwxrwx    1 root     root           0 Jan  1 08:00 pwmchip0 -> ../../devices/platform/ff930000.pwm/pwm/pwmchip0
lrwxrwxrwx    1 root     root           0 Jan  1 08:00 pwmchip1 -> ../../devices/platform/ff931000.pwm/pwm/pwmchip1
lrwxrwxrwx    1 root     root           0 Jan  1 08:00 pwmchip2 -> ../../devices/platform/ff932000.pwm/pwm/pwmchip2
root@ATK-DLRK3506:/sys/class/pwm#
```

图：/sys/class/pwm 目录下的内容

可以看到 pwmchip1 就是我们刚才设备树上配置的 GPIO1_D2 引脚复用出的 pwm 节点，我们进入到 pwmchip1 目录下：

```
root@ATK-DLRK3506:/sys/class/pwm# cd pwmchip1
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1# ls
device      export      npwm        power       subsystem  uevent      unexport
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1#
```

图：pwmchip1 目录下的内容

在这个目录下我们重点关注的是 export、npwm 以及 unexport 这三个属性文件，下面一一进行介绍：

- **npwm**：这是一个只读属性，读取该文件可以得知该 PWM 控制器下共有几路 PWM 输出，如下所示：

```
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1# cat npwm
1
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1#
```

图：读取 npwm 属性文件

所以由此可知，该 PWM 控制器一共有 1 路 PWM 输出。

- **export：**与 GPIO 控制一样，在使用 PWM 之前，也需要将其导出，通过 export 属性进行导出，以下所示：

```
echo 0 > export
```

```
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1# echo 0 > export
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1# ls
device      export      npwm        power       pwm0        subsystem   uevent      unexport
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1#
```

图：导出 PWM 通道

- **unexport：**将导出的 PWM 通道删除。当使用完 PWM 之后，我们需要将导出的 PWM 通道删除，譬如：

```
echo 0 > unexport
```

写入到 unexport 文件中的编号与写入到 export 文件中的编号是相对应的；需要注意的是，export 文件和 unexport 文件都是只写的、没有读权限。

如何控制 PWM

上例中通过 export 导出之后，便会生成 pwm0 这个目录，我们进入到该目录下看看：

```
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1# cd pwm0/
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1/pwm0# ls
capture     duty_cycle  enable      period      polarity    power       uevent
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1/pwm0#
```

图：pwm0 目录下的内容

该目录下也有一些属性文件，我们重点关注 duty_cycle、enable、period 以及 polarity 这四个属性文件，接下来一一进行介绍。

- **polarity：**用于设置极性，可读可写，可写入的值如下：

"normal"：普通；

"inversed"：反转；

```
echo normal > polarity          #默认极性
echo inversed > polarity        #极性反转
```

很多 SoC 的 PWM 外设其硬件上并不支持极性配置，所以对应的驱动程序中并未实现这个接口，应用层自然也就无法通过 polarity 属性文件对 PWM 极性进行配置。

- **period：**用于配置 PWM 周期，可读可写；写入一个字符串数字值，以 ns（纳秒）为单位，譬如配置 PWM 周期为 10us（微秒）：

```
echo 10000 > period          #PWM 周期设置为 10us（10 * 1000ns）
```

- **duty_cycle：**用于配置 PWM 的占空比，可读可写；写入一个字符串数字值，同样也是以 ns 为单位，譬如：

```
echo 5000 > duty_cycle       #PWM 占空比设置为 5us
```

- **enable：**可读可写，写入"0"表示禁止 PWM；写入"1"表示使能 PWM。读取该文件获取 PWM 当前是禁止还是使能状态。

```
echo 0 > enable              #禁止 PWM 输出
echo 1 > enable              #使能 PWM 输出
```

通常配置好 PWM 之后，再使能 PWM。

6.4.2 目标管脚 PWM 测试

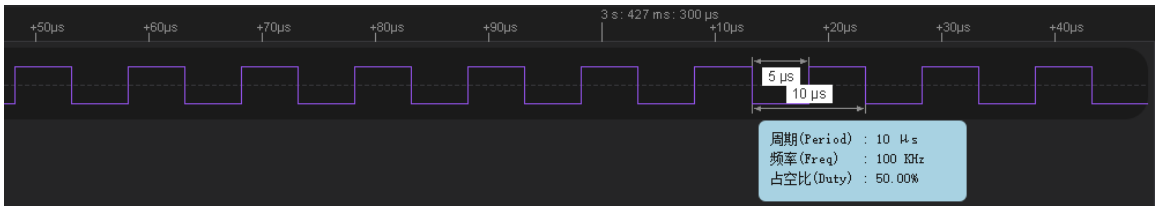
将示波器或者逻辑分析仪接到开发板的 GPIO1_D2 管脚和 GND，后续将使用示波器或者逻辑分析仪查看 PWM0 波形。

配置 PWM0 为默认极性、周期设置为 10us、占空比设置为 5us，最后使能 PWM。

```
echo 10000 > period
echo 5000 > duty_cycle
echo normal > polarity
echo 1 > enable
```

```
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1/pwm0# echo 10000 > period
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1/pwm0# echo 5000 > duty_cycle
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1/pwm0# echo normal > polarity
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1/pwm0# echo 1 > enable
root@ATK-DLRK3506:/sys/class/pwm/pwmchip1/pwm0#
```

图：PWM0 配置



图：实验现象

第七章 LED 子系统

7.1 LED 基础知识

1、LED 的基本概念

LED（发光二极管，Light Emitting Diode）是一种半导体器件，当电流通过它时能够发光。与传统光源相比，LED 具有高效、节能、寿命长等优点，被广泛应用于各种照明和显示设备中。

2、LED 的工作原理

LED 的发光原理基于半导体的 PN 结。当电流从 P 型半导体流向 N 型半导体时，电子和空穴在 PN 结处复合，释放出光子，从而发光。

3、LED 的特性

正向电压（Forward Voltage）：LED 工作时的电压降，通常为 2V 至 3.6V 之间，具体值取决于材料和颜色。

正向电流（Forward Current）：LED 工作时的电流，通常在 10mA 至 30mA 之间，具体值取决于 LED 的规格。

亮度（Luminous Intensity）：LED 的发光强度，通常以毫烛光（mcd）或流明（lm）为单位。

颜色（Color）：LED 发光的颜色，取决于半导体材料的能隙。常见颜色有红、绿、蓝、黄、白等。

4、LED 的应用

照明：如家庭照明、街道照明、车灯等。

显示：如 LED 显示屏、指示灯等。

信号指示：如电子设备的电源指示灯、状态指示灯等。

背光：如液晶显示屏的背光源。

5、LED 的电路设计

在使用 LED 时，需要通过限流电阻来控制电流，避免过流损坏 LED。

6、在嵌入式系统中控制 LED

在嵌入式 Linux 中，可以通过控制 GPIO 来操作 LED。

7.2 LED 子系统简介

嵌入式 Linux 中的 LED 子系统是内核的一部分，用于控制和管理各种 LED 设备。它提供了一个抽象层，使用户和应用程序可以方便地操作 LED，而不需要关心底层硬件的具体实现。

7.2.1 主要特点

嵌入式 Linux 的 LED 子系统具有一系列特点，使其在嵌入式系统中得以广泛应用。以下是 LED 子系统的主要特点：

1、统一抽象层

LED 子系统为各种 LED 设备提供了一个统一的抽象层，隐藏了底层硬件差异，简化了上层应用程序的开发。

2、设备树支持

LED 子系统支持通过设备树（Device Tree）进行配置，这使得在不同硬件平台上配置 LED 设备更加方便和直观。设备树定义了 LED 设备的属性及其关联的 GPIO 引脚，允许在内核启动时自动配置 LED 设备。

3、标准化接口

通过 sysfs 文件系统，LED 子系统向用户空间提供了标准化的接口。用户和应用程序可以通过这些接口方便地控制 LED 的状态和属性。

4、多种触发模式

LED 子系统支持多种触发模式（trigger），可以根据系统状态自动控制 LED 的行为。

7.2.2 组成部分

嵌入式 Linux 的 LED 子系统由多个组成部分构成，这些部分协同工作以提供对 LED 设备的高效管理和控制。以下是 LED 子系统的主要组成部分：

1、 LED 类驱动（LED Class Driver）

LED 类驱动是 LED 子系统的核心组件，定义了 LED 设备的通用接口和基本操作。它通过 sysfs 接口向用户空间暴露控制 LED 设备的文件，并提供对这些文件进行操作的方法。

- 文件位置：drivers/leds/led-class.c
- 功能：
  - 创建和管理 LED 设备节点
  - 提供统一的接口以便 LED 设备驱动程序注册和使用

2. LED 设备驱动（LED Device Driver）

LED 设备驱动是与具体硬件相关的驱动程序，实现对特定 LED 硬件的控制。它实现了 LED 类驱动定义的接口，并将具体的硬件操作封装起来。

- 文件位置：drivers/leds/
- 功能：
  - 初始化和配置 LED 硬件
  - 实现对 LED 设备的控制，如开关、亮度调节等

3. 触发器（LED Triggers）

触发器定义了 LED 的行为模式，根据系统事件或状态自动控制 LED。常见的触发器包括定时器触发、心跳触发等。触发器通过内核中的钩子函数与系统其他部分互动。

- 文件位置：drivers/leds/led-triggers.c
- 功能：
  - 定义触发器类型和行为
  - 实现触发器的注册和注销
  - 处理系统事件并控制 LED

4. sysfs 接口

sysfs 是 Linux 内核提供的虚拟文件系统，通过它可以从用户空间访问和控制 LED 设备。LED 类驱动通过 sysfs 接口向用户空间暴露文件，这些文件用于控制 LED 的亮度、状态和触发器等。

- 典型路径：/sys/class/leds/<led_name>/
- 主要文件：
  - brightness：控制 LED 亮度
  - trigger：设置 LED 的触发模式

◆ delay_on 和 delay_off：在定时器触发模式下设置 LED 的闪烁时间

5. 设备树（Device Tree）

设备树用于描述硬件信息，LED 设备的配置通常通过设备树进行定义。设备树节点中包括了 LED 设备的属性、关联的 GPIO 引脚等信息。

文件位置：设备树源文件（DTS），这里以 kernel/arch/arm/boot/dts/rk3506-atk-dlrk3506b.dtsi 为例：

```
leds: leds {
    compatible = "gpio-leds";
    work_led: work {
        gpios = <&gpio0 RK_PD0 GPIO_ACTIVE_HIGH>;
        linux,default-trigger = "heartbeat";
        status = "okay";
    };
};
```

6、内核配置选项

为了启用 LED 子系统及其相关功能，需要在内核配置中启用相应的选项。可以通过 menuconfig 工具进行配置。

位置：

```
Device Drivers  --->
  [*] LED Support  --->
    <*> LED Class Support
    <*> LED Support for GPIO connected LEDs
```

7、用户空间工具和库

用户空间工具和库可以简化对 LED 设备的控制，提供更高层次的抽象。常见的用户空间工具包括 shell 脚本和应用程序。ATK-DLRK3506B 开发板上配有 1 个心跳 LED，1 个电源指示灯。

示例：(控制心跳 LED)

```
cat /sys/class/leds/work/trigger            //查看心跳灯的当前触发方式及支持的触发方式
echo none > /sys/class/leds/work/trigger        //改变心跳灯的触发方式，设置为none
echo 1 > /sys/class/leds/work/brightness        //点亮心跳灯
echo 0 > /sys/class/leds/work/brightness        //熄灭心跳灯
echo heartbeat > /sys/class/leds/work/trigger   //设置心跳灯为 heartbeat
```

7.2.3 应用场景

嵌入式 Linux 的 LED 子系统在各种应用场景中具有广泛的应用。以下是一些常见的应用场景：

1、系统状态指示

电源状态指示、网络状态指示、工作状态指示，例如当设备正常工作时，指示灯亮起或闪烁；当设备出现故障时，指示灯以不同的频率闪烁或变成其他颜色。

2、用户交互反馈

按键反馈：在用户按下按钮时，通过点亮或闪烁 LED 来提供视觉反馈，确认用户操作已被接收。

用户提示：使用 LED 提示用户某些操作需要进行，例如需要插入存储设备、充电完成等。

3、调试和诊断

错误指示：在设备故障时，使用 LED 来指示具体的错误类型或状态。例如，设备可以通过不同的闪烁模式或颜色来指示内存错误、网络错误等。

状态监控：在设备开发和调试过程中，使用 LED 指示设备的当前状态，帮助开发人员实时监控和诊断设备状态。

4、安全和报警

安全报警：在检测到安全问题时，如门禁系统检测到非法入侵，使用 LED 闪烁或变色来发出警报。

环境监控：在环境监控系统中，LED 可以用来指示环境参数是否正常，例如温度过高或烟雾报警。

5、用户界面

简单用户界面：在一些简易设备中，使用 LED 作为用户界面的一部分。例如，通过不同颜色和闪烁模式的组合，向用户传递设备的状态信息。

6、设备功能指示

存储状态指示：在存储设备（如路由器或网络存储设备）中，使用 LED 指示存储设备的读写状态，帮助用户了解存储设备的工作状态。

无线连接指示：在无线设备（如 Wi-Fi 路由器）中，使用 LED 指示无线连接状态，包括连接成功、信号强度等。

7、工业控制

生产线状态指示：在工业生产线中，使用 LED 指示生产线的运行状态、故障状态等，帮助工人实时了解生产情况。

设备状态监控：在工业设备中，使用 LED 指示设备的运行状态和故障状态，方便维护人员进行监控和诊断。

实际应用示例

示例 1：网络路由器

在网络路由器中，LED 子系统可以用来指示以下状态：

- 电源指示灯：指示路由器是否通电。
- 网络连接指示灯：指示 WAN 和 LAN 端口的连接状态。
- 无线信号指示灯：指示 Wi-Fi 状态和信号强度。
- 活动指示灯：指示网络数据传输活动。

示例 2：智能家居设备

在智能家居设备中，LED 子系统可以用来指示以下状态：

- 设备状态指示灯：指示设备是否在线、是否正常工作。
- 报警指示灯：指示烟雾、温度等安全报警状态。
- 用户操作反馈：指示用户操作是否成功，如按键反馈等。

示例 3：工厂生产线

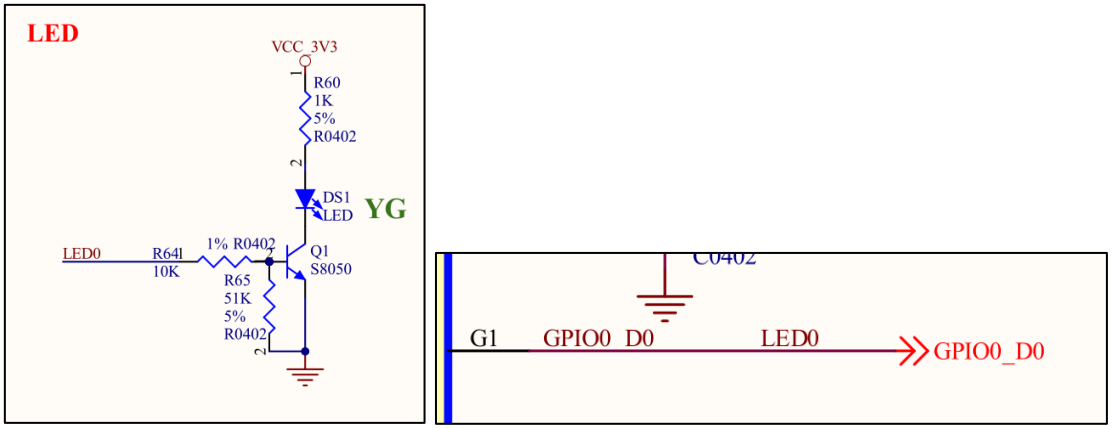
在工厂生产线中，LED 子系统可以用来指示以下状态：

- 生产状态指示灯：指示生产线是否正常运行、是否停机。
- 故障指示灯：指示生产线或设备是否出现故障。
- 操作提示灯：提示工人进行特定操作，如更换材料等。

7.3 开发板 LED 设备

7.3.1 原理图分析

开发板上配有 1 个心跳 LED。我们就以心跳 LED 来做实验，打开开发板上开发板网盘 A 盘-基础资料\02、原理图\01、底板原理图\ ATK-DLRK3506B V1.0.pdf，搜索 LED0，可以看到 GPIO0_D0 默认用做 LED0。

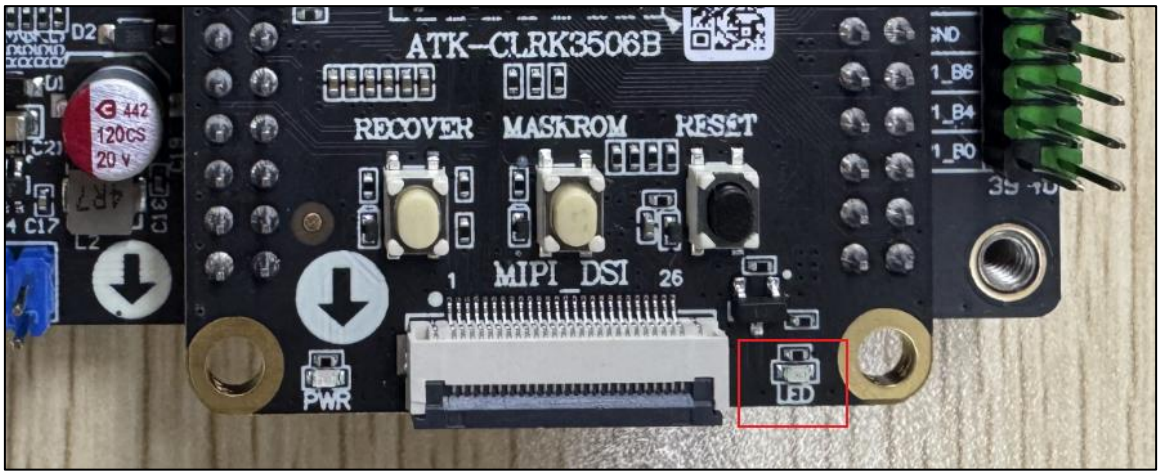


图：开发板 LED 原理图

此图中 LED 的连接方式是通过 GPIO 引脚控制三极管来点亮 LED。因为当 GPIO 引脚输出高电平时，三极管导点亮 LED。

7.3.2 实物硬件

开发板上的 LED 如下图所示，丝印为 LED，图中是没启动系统的状态，启动出厂系统默认作为绿色心跳灯闪烁：



图：开发板 LED 实物

7.4 设备树文件分析

打开内核源码 kernel/arch/arm/boot/dts/rk3506-atk-dlrk3506b.dtsi 文件，找到 led 子节点。

```
1   leds: leds {
2       compatible = "gpio-leds";
3       work_led: work {
4           gpios = <&gpio0 RK_PD0 GPIO_ACTIVE_HIGH>;
5           linux,default-trigger = "heartbeat";
6           status = "okay";
7       };
8   };
```

第 1 行，节点名称：leds，表示 LED 设备的配置。

第 2 行，compatible："gpio-leds"，表示这个节点兼容 GPIO 控制的 LED 设备驱动。对应的驱动源码为 drivers/leds/leds-gpio.c

第 4 行，gpios = <&gpio0 RK_PD0 GPIO_ACTIVE_HIGH>;，指定了控制 LED 的 GPIO 引脚：

- &gpio0：表示 GPIO 控制器。
- RK_PD0：表示 GPIO0 控制器的第 24 个引脚。
- GPIO_ACTIVE_HIGH：表示 LED 在 GPIO 引脚为高电平时点亮。

第 5 行，linux,default-trigger："heartbeat"，指定了 LED 的默认触发器为"heartbeat"。这个触发器会使 LED 以类似心跳的模式闪烁，通常用于指示系统运行状态。

7.5 LED 测试方法

7.5.1 使用 sysfs 方式操控 LED

LED 通过 sysfs 方式进行操控，进入到/sys/class/leds/work 目录下，如下所示：

```
root@ATK-DLRK3506:/sys/class/leds/work# ls
brightness      invert          power           trigger
device          max_brightness  subsystem       uevent
root@ATK-DLRK3506:/sys/class/leds/work#
```

图：/sys/class/leds/work 文件夹

- **brightness：**翻译过来就是亮度的意思，该属性文件可读可写；所以这个属性文件是用于设置 LED 的亮度等级或者获取当前 LED 的亮度等级，譬如 brightness 等于 0 表示 LED 灭，brightness 为正整数表示 LED 亮，其值越大、LED 越亮；对于 PWM 控制的 LED 来说，这通常是适用的，因为它存在亮度等级的问题，不同的亮度等级对应不同的占空比，自然 LED 的亮度也是不同的；但对于 GPIO 控制（控制 GPIO 输出高低电平）的 LED 来说，通常不存在亮度等级这样的说法，只有 LED 亮（brightness 等于 0）和 LED 灭（brightness 为非 0 值的正整数）两种状态，ATK-DLRK3506B 开发板上的这颗 LED 就是如此，所以自然就不存在亮度等级一说，只有亮和灭两种亮度等级。
- **max_brightness：**该属性文件只能被读取，不能写，用于获取 LED 设备的最大亮度等级。
- **trigger：**触发模式，该属性文件可读可写，读表示获取 LED 当前的触发模式，写表示设置 LED 的触发模式。不同的触发模式其触发条件不同，LED 设备会根据不同的

触发条件自动控制其亮、灭状态，通过 cat 命令查看该属性文件，可获取 LED 支持的所有触发模式以及 LED 当前被设置的触发模式：

```
root@ATK-DLRK3506:/sys/class/leds/work# cat trigger
none rfkill-any rfkill-none [heartbeat] mmc0 rfkill0 rfkill1
root@ATK-DLRK3506:/sys/class/leds/work#
```

图：LED 支持的所有触发模式

方括号（[heartbeat]）括起来的表示当前 LED 对应的触发模式。

通常系统启动之后，会将板子上的一颗 LED 设置为 heartbeat 触发模式，将其作为系统正常运行的指示灯，譬如我们的 ATK-DLRK3506B 开发板出厂系统启动之后，底板上的 LED 就会处于心跳呼吸模式，这个大家自己观察便可知道。

通过上面的介绍，已经知道如何去控制开发板底板上的用户 LED 了，譬如通过 echo 命令进行控制：

```
echo none > trigger        //将 LED 触发模式设置为 none
echo 1 > brightness        //点亮 LED   echo 0 > brightness//熄灭 LED
```

第八章 I2C 子系统

8.1 I2C 基础知识

I2C（Inter-Integrated Circuit）是一种常用的串行通信协议，用于在集成电路之间进行短距离通信。I2C 由飞利浦公司（现为恩智浦半导体）在 1982 年开发，主要用于主从模式的多主机控制系统。

基本概念

- **总线结构**

I2C 是一种双线制总线，包含两条信号线：SDA（数据线）和 SCL（时钟线）。
每个设备都通过这两条线连接，可以通过唯一的地址进行区分。

- **主从关系**

主设备：控制通信的设备，生成时钟信号并启动数据传输。
从设备：被动响应主设备的请求，不生成时钟信号。

- **信号线**

SDA（Serial Data Line）：
传输数据的双向线。
开漏结构，通常需要上拉电阻。
SCL（Serial Clock Line）：
传输时钟信号的单向线。
由主设备生成时钟信号，也需要上拉电阻。

- 数据传输

数据帧格式：
起始条件：主设备在 SCL 高电平期间将 SDA 线从高拉低。
地址帧：包含 7 位或 10 位从设备地址和 1 位读/写指令位。
确认位（ACK/NACK）：接收方每接收一个字节后，会在第 9 位时钟周期上响应确认（ACK）或非确认（NACK）。
数据帧：传输的数据字节，发送或接收。
停止条件：主设备在 SCL 高电平期间将 SDA 线从低拉高。

- 读写操作

写操作：主设备发送从设备地址，并在最后一位写指令为 0。
读操作：主设备发送从设备地址，并在最后一位写指令为 1。

特点与优势

简单性：I2C 总线使用两条线，实现了硬件上的简化。
灵活性：支持多主多从配置，设备可以动态加入和离开总线。
低成本：由于线数少，系统布线成本和 PCB 面积减少。
可靠性：I2C 有较强的抗干扰能力，适用于各种嵌入式系统。

限制与注意事项

速度限制：标准模式为 100kbps，高速模式为 400kbps，更高的速率如 Fast-mode Plus（1Mbps）和 High-speed mode（3.4Mbps）需要特殊处理。

线长度：由于 I2C 是并联电阻器终端，线长度有限，通常不超过 1 米。

设备数量：从设备的数量受限于 7 位或 10 位地址空间，理论上最多支持 127 个设备。

常用 I2C 接口设备

EEPROM：用于数据存储，非易失性存储器。

传感器：温度、湿度、压力等传感器。

实时时钟（RTC）：提供精确的时间和日期。

LCD 显示器：用于显示信息。

8.2 I2C 子系统简介

I2C（Inter-Integrated Circuit）是一种广泛使用的串行通信总线标准，特别适用于嵌入式系统中的芯片间通信。I2C 子系统在 Linux 内核中有一个全面的实现，提供了对 I2C 设备和驱动的支持。

8.3 主要特点

1、模块化设计

I2C 子系统采用模块化设计，将 I2C 总线适配器、设备和驱动程序分离。这样，开发人员可以独立地开发和调试每个组件，提高了系统的灵活性和可维护性。

2、设备树支持

设备树 (Device Tree) 提供了一种描述硬件的方法，使内核能够自动识别和配置 I2C 设备。设备树节点用于定义 I2C 总线和从设备的属性，使得系统在启动时可以动态配置 I2C 硬件。

3、丰富的 API 支持

I2C 子系统提供了一套丰富的 API，用于实现 I2C 总线上的数据传输。这些 API 包括简单的读写操作和复杂的 SMBus 协议支持，使开发人员能够轻松地与 I2C 设备通信。

4、用户空间工具

Linux 提供了一组用户空间工具，用于检测、访问和调试 I2C 设备。这些工具简化了开发和调试过程，使开发人员可以快速验证 I2C 总线和设备的功能。

这些特点展示了嵌入式 Linux I2C 子系统的灵活性、易用性和强大的功能支持，有助于开发人员在各种嵌入式应用中高效地实现 I2C 通信。

8.4 组成部分

嵌入式 Linux 的 I2C 子系统由多个关键组成部分构成，这些部分协同工作以提供对 I2C 总线和设备的支持。以下是 I2C 子系统的主要组成部分：

1. I2C 适配器 (Adapter)

I2C 适配器代表了 I2C 主控制器硬件，例如，处理器内置的 I2C 控制器或外部的 I2C 控制器模块。适配器驱动程序负责与这些硬件进行交互，并提供对 I2C 总线的访问接口。

i2c_adapter 结构体是内核中表示适配器的主要数据结构。

```
1 struct i2c_adapter {
2     struct module *owner;
```

```
    unsigned int class;
    const struct i2c_algorithm *algo;
    // 其他成员变量
};
```

2、I2C 客户端 (Client)

I2C 客户端表示连接到 I2C 总线上的从设备。每个从设备都有一个唯一的 I2C 地址，客户端驱动程序使用该地址与从设备通信。i2c_client 结构体表示一个 I2C 从设备。

```
struct i2c_client {
    struct i2c_adapter *adapter;
    struct device dev;
    u16 addr;
    // 其他成员变量
};
```

3、I2C 驱动程序 (Driver)

I2C 驱动程序负责管理和控制 I2C 客户端设备。驱动程序实现了与从设备的通信逻辑，例如数据读写、初始化和配置。i2c_driver 结构体表示一个 I2C 设备驱动程序。

```
struct i2c_driver {
    int (*probe)(struct i2c_client *client,
                 const struct i2c_device_id *id);
    int (*remove)(struct i2c_client *client);
    // 其他成员变量
};
```

4、I2C 消息 (Message)

I2C 消息用于表示 I2C 总线上的一次数据传输操作。i2c_msg 结构体用于封装一次读或写操作的信息，包括目标地址、数据缓冲区和传输长度。

```
struct i2c_msg {
    __u16 addr;
    __u16 flags;
    __u16 len;
    __u8 *buf;
};
```

5、I2C 算法 (Algorithm)

I2C 算法定义了适配器使用的操作方法集。每个 I2C 适配器驱动程序必须实现一个 I2C 算法，定义如何执行 I2C 操作，如启动、停止、读写数据等。i2c_algorithm 结构体表示这个接口。

```
struct i2c_algorithm {
int (*master_xfer)(struct i2c_adapter *adap,
                   struct i2c_msg *msgs, int num);
    // 其他成员变量
};
```

6、设备树支持 (Device Tree Support)

设备树是一种用于描述硬件的文件格式，特别适用于 ARM 等嵌入式平台。I2C 设备通常通过设备树描述，使内核能够自动配置和管理这些设备。

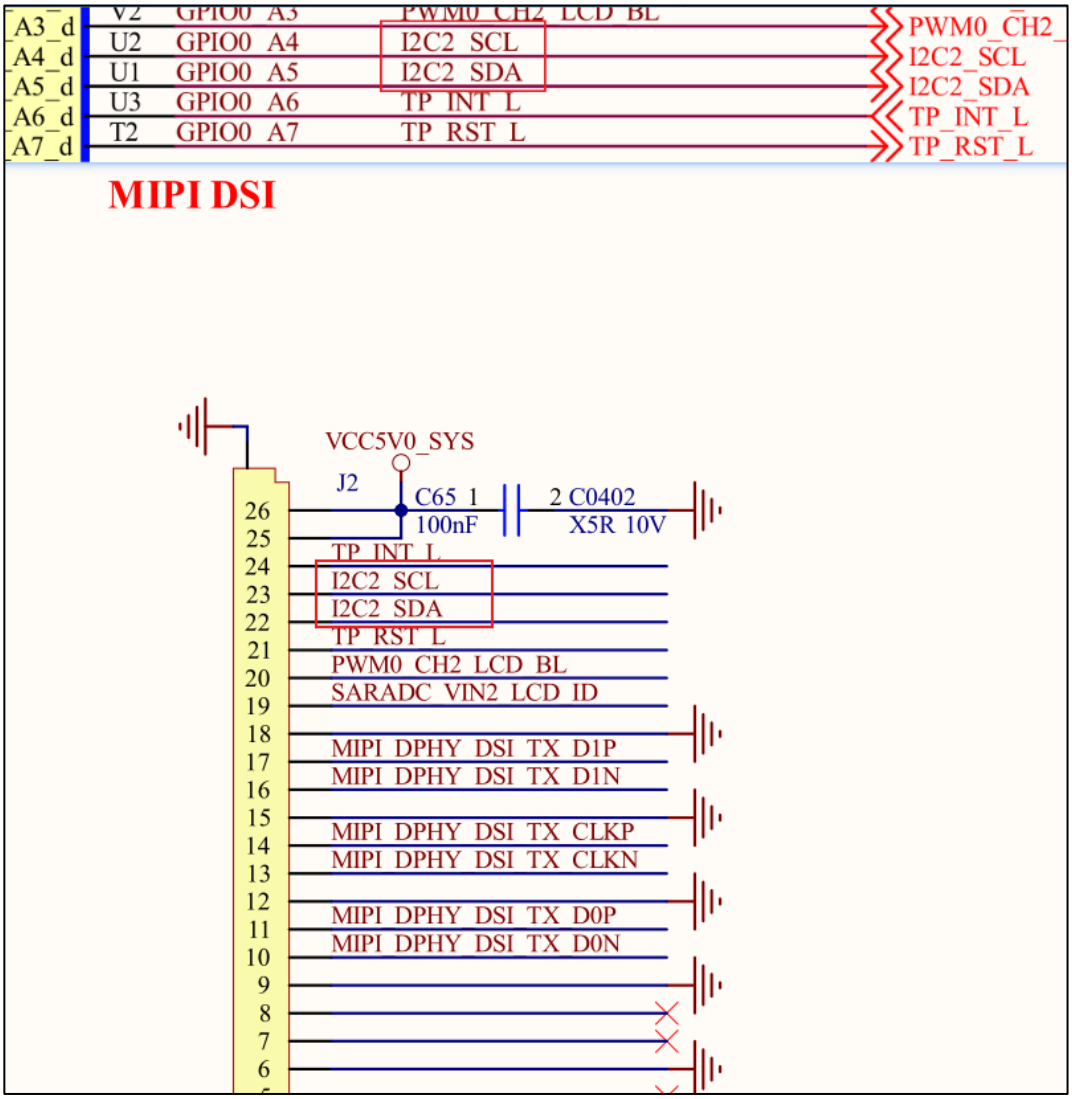
示例设备树片段：

```
i2c0: i2c@ff040000 {
    compatible = "rockchip,rk3506-i2c", "rockchip,rk3399-i2c";
    reg = <0xff040000 0x1000>;
    interrupts = <GIC_SPI 40 IRQ_TYPE_LEVEL_HIGH>;
    #address-cells = <1>;
    #size-cells = <0>;
    clocks = <&cru CLK_I2C0>, <&cru PCLK_I2C0>;
    clock-names = "i2c", "pclk";
    status = "disabled";
};
```

8.5 开发板 I2C 设备

8.5.1 原理图分析

在开发板网盘 A 盘-基础资料\02、原理图\02、核心板原理图\ ATK-CLRK3506B V1.0.pdf 搜索 I2C。可以看到 I2C 相关电路如下。



图：开发板 I2C 电路

以上是开发板 I2C2 电路节选，挂载了屏幕触摸芯片；本章节试验以正点原子的 mipi 5.5 寸屏(720*1280 分辨率)所使用的 GT 系列触摸芯片为例。

8.5.2 实物硬件



8.6 内核 I2C 子系统使能

内核源码配置文件 arch/arm/configs/rk3506_defconfig 和 arch/arm/configs/rk3506-display.config 中，I2C 相关配置如下(略有删减，只列举重点部分)：

```
CONFIG_I2C_CHARDEV=y                    //启用 I2C 字符设备支持
CONFIG_I2C_MUX=y                //启用 I2C 多路复用器支持，用于管理多个 I2C 总线
CONFIG_I2C_GPIO=y               //启用通过 GPIO 模拟 I2C 总线的支持
```

8.7 设备树文件分析

打开 arch/arm/boot/dts/rk3502.dtsi，其中关于 I2C2 的描述如下：

```
 i2c2: i2c@ff060000 {
        compatible = "rockchip,rk3506-i2c", "rockchip,rk3399-i2c";
        reg = <0xff060000 0x1000>;
        interrupts = <GIC_SPI 42 IRQ_TYPE_LEVEL_HIGH>;
        #address-cells = <1>;
        #size-cells = <0>;
```

```
        clocks = <&cru CLK_I2C2>, <&cru PCLK_I2C2>;
        clock-names = "i2c", "pclk";
        status = "disabled";
};
```

第 1 行，i2c2: i2c@ff060000 节点：表示一个 I2C2 控制器。

第 2 行，compatible = "rockchip,rk3506-i2c", "rockchip,rk3399-i2c";：这是一个属性，表示该设备与 Rk 的 I2C 兼容。

打开内核源码 arch/arm/boot/dts/rk3506-atk-dlrk3506b.dtsi 文件，i2c2 相关配置如下：

```
&i2c2 {
        dsi1_touch: gt9xx@14 {
                compatible = "goodix,gt9xx";
                reg = <0x14>;
                pinctrl-names = "default";
                pinctrl-0 = <&touch1_gpio>;
                touch-gpio = <&gpio0 RK_PA6 IRQ_TYPE_LEVEL_LOW>;
                reset-gpio = <&gpio0 RK_PA7 GPIO_ACTIVE_HIGH>;
                max-x = <720>;
                max-y = <1280>;
                tp-size = <911>;
                tp-supply = <&vcc_sys>;
                wakeup-source;
                goodix-ts-name = "dsi1_ts_gt9xx";
                status = "okay";
        };
      /delete-node/ gt1x;
};
```

上面给出的是一个设备树 rk3506-atk-dlrk3506b.dtsi 文件的片段，用于配置 RK3506 处理器的 I2C2 控制器及其 mipi 5.5 寸屏 720P 触摸芯片配置。下面是对该设备树片段简单解析：

&i2c2：引用之前定义的 i2c2 节点，表示配置该 I2C 控制器。

status = "ok"：表明该 I2C 总线（及其上的设备）的状态是“ok”，即正常工作。

dsi1_touch: gt9xx@14：定义了一个名为 dsi1_touch 的设备节点，gt9xx 是设备型号，@14 表示该设备在 I2C 总线上的地址是 0x14。

compatible = "goodix,gt9xx";：这是一个属性，指定了设备的兼容性字符串。触摸驱动文件为 drivers/input/touchscreen/gt9xx/gt9xx.c。

touch-gpio = <&gpio0 RK_PA6 IRQ_TYPE_LEVEL_LOW>：指定了触摸中断为 gpio0_PA6 引脚。

reset-gpio = <&gpio0 RK_PA7 GPIO_ACTIVE_HIGH>;：指定了复位引脚为 gpio0_PA7。

max-x = <720>和 max-y = <1280>：表示屏幕长宽。

8.8 屏幕触摸的使用

出厂系统上已经配置好 GT9xx 触摸驱动并默认编译进内核了，在系统中会生成一个专门的路径/sys/bus/i2c/devices/i2c-2。

```
cd /sys/bus/i2c/devices/i2c-2/
```

```
ls
```

```
root@ATK-DLRK3506:/sys/bus/i2c/devices/i2c-2# ls
2-0014                  name                    subsystem
delete_device           new_device              uevent
device                  of_node                 waiting_for_supplier
i2c-dev                 power
root@ATK-DLRK3506:/sys/bus/i2c/devices/i2c-2#
```

图：/sys/bus/i2c/devices/i2c-2/

目录 /sys/bus/i2c/devices/i2c-2/ 包含了与 I2C 设备 2-0014（地址为 0x14）的相关信息。以下是对该目录下的文件简单讲解：

- **i2c-dev**: 字符设备文件，允许用户空间程序通过标准的文件 I/O 操作来访问 I2C 总线。
- **name**: 包含了 I2C 总线的名称或描述信息。
- **power**: 这个目录包含与设备电源管理相关的文件
- **uevent**: 这个文件包含设备的 uevent 信息，用于通知用户空间关于设备的事件。
- **of_node**: 这是一个符号链接，指向设备树节点，包含设备的设备树信息。
- **subsystem**: 这是一个符号链接，指向设备所属的子系统（在这里是 I2C）。

8.8.1 触摸测试

在出厂文件系统输入如下指令，查看输入事件对应的设备。

```
root@ATK-DLRK3506:/sys/bus/i2c/devices/i2c-2# cat /proc/bus/input/devices
I: Bus=0018 Vendor=dead Product=beef Version=28bb
N: Name="dsi1_ts_gt9xx"
P: Phys=
S: Sysfs=/devices/platform/ff060000.i2c/i2c-2/2-0014/input/input0
U: Uniq=
H: Handlers=event0
B: PROP=2
B: EV=b
B: KEY=400 0 0 0 0 0 0 0 0 0
B: ABS=2650000 0

root@ATK-DLRK3506:/sys/bus/i2c/devices/i2c-2#
```

图：查看输入事件

从上图可以知道，此时的屏幕触摸芯片(GT9xx)已经挂载到 I2C2 总线下，并且 event0 为触摸屏事件，可以用 hexdump 指令来查看输入信息。按 Ctrl+c 终止指令。

```
./etc/init.d/S50systemui stop          //测试前先关闭出厂 QT 界面
hexdump /dev/input/event0
```

```
root@ATK-DLRK3506:/# ./etc/init.d/S50systemui stop
stoping systemui... done.
root@ATK-DLRK3506:/# hexdump /dev/input/event0
0000000 0df2 0001 64f6 0004 0001 014a 0001 0000
0000010 0df2 0001 64f6 0004 0003 0035 0074 0000
0000020 0df2 0001 64f6 0004 0003 0036 0055 0000
0000030 0df2 0001 64f6 0004 0003 0030 0030 0000
0000040 0df2 0001 64f6 0004 0003 0032 0030 0000
0000050 0df2 0001 64f6 0004 0003 0039 0000 0000
0000060 0df2 0001 64f6 0004 0000 0002 0000 0000
0000070 0df2 0001 64f6 0004 0000 0000 0000 0000
0000080 0df2 0001 9115 0004 0003 0035 0074 0000
0000090 0df2 0001 9115 0004 0003 0036 0055 0000
00000a0 0df2 0001 9115 0004 0003 0030 0030 0000
00000b0 0df2 0001 9115 0004 0003 0032 0030 0000
00000c0 0df2 0001 9115 0004 0003 0039 0000 0000
```

图：hexdump 读取数据

测试完可以执行./etc/init.d/S50systemui start 重启 QT ui 界面。

第九章 SPI 子系统

9.1 SPI 基础知识

SPI（Serial Peripheral Interface，串行外设接口）是一种同步串行通信协议，常用于微控制器与外围设备之间的通信。它是一种全双工通信协议，能够实现高速数据传输，通常用于短距离通信，如传感器、存储器和显示器等外设的连接。

SPI 的基本概念

- **主从模式（Master-Slave）：**

主设备（Master）：通常是微控制器，控制整个通信过程，生成时钟信号。
从设备（Slave）：被动响应主设备的命令，接收或发送数据。

- **四条主要信号线：**

MOSI（Master Out Slave In）：主设备发送数据，从设备接收数据。
MISO（Master In Slave Out）：从设备发送数据，主设备接收数据。
SCLK（Serial Clock）：时钟信号，由主设备生成，控制数据传输的同步。
SS（Slave Select）：从设备选择信号，当该信号为低电平时，从设备被选中并参与通信。

- **数据传输模式：**

全双工通信：同时发送和接收数据，MOSI 和 MISO 信号线同时工作。
数据帧格式：数据以帧的形式传输，每帧通常为 8 位或 16 位。

- **时钟极性（CPOL）和时钟相位（CPHA）：**

CPOL：定义时钟信号的空闲状态，0 表示空闲时钟为低电平，1 表示空闲时钟为高电平。
CPHA：定义数据的采样时间，0 表示第一个时钟边沿进行采样，1 表示第二个时钟边沿进行采样。

SPI 的优缺点

优点：

高速传输：SPI 能够支持非常高的时钟频率，通常高于 I2C。
全双工通信：同时进行数据发送和接收，提高了通信效率。
简单协议：实现简单，无需复杂的握手机制，硬件开销低。

缺点：

占用引脚多：每增加一个从设备，需要增加一个独立的 SS 引脚。
短距离通信：适用于短距离设备间通信，不适合长距离传输。
缺乏标准化：不同设备可能会有不同的 SPI 实现方式，需要根据具体设备手册进行配置。

SPI 通信过程

初始化：主设备配置 SPI 参数，包括时钟频率、数据格式（CPOL、CPHA）、以及相关引脚。
选择从设备：主设备将相应的 SS 引脚拉低，选择要通信的从设备。

数据传输：主设备生成时钟信号，并通过 MOSI 发送数据，同时通过 MISO 接收从设备的数据。

结束传输：数据传输完成后，主设备将 SS 引脚拉高，结束与从设备的通信。

SPI 应用实例

存储器：如闪存（Flash）、EEPROM 等，常通过 SPI 接口进行数据读写。

传感器：如温度传感器、加速度传感器等，通过 SPI 接口传输测量数据。

显示器：如 LCD、OLED 显示器，通过 SPI 接口发送图像数据。

SPI 是一种灵活且高效的通信协议，广泛应用于嵌入式系统中。通过合理配置和使用 SPI，可以实现各种外设与微控制器之间的高速数据交换。

SPI 与其他通信协议的比较

与 I2C 的比较：

速度：SPI 通常比 I2C 速度更快，支持更高的时钟频率。

引脚数：I2C 仅需两根信号线（SCL 和 SDA），而 SPI 需要四根或更多信号线。

复杂度：SPI 协议相对简单，无需地址选择和确认机制，I2C 则需要处理地址冲突和握手信号。

与 UART 的比较：

模式：SPI 是同步通信，UART 是异步通信。

速度：SPI 速度通常高于 UART，支持更高的传输速率。

连接方式：SPI 通常用于短距离通信，UART 可以用于较长距离通信，且支持点对点和多点连接。

9.2 SPI 子系统简介

SPI（Serial Peripheral Interface，串行外围设备接口）是嵌入式系统中常用的一种同步串行通信协议。它由 Motorola（现为 Freescale）公司在上世纪 80 年代开发，用于微控制器与外围设备之间的高速数据传输。SPI 协议广泛应用于 EEPROM、Flash、实时时钟（RTC）、ADC、DAC 和显示控制器等设备中。

9.3 主要特点

嵌入式 Linux SPI 子系统主要有以下几个特点：

1、高效的同步串行通信

SPI 协议是一种同步串行通信协议，具有全双工通信能力，可以同时进行数据的发送和接收，适用于高速数据传输。SPI 子系统通过高效的驱动程序实现了对 SPI 设备的支持，能够在嵌入式系统中实现快速、可靠的数据传输。

2、灵活的设备树支持

嵌入式 Linux SPI 子系统通过设备树（Device Tree）机制描述硬件架构和设备信息，方便系统在启动时自动加载和配置 SPI 设备。设备树支持使得硬件平台的移植和设备的添加变得更加简单和灵活。

3、统一的用户空间接口

SPI 子系统提供了统一的用户空间接口，用户应用程序可以通过标准文件接口（如 /dev/spidevX.Y）与 SPI 设备进行通信。用户空间接口的标准化和统一化简化了应用程序与 SPI 设备交互的过程，提高了开发效率。

4、广泛的设备支持

SPI 子系统支持多种类型的 SPI 设备，如 EEPROM、Flash、实时时钟（RTC）、ADC、DAC 和显示控制器等。通过 SPI 主机控制器驱动和设备驱动的协作，SPI 子系统能够兼容和支持各种不同的 SPI 从设备，满足不同应用场景的需求。

这些特点使得嵌入式 Linux SPI 子系统在嵌入式开发中具有重要的地位，广泛应用于各类嵌入式系统和设备中。

9.4 组成部分

嵌入式 Linux SPI 子系统主要由以下几个部分组成：

1、SPI 核心层

SPI 核心层提供了 SPI 子系统的通用接口和基础功能。它负责 SPI 设备和主机控制器的注册、总线管理、数据传输和设备通信等。核心层位于内核源代码的 drivers/spi/spi.c 文件中。

2、SPI 主机控制器驱动

SPI 主机控制器驱动负责管理 SPI 总线上的主机控制器。它实现了与具体硬件平台相关的功能，允许 SPI 核心层通过统一的接口与不同的硬件进行交互。这些驱动程序通常位于 drivers/spi 目录中。

3、SPI 设备驱动

SPI 设备驱动负责与具体的 SPI 从设备进行通信。每个 SPI 从设备都有一个相应的设备驱动程序，这些驱动程序通过 SPI 核心层与主机控制器驱动进行数据传输。设备驱动通常位于 drivers/spi 或与设备类型相关的其他目录中。

4、设备树支持

设备树（Device Tree）用于描述硬件架构和设备信息，SPI 子系统利用设备树节点来配置和初始化 SPI 主机和从设备。设备树支持使得系统在启动时能够自动识别和配置 SPI 设备，通常在设备树源文件（*.dts）中进行描述。

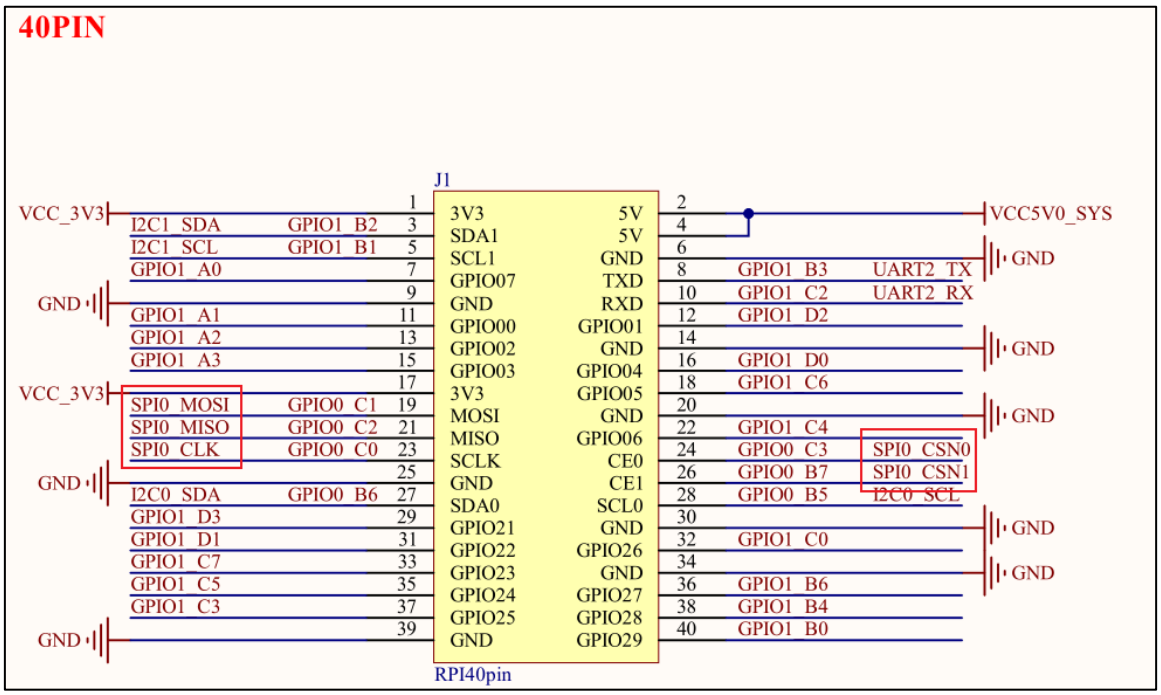
5、用户空间接口

SPI 子系统提供了用户空间接口，允许用户应用程序通过标准文件操作与 SPI 设备进行通信。常用的接口文件包括 /dev/spidevX.Y，用户可以使用 open、read、write 和 ioctl 等标准文件操作函数与 SPI 设备进行交互。

9.5 开发板 SPI 设备

9.5.1 原理图分析

打开开发板上开发板网盘 A 盘-基础资料\02、原理图\01、底板原理图\ ATK-DLRK3506B V1.0.pdf 搜索 SPI。可以看到 SPI 相关电路如下。



图：开发板 SPI 设备原理图

底板上引出了一组 SPI0。

9.6 内核 SPI 子系统使能

内核源码配置文件 kernel/arch/arm/configs/rk3506-atk-dlrk3506.config 和 rk3506_defconfig 中，SPI 相关配置如下(略有删减，只列举重点部分)：

```
CONFIG_SPI=y                    //启用 SPI 支持
CONFIG_SPI_SPIDEV=y             //启用用户空间 SPI 设备驱动程序
CONFIG_SPI_ROCKCHIP=y           //启用 RK 平台的 SPI 控制驱动
```

9.7 设备树文件分析

打开 kernel/arch/arm/boot/dts/rk3502.dtsi，关于 SPI0 的描述如下：

```
spi0: spi@ff120000 {
    compatible = "rockchip,rk3506-spi", "rockchip,rk3066-spi";
    reg = <0xff120000 0x1000>;
    interrupts = <GIC_SPI 43 IRQ_TYPE_LEVEL_HIGH>;
    #address-cells = <1>;
    #size-cells = <0>;
    clocks = <&cru CLK_SPI0>, <&cru PCLK_SPI0>;
    clock-names = "spiclk", "apb_pclk";
```

```
        dmas = <&dmac0 0 0xff2880a8 0x00030001 0x0 0x0>,
               <&dmac0 1 0xff2880a8 0x000c0004 0x0 0x0>;
        dma-names = "tx", "rx";
        num-cs = <2>;
        pinctrl-names = "default";
        pinctrl-0 = <&spi0_csn0_pins &spi0_csn1_pins &spi0_clk_pins>;
        status = "disabled";
    };
```

spi0: spi@ff120000：表示了 SPI0 控制器的节点，地址为 0x ff120000。

compatible = "rockchip,rk3506-spi", "rockchip,rk3066-spi"; ：这是一个属性，表示该设备与 RK 的 SPI0 兼容。

reg = <0xff120000 0x1000>; ：指定了 SPI0 寄存器地址范围，起始地址是 0xff120000，长度是 0x1000。

interrupts = <GIC_SPI 43 IRQ_TYPE_LEVEL_HIGH>; ：表示该 SPI 控制器使用 GIC（Generic Interrupt Controller）的 SPI（Shared Peripheral Interrupt）类型中断，中断号是 43，中断触发类型为高电平触发。

clocks = <&cru CLK_SPI0>, <&cru PCLK_SPI0>; ：用于指定设备所使用的时钟源。

clock-names = "spiclk", "apb_pclk";：为每个时钟源指定名称。

dmas = <&dmac0 0 0xff2880a8 0x00030001 0x0 0x0>,
<&dmac0 1 0xff2880a8 0x000c0004 0x0 0x0>;用于指定设备所使用的 DMA（Direct Memory Access）通道。

pinctrl-0 = <&spi0_csn0_pins &spi0_csn1_pins &spi0_clk_pins>;用于指定默认引脚控制状态对应的引脚配置节点。

打开内核源码 kernel/arch/arm/boot/dts/rk3506-atk-dlrk3506b.dtsi 文件，在 SPI2 节点下添加一个 SPI 虚拟接口设备，相关配置如下：

```
&spi0{
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&spi0_csn0_pins &spi0_csn1_pins &spi0_clk_pins>;
    spidev@0{
        compatible = "rockchip,spidev";
        spi-max-frequency = <50000000>;
        reg = <0>;
    };

    spidev@1{
        compatible = "rockchip,spidev";
        spi-max-frequency = <50000000>;
        reg = <1>;
    };
};
```

status = "okay" ：将 SPI2 控制器启用。

spidev@0：spi0 的 0 号片选。

spidev@1：spi0 的 1 号片选。

9.8 SPI 调试工具

9.8.1 spi-config

spi-config：用于配置 SPI 设备的底层参数，如通信模式、数据顺序、时钟速度等。执行 spi-config -h 查看帮助：

```
root@ATK-DLRK3506:/# spi-config -h
usage: spi-config options...
  options:
    -d --device=<dev>  use the given spi-dev character device.
    -q --query         print the current configuration.
    -m --mode=[0-3]    use the selected spi mode:
             0: low idle level, sample on leading edge,
             1: low idle level, sample on trailing edge,
             2: high idle level, sample on leading edge,
             3: high idle level, sample on trailing edge.
    -l --lsb={0,1}     LSB first (1) or MSB first (0).
    -b --bits=[7...]   bits per word.
    -s --speed=<int>   set the speed in Hz.
    -r --spirdy={0,1}  consider SPI_RDY signal (1) or ignore it (0).
    -w --wait          block keeping the file descriptor open to avoid speed reset.
    -h --help          this screen.
    -v --version       display the version number.
root@ATK-DLRK3506:/#
```

图：spi-config 说明

9.8.2 spi-pipe

spi-pipe：用于 SPI 数据传输，支持发送、接收或双向测试。我们也可以用 spi-pipe -h 指令查看帮助：

```
root@ATK-DLRK3506:/# spi-pipe -h
usage: spi-pipe options...
  options:
    -d --device=<dev>    Use the given spi-dev character device.
    -m --mode=[0-3]      Use the selected spi mode:
             0: low idle level, sample on leading edge,
             1: low idle level, sample on trailing edge,
             2: high idle level, sample on leading edge,
             3: high idle level, sample on trailing edge.
    -s --speed=<speed>   Maximum SPI clock rate (in Hz).
    -l --lsb={0,1}     LSB first (1) or MSB first (0).
    -B --bits=[7...]     Bits per word.
    -b --blocksize=<int> transfer block size in byte.
    -n --number=<int>    number of blocks to transfer (-1 = infinite).
    -h --help            this screen.
    -v --version         display the version number.
root@ATK-DLRK3506:/#
```

图：spi-piep 说明

根据帮助信息，我们可以用如下指令去测试 spi，

```
spi-config -d /dev/spidev0.0 -m 0 -s 1000000 -l 0 -b 8
```

-d /dev/spidev0.0：指定要操作的 SPI 设备节点为 /dev/spidev0.0。

-m 0：设置 SPI 的通信模式为模式 0。SPI 有四种工作模式（模式 0 - 模式 3），不同的模式由时钟极性（CPOL）和时钟相位（CPHA）决定。模式 0 表示时钟空闲时为低电平（CPOL = 0），数据在时钟的上升沿（即领先边沿）采样（CPHA = 0）。

-s 1000000：设置 SPI 的通信时钟速度为 1000000 Hz（即 1 MHz）。

-l 0：指定数据传输时采用 MSB（Most Significant Bit，最高有效位）优先的方式。

-b 8：设置每次传输的数据位数为 8 位。

配置好 spi 参数，接着我们测试 spi 是否能正常通信，因为我们底板上没有 spi 外设，所以我们利用回环测试来检测 spi 通信是否正常，将 spi 中的 MOSI 与 MISO 用杜邦线连接在一起，这样就相当于 spi 自己发送数据给自己接收。然后执行如下指令：

```
echo "Hello SPI!" | spi-pipe -d /dev/spidev0.0 -s 1000000 -m 0 -B 8 -b 1024 -n 1
```

echo "Hello SPI!"：这是一个简单的 shell 命令，用于在终端输出字符串 "Hello SPI!"。

-d /dev/spidev0.0：同样指定要操作的 SPI 设备节点为 /dev/spidev0.0，需与 spi-config 中指定的设备一致。

-s 1000000：设置 SPI 的通信时钟速度为 1000000 Hz（即 1 MHz），要和 spi-config 中设置的速度保持一致。

-m 0：设置 SPI 的通信模式为模式 0，和 spi-config 中的模式设置相同。

-B 8：设置每次传输的数据位数为 8 位，与 spi-config 中的 -b 8 作用相同。

-b 1024：指定每次传输的数据块大小为 1024 字节。spi-pipe 会按照这个块大小来组织数据进行传输。

-n 1：指定要传输的数据块数量为 1 个。也就是说，spi-pipe 会发送一次数据块。

```
root@ATK-DLRK3506:/# spi-config -d /dev/spidev0.0 -m 0 -s 1000000 -l 0 -b 8
root@ATK-DLRK3506:/# echo "Hello SPI!" | spi-pipe -d /dev/spidev0.0 -s 1000000 -m 0 -B 8 -b 1024 -n 1
Hello SPI!
root@ATK-DLRK3506:/#
```

图：使用 spi 测试

可以看到，将 spi 的 MOSI 和 MISO 接上后，终端会打印出 spi 发送的字符串。

第十章 UART 子系统

10.1 UART 子系统简介

嵌入式 Linux 的 UART（Universal Asynchronous Receiver/Transmitter）子系统是嵌入式系统中非常重要的部分，它负责串行通信的管理和控制。UART 子系统在嵌入式设备中用于与其他设备（如传感器、外设、计算机等）进行数据交换。

UART 的基本概念：

UART 是一种串行通信协议，用于异步数据传输。它可以将并行数据转换为串行数据，并通过串口发送。

波特率：数据传输速率，通常以比特每秒（bps）表示。

数据位、停止位和校验位：控制数据传输的格式。

10.2 主要特点

嵌入式 Linux 的 UART 子系统有几个显著的特点，这些特点使得它在嵌入式系统中广泛应用于处理低速、串行的数据通信。以下是 UART 子系统的主要特点：

1、低成本和简单性：

- UART 硬件通常非常简单，这意味着成本较低，适合用在成本敏感的嵌入式项目中。
- 实现 UART 通信不需要复杂的硬件支持，如时钟信号同步，因为它是异步通信。

2、广泛的设备兼容性：

- UART 是一种极为普遍的通信标准，被多种微控制器、处理器及其他电子组件支持。
- 这种通用性确保了它可以与各种传感器、模块和外部系统（如调试设备、PC 等）轻松集成。

3、直接的内核支持和标准接口：

- Linux 内核提供了对 UART 的广泛支持，包括标准的设备驱动程序和 tty 接口，这使得从用户空间访问 UART 设备变得非常简单。
- 开发者可以通过标淨的系统调用，如 open、read、write 和 close 来操作 UART 设备，无需复杂的配置。

4、可扩展的串行通信选项：

- UART 子系统支持多种配置选项，包括不同的波特率、停止位、奇偶校验等，使其能够灵活地适应各种通信需求。
- 通过使用设备树（Device Tree），可以灵活地配置和管理硬件资源，例如指定中断和 GPIO 映射，以支持更多的串行端口。

这些特点使得 UART 在嵌入式系统中成为实现设备间低速数据交换的理想选择，尤其是在资源受限或成本敏感的应用场景中。

10.3 组成部分

嵌入式 Linux 的 UART 子系统由多个部分组成，这些部分共同作用，确保串行通信的有效性和可靠性。以下是 UART 子系统的主要组成部分：

1、硬件部分

- UART 控制器：UART 硬件模块，负责串行数据的发送和接收。典型的 UART 控制器具有发送（TX）和接收（RX）引脚。

- 时钟和电源管理：提供给 UART 控制器的时钟信号和电源，确保其正常工作。
- 引脚复用（Pin Multiplexing）：在许多嵌入式系统中，UART 引脚可能与其他功能共享，需要通过引脚复用来配置。

2、设备树（Device Tree）

- 设备树源文件（.dts）：描述硬件平台的配置文件，包括 UART 控制器的内存映射、中断号和其他硬件参数。
- 设备树编译文件（.dtb）：编译后的设备树二进制文件，由内核在启动时加载。

3、驱动程序

- 内核 UART 驱动程序：位于 Linux 内核中的驱动代码，负责初始化和管理 UART 硬件。典型的 UART 驱动程序包括：
  - 低级驱动程序：直接与 UART 硬件交互，进行数据传输和接收的基本操作。
  - 中断处理程序：处理 UART 控制器发出的中断信号，管理数据的缓冲和传输。

4、tty 子系统

- tty 层：Linux 内核中的字符设备子系统，负责管理终端设备和串行通信接口。tty 层为用户空间应用程序提供标准的字符设备接口。
- tty 驱动程序：UART 驱动程序通常实现为 tty 驱动，使得 UART 设备可以作为 tty 设备被访问，如/dev/ttyS0。

5、用户空间工具和库

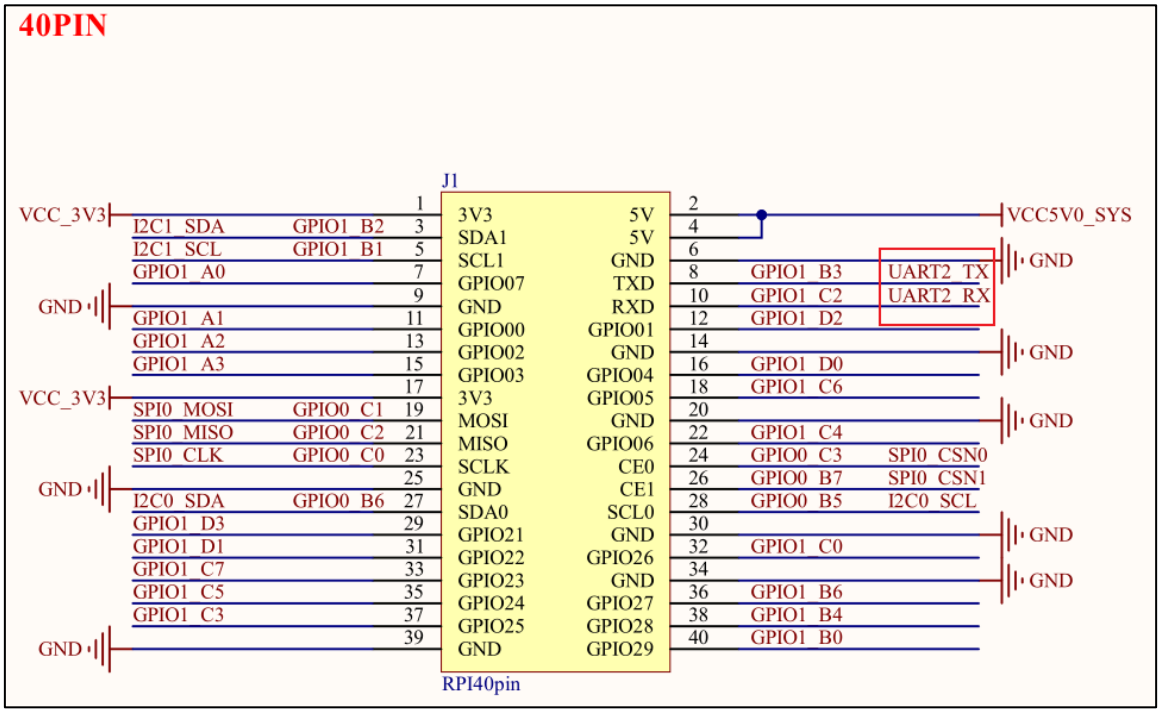
- 设备节点：位于/dev 目录下的设备文件，如/dev/ttyS0，用户空间应用程序通过这些设备文件与 UART 硬件进行交互。
- 配置和测试工具：
  - stty：用于设置和显示串行设备的参数。
  - minicom：一个串行终端仿真程序，用于调试和测试 UART 设备。

6、调试和诊断

- 日志和调试信息：内核日志（通过 dmesg 查看）和调试信息有助于识别和解决 UART 通信中的问题。
- 调试接口：许多 UART 驱动程序提供调试接口，通过这些接口可以获取驱动程序的运行状态和统计信息。

10.4 原理图分析

打开开发板上开发板网盘 A 盘-基础资料\02、原理图\01、底板原理图\ ATK-DLRK3506B V1.0.pdf 搜索 UART。可以看到 UART 相关电路如下。



图：开发板 UART 电路

开发板底板上有引出一路 UART2。

10.5 内核 UART 子系统使能

内核源码配置文件 kernel/arch/arm/configs/rk3506_defconfig 中，串口 SERIAL 相关配置如下(略有删减，只列举重点部分)：

```
CONFIG_SERIAL_8250=y //启用标准的 8250 系列 UART 控制器支持
CONFIG_SERIAL_8250_CONSOLE=y //将 8250 UART 用作系统控制台输出
CONFIG_SERIAL_8250_NR_UARTS=6  //设置支持的串口数量
CONFIG_SERIAL_8250_RUNTIME_UARTS=6  //动态分配串口资源
```

10.6 设备树文件分析

打开 kernel/arch/arm/boot/dts/rk3502.dtsi，关于 UART 的描述如下：

```
uart2: serial@ff0c0000 {
    compatible = "rockchip,rk3506-uart", "snps,dw-apb-uart";
    reg = <0xff0c0000 0x100>;
    interrupts = <GIC_SPI 36 IRQ_TYPE_LEVEL_HIGH>;
    reg-shift = <2>;
    reg-io-width = <4>;
    dmas = <&dmac0 8 0xff2880ac 0x00030001 0x0 0x0>,
           <&dmac0 9 0xff2880ac 0x000c0004 0x0 0x0>;
    clocks = <&cru SCLK_UART2>, <&cru PCLK_UART2>;
    clock-names = "baudclk", "apb_pclk";
    status = "disabled";
};
```

uart2: serial@ff0c0000：节点名称，表示这是一个串口设备，基地址为 0xff0c0000。
compatible = "rockchip,rk3506-uart", "snps,dw-apb-uart";：定义设备与驱动的匹配规则。
reg = <0xff0c0000 0x100>;：描述设备的物理地址和寄存器空间大小。
interrupts = <GIC_SPI 36 IRQ_TYPE_LEVEL_HIGH>;：定义设备的中断信息。
dmas = XXX：配置 DMA 控制器和通道。
clocks = <&cru SCLK_UART2>, <&cru PCLK_UART2>;：定义设备所需的时钟源。
打开 kernel/arch/arm/boot/dts/rk3506-atk-dlrk3506b.dtsi，关于 UART2 的描述如下：.

```
1   uart2 {
2       /omit-if-no-ref/
3       uart2_xfer_pins: uart2-xfer-pins {
4           rockchip,pins =
5               /* uart2_rx */
6               <1 RK_PC2 19 &pcfg_pull_up>,
7               /* uart2_tx */
8               <1 RK_PB3 18 &pcfg_pull_none>;
9       };
10    };
```

这段设备树内容主要就是添加 uart2 对应的引脚。

10.7 系统接口

10.7.1 /dev/ttyS*

在嵌入式 Linux 系统中，/dev/ttyS* 设备节点通常表示串口设备。

```
root@ATK-DLRK3506:/# ls /dev/ttyS*
/dev/ttyS2
root@ATK-DLRK3506:/#
```

图：串口设备

可以看到，上图中的 ttyS2 节点就是我们注册的 uart2。

10.7.2 /sys/class/tty/ttyS*

从 /sys/class/tty/ttyS*路径下列出的信息可以进一步确认和了解这些串口设备的详细属性。

```
root@ATK-DLRK3506:/# ls /sys/class/tty/ttyS*
close_delay      dev              iomem_base       port             type
closing_wait     device           iomem_reg_shift  power            uartclk
console          flags            irq              rx_trig_bytes    uevent
custom_divisor   io_type          line             subsystem        xmit_fifo_size
root@ATK-DLRK3506:/#
```

图：串口设备属性

下面是这些属性的含义：

关键属性解析

- close_delay 和 closing_wait：

这些文件控制串口关闭时的延迟设置，单位为毫秒。

- dev：

这个文件包含设备的主次设备号。例如，242:0 表示主设备号为 242，次设备号为 0。

- device：

指向该串口设备对应的硬件设备目录。

- flags：

显示串口设备的标志状态。

- irq：

显示分配给该串口设备的中断请求号（IRQ）。

- port：

显示串口设备的 I/O 端口地址。

- uartclk：

显示 UART 时钟频率。

- type：

显示串口类型。

确认具体硬件设备

- 查看设备的主次设备号。例如：

```
cat /sys/class/tty/ttyS2/dev
```

```
root@ATK-DLRK3506:/# cat /sys/class/tty/ttyS2/dev
4:66
root@ATK-DLRK3506:/#
```

图：查看设备的主次设备号

输出类似 4:66，表示主设备号和次设备号。

- 查看串口设备的 IRQ 号

```
cat /sys/class/tty/ttyS2/irq
```

```
root@ATK-DLRK3506:/# cat /sys/class/tty/ttyS2/irq
44
root@ATK-DLRK3506:/#
```

图：查看设备中断号

- 可以通过查看 device 文件夹进一步确认串口设备对应的具体硬件设备。例如：

```
ls -l /sys/class/tty/ttyS2/device
```

这个命令会展示串口设备 ttyS2 关联的硬件设备信息，包括设备树路径或平台设备信息。

```
root@ATK-DLRK3506:/# ls -l /sys/class/tty/ttyS2/device
lrwxrwxrwx    1 root     root           0 Apr  3 15:59 /sys/class/tty/ttyS2/device -> ../../../ff0c0000.serial
root@ATK-DLRK3506:/#
```

图：查看串口设备关联的硬件设备

10.8 UART 测试

因为底板上 uart2 没有接到对应的串口设备，所以我们只能通过将串口 2 的 TX 和 RX 对接。我们可以通过监听串口 2 的方式去检查是不是有正常通信。执行如下指令：

```
stty -F /dev/ttyS2 -echo        //禁用串口 /dev/ttyS2 的输入回显功能。
cat /dev/ttyS2 &                //后台监听串口 2
echo "Hello,uart!" > /dev/ttyS2     //通过串口 2 发送字符串
```

```
root@ATK-DLRK3506:/# stty -F /dev/ttyS2 -echo
[    27.925634] dw-apb-uart ff0c0000.serial: got rx and tx dma channels
root@ATK-DLRK3506:/# cat /dev/ttyS2 &
[1] 950
root@ATK-DLRK3506:/# [   178.128153] dw-apb-uart ff0c0000.serial: got rx and tx dma channels

root@ATK-DLRK3506:/# echo "Hello,uart!" > /dev/ttyS2
root@ATK-DLRK3506:/# Hello,uart!
接收到的数据


root@ATK-DLRK3506:/#
```

图：查看串口设备关联的硬件设备

可以看到 tx 和 rx 对接后，通过向/dev/ttyS2 发送的字符串，被/dev/ttyS2 接收到。

第十一章 Display 子系统

11.1 Display 子系统简介

嵌入式 Linux 的显示子系统通常包括以下几个主要组件：显示驱动、帧缓冲设备（Framebuffer）、显卡驱动、图形用户接口（GUI）库以及显示服务器等。这些组件协同工作，提供从硬件图形渲染到高级图形用户界面的全套功能。

11.2 主要特点

嵌入式 Linux 的显示子系统具有以下几个主要特点：

1、轻量级设计

嵌入式显示子系统设计得尽可能轻量级，以减少资源占用。这对于资源有限的嵌入式设备尤其重要，确保系统能够高效运行。

2、模块化结构

显示子系统由独立的组件组成，如显示驱动、帧缓冲、图形库和显示服务器。这种模块化设计使系统更易于定制和优化，以满足不同应用场景的需求。

3、多种显示接口支持

嵌入式 Linux 支持多种显示接口，包括 HDMI、LVDS、MIPI-DSI 和 RGB。这种多样性使系统能够适配各种显示屏和输出设备。

4、硬件加速支持

许多嵌入式系统的显示子系统支持硬件加速图形处理，通过 GPU 驱动和图形库（如 OpenGL ES）实现。这显著提升了图形渲染性能，适用于需要高性能图形处理的应用场景。

嵌入式 Linux 显示子系统通过其轻量级、模块化、多接口支持和硬件加速等特点，适应了嵌入式设备对高效、灵活和低功耗显示解决方案的需求。这些特点使嵌入式 Linux 成为构建复杂图形界面和高性能显示系统的理想选择。

11.3 组成部分

1、显示驱动 (Display Driver)

显示驱动程序是直接与显示硬件交互的内核模块。它们负责初始化显示设备、设置分辨率、管理帧缓冲区等。显示驱动通常依赖于设备树（Device Tree）中的硬件描述信息。

2、帧缓冲设备 (Framebuffer)

Framebuffer 是一种内核机制，它允许应用程序直接访问显示设备的显存（视频内存）。Framebuffer 设备通常在/dev/fb0 等文件中表示。应用程序可以通过这些设备文件直接读写图像数据。

3、显卡驱动 (Graphics Driver)

显卡驱动程序（如 DRM/KMS 驱动）提供更高级的图形渲染功能。DRM（Direct Rendering Manager）和 KMS（Kernel Mode Setting）是 Linux 内核子系统，用于管理图形硬件并提供直接渲染支持。

4、图形用户接口库 (GUI Libraries)

嵌入式系统中常用的 GUI 库包括 Qt、GTK、DirectFB 等。这些库提供了创建和管理窗口、控件、事件处理等功能，简化了图形用户界面的开发。

5、显示服务器 (Display Server)

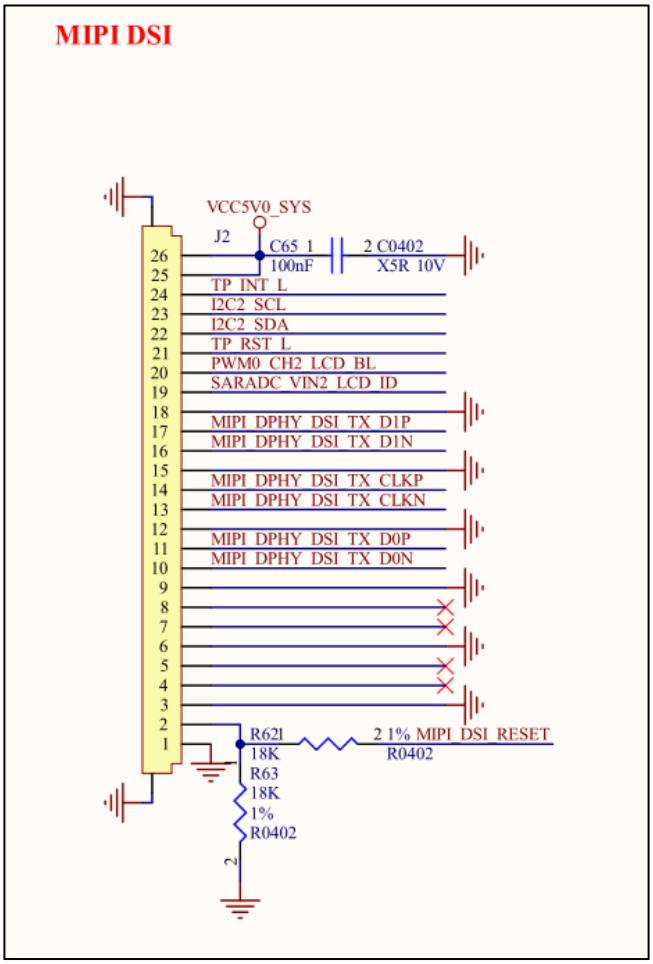
显示服务器（如 X.Org、Wayland）管理窗口系统和图形渲染，处理来自多个应用程序的图形请求，并将它们合成到屏幕上。Wayland 是现代显示服务器协议，比 X.Org 更适合嵌入式系统。

嵌入式 Linux 的显示子系统涉及多个层次的组件，从底层的显示驱动和帧缓冲设备，到高级的图形库和显示服务器。设备树用于描述硬件资源，驱动程序实现硬件的具体控制，图形库和显示服务器则提供应用程序开发接口。通过这些组件的协同工作，可以构建功能强大的嵌入式图形界面。

11.4 开发板 MIPI 接口

11.4.1 原理图分析

在开发板网盘 A 盘-基础资料\02、原理图\02、核心板原理图\ ATK-CLRK3506B V1.0.pdf 搜索 dsi。可以看到 MIPI DSI 相关电路如下。



图：MIPI DSI 电路

原理图显示了一个 MIPI DSI 接口的连接和相关的驱动电路。以下是对该原理图的分析：

VCC5V0_SYS：5V 电源输入。

TP_INT_L：触摸中断引脚。

I2C2_SCL，I2C2_SDA 触摸 iic 引脚。

TP_RSR_L：触摸复位引脚。

PWM0_CH2_LCD_BL：屏幕背光引脚。

SARADC_VIN2_LCD_ID：屏幕 ID 引脚。

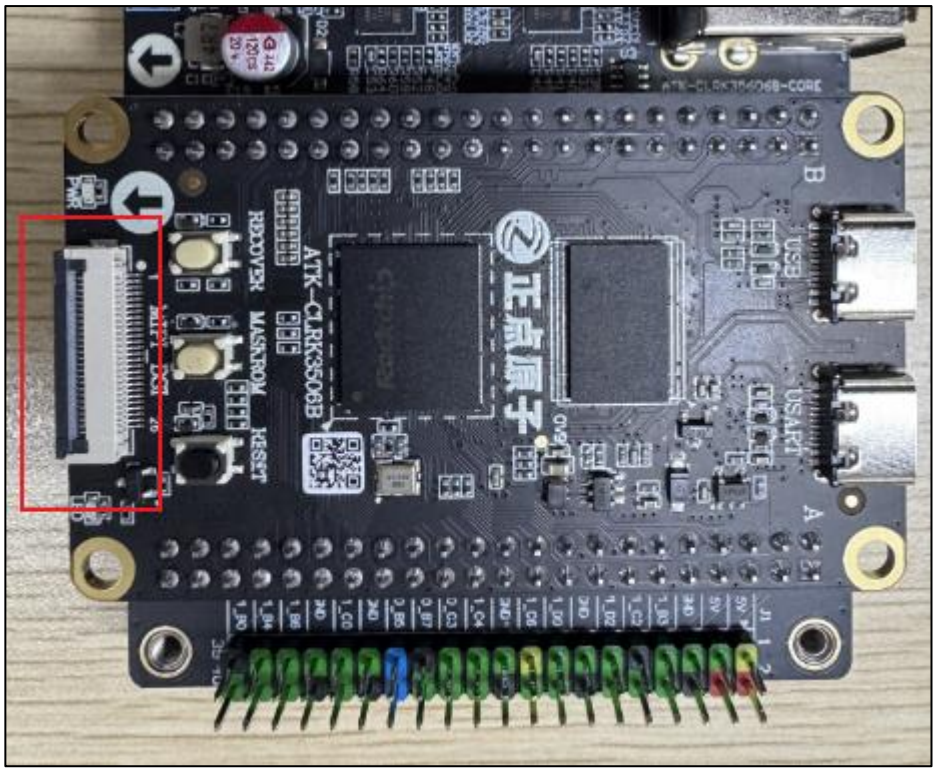
MIPI_DPHY_DSI_TX_D1P、MIPI_DPHY_DSI_TX_D1N、MIPI_DPHY_DSI_TX_D0P、MIPI_DPHY_DSI_TX_D0N：屏幕数据引脚。在 RK3506 上使用的是 2lane。

MIPI_DPHY_DSI_TX_CLKP、MIPI_DPHY_DSI_TX_CLKN：屏幕时钟引脚。

MIPI_DSI_RESET：屏幕复位引脚。

11.4.2 实物硬件

开发板上的 MIPI dsi 接口位置如下图所示：



图：mipi dsi 接口

11.5 设备树分析

打开 kernel/arch/arm/boot/dts/rk3506-mipi-screen.dtsi，中关于 display 的描述如下：

```
&dsi {
    status = "okay";
        dsi_panel: panel@0 {
                panel-init-sequence = [
                        39 00 04 B9 FF 83 94
                        //15 00 02 36 01(cŸ')
                        //15 00 02 36 02(ucŸ')
                        15 00 02 36 01
                        39 00 07 BA 61 03 68 6B B2 C0
                        39 00 0B B1 48 12 72 09 32 54 71 71 57 47
                        39 00 07 B2 00 80 64 0C 0D 2F
```

```
                        39 00 16 B4 73 74 73 74 73 74 01 0C 86 75 00 3F
73 74 73 74 73 74 01 0C 86
                        39 00 03 B6 6E 6E
                        39 00 22 D3 00 00 07 07 40 07 0C 00 08 10 08 00
08 54 15 0A 05 0A 02 15 06 05 06 47 44 0A 0A 4B 10 07 07 0C 40
                        39 00 2D D5 1C 1C 1D 1D 00 01 02 03 04 05 06 07
08 09 0A 0B 24 25 18 18 26 27 18 18 18 18 18 18 18 18 18 18 18 18 18 18
18 18 20 21 18 18 18 18
                        39 00 2D D6 1C 1C 1D 1D 07 06 05 04 03 02 01 00
0B 0A 09 08 21 20 18 18 27 26 18 18 18 18 18 18 18 18 18 18 18 18 18 18
18 18 25 24 18 18 18 18
                        39 00 3B E0 00 0A 15 1B 1E 21 24 22 47 56 65 66
6E 82 88 8B 9A 9D 98 A8 B9 5D 5C 61 66 6A 6F 7F 7F 00 0A 15 1B 1E 21 24
22 47 56 65 65 6E 81 87 8B 98 9D 99 A8 BA 5D 5D 62 67 6B 72 7F 7F
                        39 00 03 C0 1F 31
                        15 00 02 CC 03
                        15 00 02 D4 02
                        15 00 02 BD 02
                        39 00 0D D8 FF FF FF FF FF FF FF FF FF FF FF FF
                        15 00 02 BD 00
                        15 00 02 BD 01
                        15 00 02 B1 00
                        15 00 02 BD 00
                        39 00 08 BF 40 81 50 00 1A FC 01
                        15 00 02 C6 ED
                        05 64 01 11
                        05 78 01 29

                ];

                disp_timings0: display-timings {
                        native-mode = <&dsi_timing0>;
                        dsi_timing0: timing0 {
                                clock-frequency = <65000000>;
                                hactive = <720>;
                                vactive = <1280>;
                                hfront-porch = <48>;
                                hsync-len = <8>;
                                hback-porch = <52>;
                                vfront-porch = <16>;
                                vsync-len = <6>;
                                vback-porch = <15>;
                                hsync-active = <0>;
```

```
                                vsync-active = <0>;
                                de-active = <0>;
                                pixelclk-active = <0>;
                        };
                };

        };
};
```

这段设备树描述了一个 RK3506 的显示子系统配置。主要可以分为两个部分：

dsi_panel: panel@0：该节点下描述了 mipi 屏幕的初始化序列。

disp_timings0: display-timings：节点主要是一些屏幕的配置信息，如屏幕时钟，屏幕长宽还有一些屏幕极性之类的配置。

第十二章 Ethernet 子系统

12.1 Ethernet 子系统简介

嵌入式 Linux 的以太网子系统（Ethernet Subsystem）是用于处理网络通信的关键组件。它主要涉及管理网络接口、处理数据包传输和接收、支持各种网络协议等。

12.2 组成部分

1、网络接口（Network Interfaces）：
管理物理和虚拟网络接口，如以太网卡、Wi-Fi 等。
支持多种网络设备驱动程序，确保硬件兼容性。

2、网络协议栈（Network Protocol Stack）：
提供 TCP/IP 协议栈，支持标准网络通信协议（如 ARP、ICMP、UDP、TCP 等）。
处理数据包的封装和解封装，确保数据正确传输。

3、网络配置工具（Network Configuration Tools）：
提供命令行工具（如 ifconfig、ip、ethtool 等）用于配置和管理网络接口。
支持动态 IP 配置（DHCP）和静态 IP 配置。

4、网络服务（Network Services）：
提供基本的网络服务，如 DHCP 客户端/服务器、DNS 解析、NTP 时间同步等。
支持高级网络服务和功能，如 VPN、网桥、虚拟局域网（VLAN）等。

12.3 主要功能

1、网络接口管理：
配置和管理多种网络接口，支持网络接口的启用、禁用和参数调整。

2、数据包处理：
高效处理数据包的发送和接收，确保低延迟和高吞吐量。
处理数据包的路由和转发，支持多种路由协议。

3、网络协议支持：
支持多种标准网络协议，确保嵌入式设备与其他设备的互操作性。
提供可靠的传输控制和流量控制，确保数据传输的稳定性。

4、网络安全：
提供基本的网络安全功能，如防火墙（iptables）、网络地址转换（NAT）等。
支持安全的通信协议，如 TLS/SSL 等。

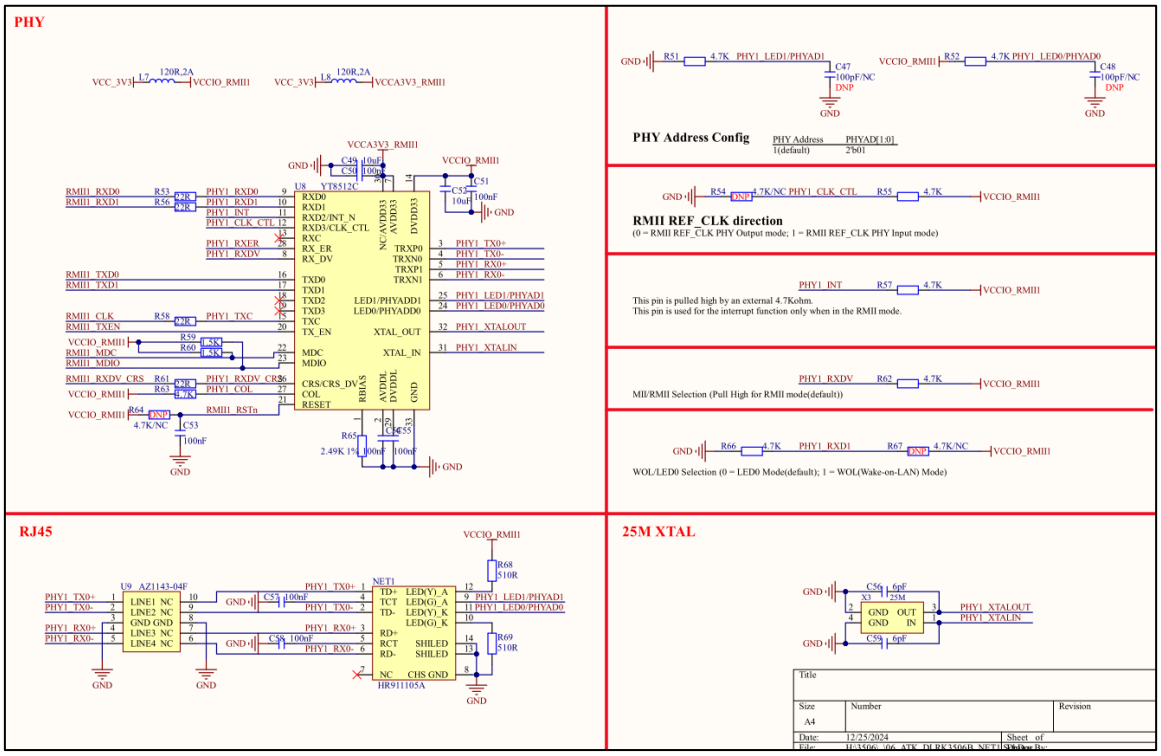
5、质量服务（QoS）：
提供服务质量管理，确保关键数据流的优先级和带宽分配。

支持流量整形和带宽管理，优化网络资源使用。

12.4 开发板 Ethernet 接口

12.4.1 原理图分析

开发板上有两个 Ethernet 接口。打开开发板上开发板网盘 A 盘-基础资料\02、原理图\01、底板原理图\ ATK-DLRK3506B V1.0.pdf 搜索 RMII。可以看到相关电路如下。



图：开发板 ETH1 电路

此图展示了一个 100 Mbps 以太网（ENET）接口的电路图。以下是主要组件和连接的解析：

网络芯片：电路图的中心组件，是一个以太网控制器或 MAC 芯片 YT8512C，负责核心的以太网功能。

MDC/MDIO：管理数据时钟和管理数据输入/输出引脚，用于 PHY 管理。

TX/RX 线：发送（TX）和接收（RX）线连接到变压器和磁性器件，表明以太网通信的差分信号。

LED 指示灯：连接到芯片的指示灯，用于显示链接、活动和速度状态。

变压器和磁性器件：这些组件连接到 TX 和 RX 线，提供隔离和阻抗匹配，这是正确传输和接收以太网信号所必需的。

RJ45 接口：以太网电缆的物理接口，连接到磁性器件和变压器，用于数据的发送和接收。

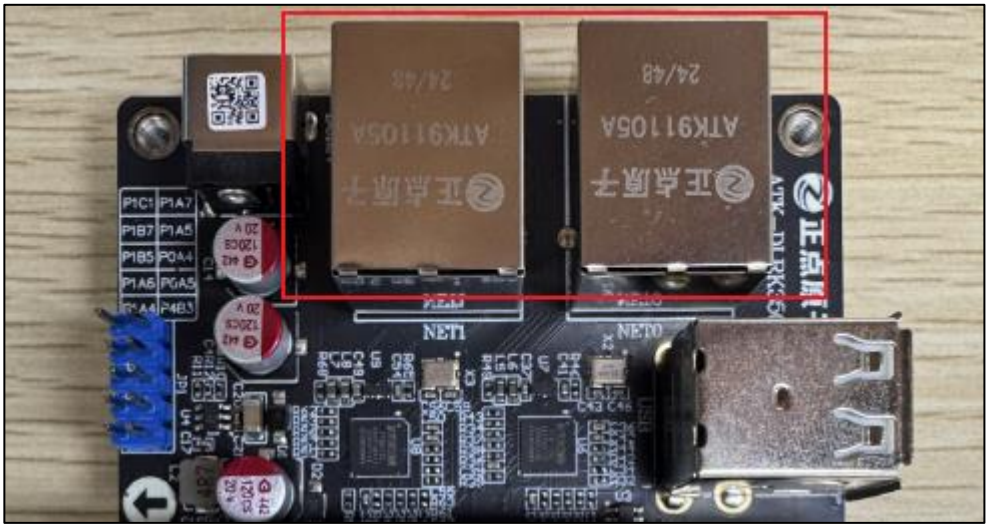
电源供应和去耦：电路包括几个电源连接（VCC）和去耦电容，确保稳定的操作和噪声减少。

控制和配置引脚：有各种控制引脚，如复位和中断线，允许外部设备或微控制器控制或监控 ENET 芯片。

振荡器/时钟输入：电路图还显示了 ENET 芯片的时钟输入，这是数据传输中计时和同步的必要部分。

12.4.2 硬件实物

开发板上的 Ethernet 接口位置如下图所示：



图：开发板 Ethernet 接口

12.5 内核 Ethernet 使能

在 ATK-DLRK3506 内核源码 kernel/arch/arm/configs/rk3506-ethernet.config 和 rk3506-atk-dlrk3506.config 中，已经有 Ethernet 相关驱动配置。

```
CONFIG_MOTORCOMM_PHY=y          //phy 芯片厂家对应的内核配置
CONFIG_NETDEVICES=y             //通用的网络设备支持
CONFIG_PHYLIB=y                 //PHY 库
```

12.6 设备树分析

打开 kernel/arch/arm/boot/dts/rk3506-atk-dlrk3506b.dtsi，设备树中关于 Ethernet1(Ethernet0 类同)描述如下：

```
&gmac1 {
        phy-mode = "rmii";
        clock_in_out = "output";

        snps,reset-gpio = <&gpio0 RK_PC4 GPIO_ACTIVE_LOW>;
        snps,reset-active-low;
        snps,reset-delays-us = <0 20000 100000>;

        pinctrl-names = "default";
        pinctrl-0 = <&eth_rmii1_miim_pins
                     &eth_rmii1_tx_bus2_pins
                     &eth_rmii1_rx_bus2_pins
```

```
                        &eth_rmii1_clk_pins>;

        phy-handle = <&rmii_phy1>;
        status = "okay";
};
```

&gmac1：对应网口 1。

snps,reset-gpio = <&gpio0 RK_PC4 GPIO_ACTIVE_LOW>;：网口 1 复位引脚，低电平有效。

pinctrl-0：这里指定了连接网络芯片的控制线，网络传输线，网络时钟。

phy-handle = <&rmii_phy1>;：匹配对应的网络驱动。

在 kernel/arch/arm/boot/dts/rk3506.dtsi 下有原厂提供的网口基础配置：

```
gmac1: ethernet@ff4d0000 {
    compatible = "rockchip,rk3506-gmac", "snps,dwmac-4.20a";
    reg = <0xff4d0000 0x2000>;
    interrupts = <GIC_SPI 70 IRQ_TYPE_LEVEL_HIGH>,
            <GIC_SPI 73 IRQ_TYPE_LEVEL_HIGH>;
    interrupt-names = "macirq", "eth_wake_irq";
    rockchip,grf = <&grf>;
    clocks = <&cru CLK_MAC1>, <&cru CLK_MAC1_PTP>,
        <&cru PCLK_MAC1>, <&cru ACLK_MAC1>;
    clock-names = "stmmaceth", "ptp_ref",
            "pclk_mac", "aclk_mac";
    resets = <&cru SRST_A_MAC1>;
    reset-names = "stmmaceth";

    snps,mixed-burst;
    snps,tso;

    snps,axi-config = <&gmac1_stmmac_axi_setup>;
    snps,mtl-rx-config = <&gmac1_mtl_rx_setup>;
    snps,mtl-tx-config = <&gmac1_mtl_tx_setup>;

    phy-mode = "rmii";
    status = "disabled";

    mdio1: mdio {
        compatible = "snps,dwmac-mdio";
        #address-cells = <0x1>;
        #size-cells = <0x0>;
    };

    gmac1_stmmac_axi_setup: stmmac-axi-config {
```

```
32              snps,wr_osr_lmt = <4>;
33              snps,rd_osr_lmt = <8>;
34              snps,blen = <0 0 0 0 16 8 4>;
35          };
36
37          gmac1_mtl_rx_setup: rx-queues-config {
38              snps,rx-queues-to-use = <1>;
39              queue0 {
40                  status = "okay";
41              };
42          };
43
44          gmac1_mtl_tx_setup: tx-queues-config {
45              snps,tx-queues-to-use = <1>;
46              queue0 {
47                  status = "okay";
48              };
49          };
50  };
```

ethernet@ff4d0000：设备类型为以太网控制器，寄存器基地址为 0xff4d0000。

interrupts，interrupt-names：一些中断配置。

7-12 行：时钟和一些复位配置。

mdio1：MDIO 总线子节点，用于管理连接的 PHY 设备。

gmac1_stmmac_axi_setup：以太网控制器的 AXI 接口配置。

gmac1_mtl_rx_setup：接收队列。

gmac1_mtl_tx_setup：发送队列。

12.7 ETH 工具

12.7.1 dmesg

Linux 内核通过串口输出日志消息，可以使用 dmesg 命令来查看这些消息。这对于调试引导过程中的问题非常有用，也可以用于检查内核模块加载和设备初始化过程中的消息。grep 是一个 Linux/Unix 下的命令行工具，用于在文本文件中搜索指定的字符串模式，并将包含匹配的行输出到标准输出（通常是终端）。

示例：

```
dmesg | grep eth0
```

```
root@ATK-DLRK3506:/# dmesg | grep eth0
[    7.766160] rk_gmac-dwmac ff4c8000.ethernet eth0: Register MEM_TYPE_PAGE_POOL RxQ-0
[    7.771056] rk_gmac-dwmac ff4c8000.ethernet eth0: PHY [stmmac-0:01] driver [YT8512 100M Ethernet] (i
rq=POLL)
[    7.771706] rk_gmac-dwmac ff4c8000.ethernet eth0: No Safety Features support found
[    7.771746] rk_gmac-dwmac ff4c8000.ethernet eth0: IEEE 1588-2008 Advanced Timestamp supported
[    7.773103] rk_gmac-dwmac ff4c8000.ethernet eth0: registered PTP clock
[    7.777824] rk_gmac-dwmac ff4c8000.ethernet eth0: configuring for phy/rmii link mode
[148318.220454] rk_gmac-dwmac ff4c8000.ethernet eth0: Link is Up - 100Mbps/Full - flow control off
root@ATK-DLRK3506:/#
```

图：eth0 的打印信息

根据 dmesg 输出的信息，可以确认 eth0 接口已经正常工作，并连接到网络，这表示以太网设备 eth0 已注册并正常工作。

```
rk_gmac-dwmac ff4c8000.ethernet eth0: Link is Up - 100Mbps/Full - flow control off
```

表示 eth0 接口已经连接，并且连接速度为 100Mbps，即百兆，开发板网口是百兆的，需要通过网线接到对应的百兆设备端口。

12.7.2 ifconfig

ifconfig（interface configuration）指令是用于配置网络接口和查看网络接口状态的工具。它通常用于显示或修改网络接口的配置参数，例如 IP 地址、子网掩码、广播地址等。

开发板网口 ETH0 通过网线接到路由器接口后，会自动获取 IP 地址，此时执行 ifconfig 指令可以看到对应的网络接口信息，参考如下：

```
ifconfig
```

```
root@ATK-DLRK3506:/# ifconfig
eth0      Link encap:Ethernet  HWaddr FA:A3:84:57:8A:01
          inet addr:192.168.6.191  Bcast:192.168.6.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:320857 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2039 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:21448236 (20.4 MiB)  TX bytes:297396 (290.4 KiB)
          Interrupt:48

eth1      Link encap:Ethernet  HWaddr FE:A3:84:57:8A:01
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:331 errors:0 dropped:0 overruns:0 frame:0
          TX packets:75 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:72284 (70.5 KiB)  TX bytes:7354 (7.1 KiB)
          Interrupt:50 Base address:0xc000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:64824 errors:0 dropped:0 overruns:0 frame:0
          TX packets:64824 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:3242313 (3.0 MiB)  TX bytes:3242313 (3.0 MiB)

wlan0     Link encap:Ethernet  HWaddr 48:8F:4C:6F:E8:C0
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

root@ATK-DLRK3506:/#
```

图：ifconfig 信息

- eth0：这是网络接口的名称，这里是 gmac0。
- Link encap:Ethernet：表示这个接口使用的是以太网封装。
- HWaddr FA:A3:84:57:8A:01：这是接口的硬件地址，也就是 MAC 地址。MAC 地址是全球唯一的，用于在物理网络层标识网络接口。
- inet addr: 192.168.6.191：这是接口的 IPv4 地址。在这个例子中，接口的 IP 地址是 192.168.6.35(按路由器分配的实际 IP 为主)。
- Bcast:192.168.6.255：这是广播地址。在这个子网中，所有发送到 192.168.6.255 的数据包都会被该子网内的所有设备接收。
- Mask:255.255.255.0：这是网络掩码，用于区分网络地址和主机地址。在这个例子中，掩码表示网络地址是前 24 位，剩下的 8 位用于主机地址。
- Interrupt:48：这是分配给该网络接口的中断号。

12.7.3 udhcpc

udhcpc 是一个用于嵌入式系统的非常小的 DHCP 客户端。它的主要功能是向 DHCP 服务器请求网络配置参数，如 IP 地址、子网掩码和网关等。在使用指令之前，确保开发板网口网线连接到 DHCP 服务器，比如路由器接口。

查看 udhcp 使用方法：

```
udhcpc --help
```

```
root@ATK-DLRK3506:/# udhcpc --help
BusyBox v1.36.1 (2025-03-18 12:21:55 CST) multi-call binary.

Usage: udhcpc [-fbqRB] [-a[MSEC]] [-t N] [-T SEC] [-A SEC|-n]
        [-i IFACE] [-s PROG] [-p PIDFILE]
        [-oC] [-r IP] [-V VENDOR] [-F NAME] [-x OPT:VAL]... [-O OPT]...

        -i IFACE        Interface to use (default eth0)
        -s PROG         Run PROG at DHCP events (default /usr/share/udhcpc/default.script)
        -p FILE         Create pidfile
        -B              Request broadcast replies
        -t N            Send up to N discover packets (default 3)
        -T SEC          Pause between packets (default 3)
        -A SEC          Wait if lease is not obtained (default 20)
        -b              Background if lease is not obtained
        -n              Exit if lease is not obtained
        -q              Exit after obtaining lease
        -R              Release IP on exit
        -f              Run in foreground
        -S              Log to syslog too
        -a[MSEC]        Validate offered address with ARP ping
        -r IP           Request this IP address
        -o              Don't request any options (unless -O is given)
        -O OPT          Request option OPT from server (cumulative)
        -x OPT:VAL      Include option OPT in sent packets (cumulative)
                        Examples of string, numeric, and hex byte opts:
                        -x hostname:bbox - option 12
                        -x lease:3600 - option 51 (lease time)
                        -x 0x3d:0100BEEFC0FFEE - option 61 (client id)
                        -x 14:'"dumpfile"' - option 14 (shell-quoted)
        -F NAME         Ask server to update DNS mapping for NAME
        -V VENDOR       Vendor identifier (default 'udhcp VERSION')
        -C              Don't send MAC as client identifier
Signals:
        USR1    Renew lease
        USR2    Release lease
root@ATK-DLRK3506:/#
```

图：udhcp 使用说明

基本用法：

```
udhcpc [选项]
```

选项解释

- -f：前台运行，显示日志信息。
- -b：在后台运行（默认行为）。
- -q：快速模式，仅进行一次 DHCP 请求，然后退出。
- -R：禁用对 DNS 选项的请求。
- -B：禁用广播。
- -a [MSEC]：设置 DHCP ACK 的超时时间（默认超时时间单位为毫秒）。
- -t N：设置发送 DHCP DISCOVER 请求的次数。
- -T SEC：设置重试发送 DHCP DISCOVER 请求的间隔时间（单位为秒）。
- -A SEC：设置在无响应情况下，程序退出前的最大等待时间（单位为秒）。
- -n：在失败时不进行后台重试，立即退出。
- -i IFACE：指定网络接口。

- -s PROG：指定处理 DHCP 事件的脚本。
- -p PIDFILE：指定 PID 文件的位置。
- -o：在没有成功获取租约时退出。
- -C：忽略缓存的 DHCP 文件。
- -r IP：请求特定的 IP 地址。
- -V VENDOR：设置供应商类标识符。
- -F NAME：设置主机名。
- -x OPT:VAL：传递其他 DHCP 选项和值。
- -O OPT：请求特定的 DHCP 选项。

示例：获取 IP 地址，前提是接在网口的网线另外一端是连着路由器或者交换机。

```
udhcpc -i eth0
```

```
root@ATK-DLRK3506:/# udhcpc -i eth0
udhcpc: started, v1.36.1
udhcpc: broadcasting discover
udhcpc: broadcasting select for 192.168.6.191, server 192.168.6.1
udhcpc: lease of 192.168.6.191 obtained from 192.168.6.1, lease time 300
deleting routers
route: SIOCADDRT: Network is unreachable
adding dns 192.168.6.1
root@ATK-DLRK3506:/#
```

图：udhcpc 获取 IP 地址

12.7.4 ping

我们可以执行 ping 指令测试网络能否正常通信

```
ping 192.168.6.1        //ping 网关，根据自己实际网关填写
ping www.baidu.com      //ping 百度，需要自己的网络本身能连接外网，搭建的局域网不行
```

```
root@ATK-DLRK3506:/# ping 192.168.6.1
PING 192.168.6.1 (192.168.6.1): 56 data bytes
64 bytes from 192.168.6.1: seq=0 ttl=128 time=1.270 ms
64 bytes from 192.168.6.1: seq=1 ttl=128 time=1.258 ms
64 bytes from 192.168.6.1: seq=2 ttl=128 time=1.054 ms
64 bytes from 192.168.6.1: seq=3 ttl=128 time=1.014 ms
^Z
[1]+  Stopped(SIGTSTP)        ping 192.168.6.1
root@ATK-DLRK3506:/# ping www.baidu.com
PING www.baidu.com (103.235.46.102): 56 data bytes
64 bytes from 103.235.46.102: seq=1 ttl=48 time=212.335 ms
64 bytes from 103.235.46.102: seq=2 ttl=48 time=213.695 ms
64 bytes from 103.235.46.102: seq=3 ttl=48 time=216.178 ms
64 bytes from 103.235.46.102: seq=4 ttl=48 time=212.800 ms
^Z
[2]+  Stopped(SIGTSTP)        ping www.baidu.com
root@ATK-DLRK3506:/#
```

图：网络测试

第十三章 Audio 子系统

13.1 Audio 子系统简介

嵌入式 Linux 的音频子系统（Audio Subsystem）是一个复杂而强大的系统，用于管理和控制音频硬件。它提供了必要的接口和驱动程序，支持音频数据的捕获、回放、混音等操作。

13.2 组成部分

嵌入式 Linux 的音频子系统由多个部分组成，每个部分都有其特定的功能和角色，以协同工作来实现音频管理和控制。以下是嵌入式 Linux 音频子系统的主要组成部分：

1、ALSA（Advanced Linux Sound Architecture）

ALSA 核心

- PCM（Pulse Code Modulation）接口：用于处理音频数据的捕获和回放。
- 控制接口：用于设置和获取音频设备的参数。
- Mixer 接口：用于音量控制和音频信号的混合。

用户空间库

- libasound：提供高级 API，使应用程序可以方便地进行音频操作。

2、音频驱动程序

平台驱动（Platform Driver）

- 负责与特定硬件平台相关的音频设备初始化和控制。

Codec 驱动（Codec Driver）

- 控制音频编解码器，处理音频信号的转换和处理。

机器驱动（Machine Driver）

- 描述平台和 Codec 之间的连接和交互。

3、设备树（Device Tree）

- 描述硬件布局：用于描述系统中音频硬件的配置和连接。
- 配置参数：提供驱动程序初始化所需的配置参数。

4、音频框架

ASoC（ALSA System on Chip）

- 专为嵌入式系统设计的音频框架，简化了复杂音频硬件的配置。

5、用户空间应用

命令行工具

- aplay/arecord：用于音频数据的播放和录制。
- alsamixer：基于命令行的音量控制和混音工具。
- amixer：命令行音频控制工具。

6、Use Case Manager（UCM）

- 管理使用场景：提供不同音频使用场景的灵活配置和优化。

7、调试和开发工具

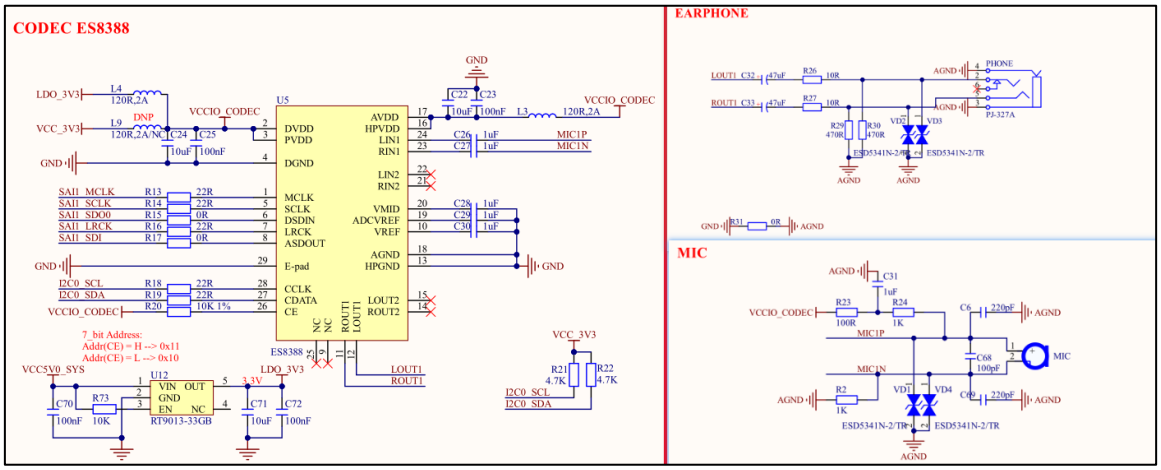
- 调试工具：如 alsa-info.sh 用于收集系统音频信息，alsabat 用于音频环回测试。

通过以上各组成部分，嵌入式 Linux 音频子系统能够提供灵活、强大且高效的音频管理功能，支持多种音频硬件和复杂的音频应用需求。

13.3 开发板 Audio 接口

13.3.1 原理图分析

开发板上有一个音频芯片 ES8388。打开开发板上开发板网盘 A 盘-基础资料\02、原理图\01、底板原理图\ ATK-DLRK3506B V1.0.pdf，搜索 ES8388。可以看到相关电路如下。



图：开发板音频电路原理图

音频相关原理图共分为 CODEC ES8388、EARPHONE 和 MIC 三部分：

1、 ES8388 部分：电路图显示了 ES8388 音频编解码器芯片及其连接的外围电路。

- 电源引脚
  - VCCIO_DVDD：提供 3.3V 电源
  - DVDD：数字电源，连接到 3.3V
  - PVDD：PLL 电源，连接到 3.3V
  - AVDD：模拟电源，连接到 3.3V
  - HPVDD：耳机放大器电源，连接到 3.3V
- 接地引脚
  - DGND：数字地
  - AGND：模拟地
  - HPGND：耳机放大器地
- 音频接口
  - I2S 接口
    - SAI1_MCLK：主时钟输入
    - SAI1_SCLK：发送时钟
    - SAI1_SDO0：发送数据
    - SAI1_LRCK： 接收时钟
    - SAI1_SDI：接收数据
  - I2C0 接口
    - I2C0_SCL：I2C 时钟线
    - I2C0_SDA：I2C 数据线
- 音频输入
  - LINE 输入
    - LIN1：线性输入 1，连接到外部音频信号

- LIN2：线性输入 2，连接到外部音频信号

- MIC 输入
  - MIC2P：麦克风正输入
  - MIC2N：麦克风负输入
- 音频输出
  - LOUT1 和 ROUT1：音频输出 1，用于驱动耳机或扬声器
  - LOUT2 和 ROUT2：音频输出 2，用于驱动耳机或扬声器
- 其他功能引脚
  - CCLK：控制时钟
  - CDATA：控制数据
  - CE：芯片使能信号

通过这些连接和元件，ES8388 编解码器能够处理从模拟音频信号到数字音频信号的转换，支持多种音频输入和输出设备，如麦克风、耳机和扬声器。

2、EARPHONE 部分：电路图展示了耳机接口及其相关电路，主要用于音频信号的输出和麦克风信号的输入。

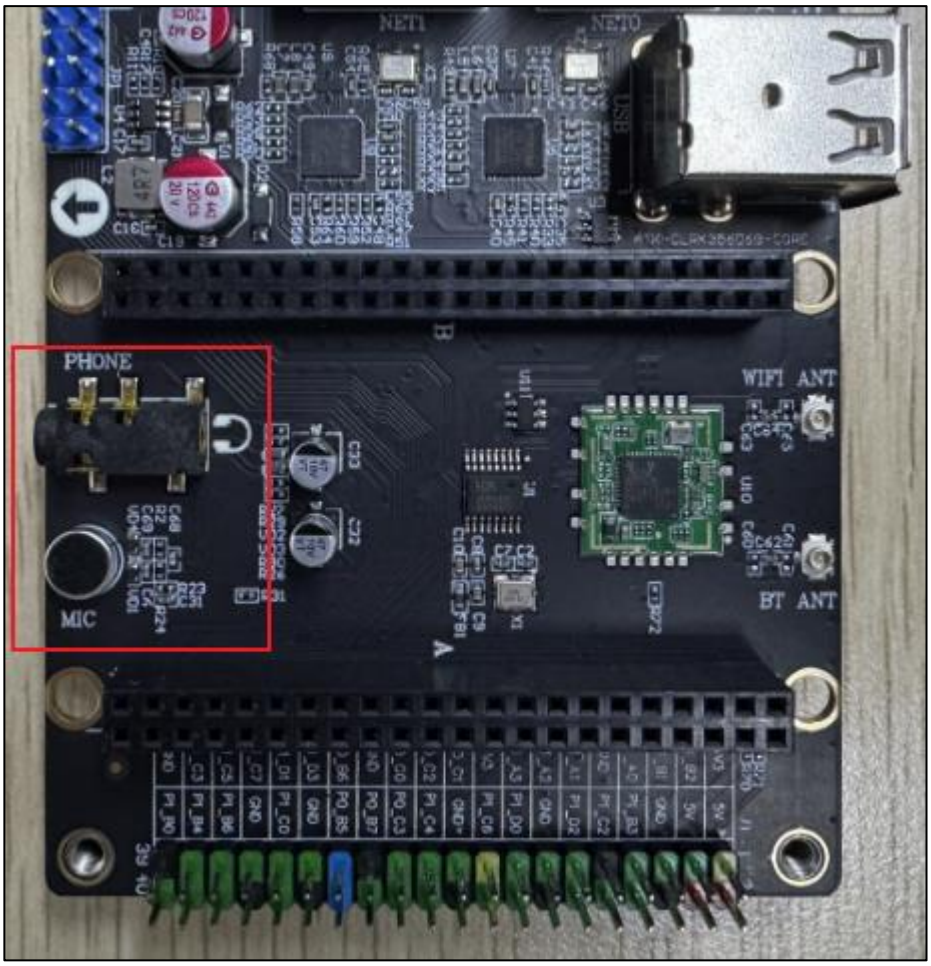
- 音频信号
  - LOUT1：左声道音频输出
  - ROUT1：右声道音频输出

3、MIC 部分：电路图展示了麦克风接口及其相关电路，用于处理麦克风的音频信号输入。

- 主要组件
  - 电源
    - VCCIO_CODEC：提供 3.3V 电源
  - 麦克风信号
    - MIC：麦克风接口，包括正（MIC1P）和负（MIC1N）信号
- 总结
  - 电源和偏置电路：通过电阻和电容提供稳定的电源和正确的偏置电压，确保麦克风正常工作
  - 麦克风信号处理电路：通过滤波电容去除高频噪声，通过 ESD 保护二极管防止静电放电损害电路
  - 信号处理电阻：用于进一步处理麦克风信号，具体功能取决于其阻值
  - 这个电路设计确保了麦克风信号的稳定输入，并通过多重 ESD 保护措施，防止静电放电对电路的损害，保证音频信号的质量

13.3.2 硬件实物

开发板上的 Audio 相关的电路位置如下图所示：



图：电路实物

开发板板载 1 个耳机接口。耳机座规格 3.5mm 的耳机，支持放音。放音时，支持耳机热插拔。

开发板板载 1 个驻极体麦克风，即录音输入口（MIC），可以用来实现录音功能。

顺芯 ES8388 是一款高性能、低功耗、低成本的立体声音频编解码器，该芯片内部集成了 24 位高性能 ADC 和 DAC，由双通道 ADC、双通道 DAC、麦克风放大器、耳机放大器、数字音效以及模拟混音和增益功能组成，支持 8KHZ~96KHz 采样率，可通过 I2C 接口进行配置。

13.4 内核 Audio 相关使能

内核源码配置文件 kernel/arch/arm/configs/rk3506_defconfig 中，Audio 相关配置如下(略有删减，只列举重点部分。)：

```
1  CONFIG_SOUND=y     // 启用内核中的声音系统支持。
2  CONFIG_SND=y      // 启用高级 Linux 声音系统（ALSA）支持。
3  CONFIG_SND_SOC=y      // 启用 Linux 音频子系统（ALSA SoC）支持。
4  CONFIG_SND_SOC_ROCKCHIP=y // 启用 RK 处理器上的 SoC 音频支持。
```

13.5 设备树分析

打开 kernel/arch/arm/boot/dts/rk3502-evb1-v10.dtsi，中关于 Audio 描述如下：

```
1   es8388_sound: es8388-sound {
```

```
2       status = "disabled";
3       compatible = "rockchip,multicodecs-card";
4       rockchip,card-name = "rockchip-es8388";
5       spk-con-gpio = <&gpio1 RK_PC6 GPIO_ACTIVE_HIGH>;
6       rockchip,pre-power-on-delay-ms = <30>;
7       rockchip,post-power-down-delay-ms = <40>;
8       rockchip,format = "i2s";
9       rockchip,mclk-fs = <256>;
10      rockchip,cpu = <&sai1>;
11      rockchip,codec = <&es8388>;
12      rockchip,audio-routing =
13          "Speaker", "LOUT1",
14          "Speaker", "ROUT1",
15          "Speaker", "Speaker Power",
16          "Speaker", "Speaker Power",
17          "LINPUT1", "Main Mic",
18          "LINPUT2", "Main Mic",
19          "RINPUT1", "Main Mic",
20          "RINPUT2", "Main Mic";
21      pinctrl-names = "default";
22      pinctrl-0 = <&spk_ctrl>;
23  };
```

compatible = "rockchip,multicodecs-card";:用于内核在驱动程序和设备之间进行匹配。对应的驱动在 kernel/sound/soc/rockchip/rockchip_multicodecs.c。

rockchip,card-name = "rockchip-es8388";：用于指定该音频设备的名称。

rockchip,pre-power-on-delay-ms：自定义属性，用于指定设备上电前的延迟时间。

rockchip,post-power-down-delay-ms：自定义属性，用于指定设备下电后的延迟时间。

rockchip,format：自定义属性，用于指定音频数据的传输格式。

rockchip,codec：自定义属性，用于指定音频编解码器

12-20 行：用于指定音频信号的路由。

13.6 ALSA 工具

13.6.1 alsamixer

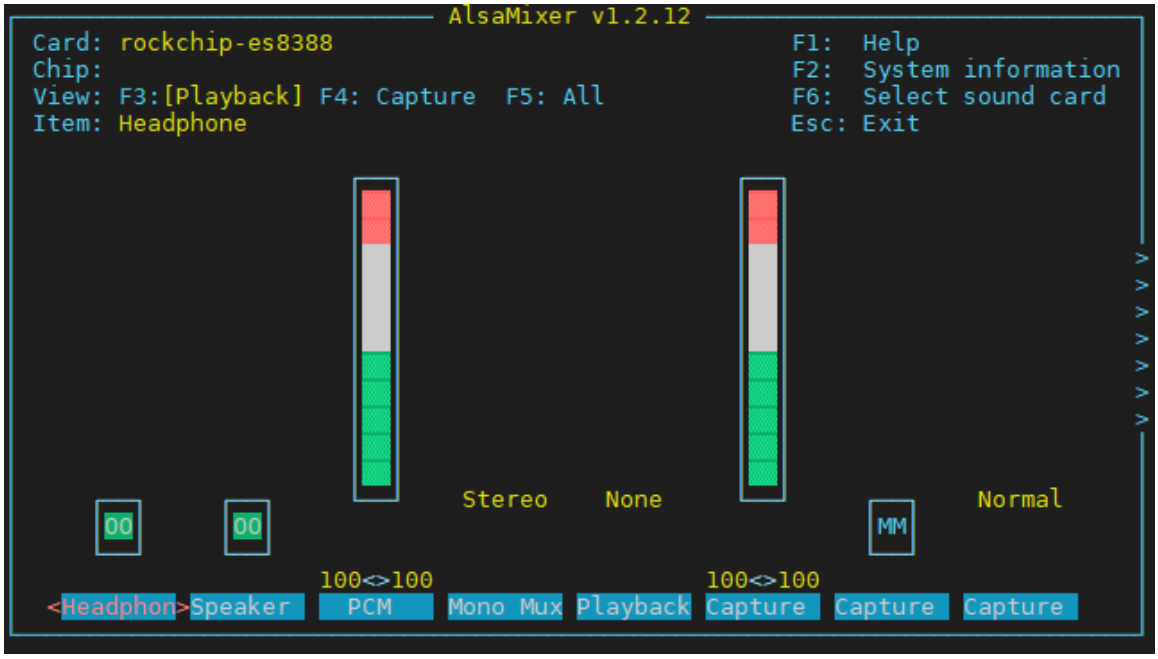
alsamixer 是 ALSA 的混音器控制工具，可以在命令行中提供一个图形化界面，用于调整音量和配置音频设备。下面是如何使用 alsamixer 的一些基本示例：

启动 alsamixer

在终端中运行以下命令来启动 alsamixer：

```
alsamixer
```

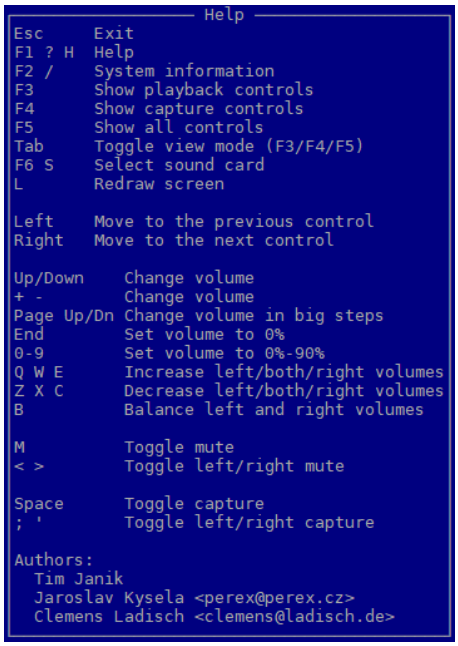
此时会看到一个带有音量控制条的图形化界面，通过键盘可以进行导航和调整。



图：alsamixer 界面

基本键盘操作

- 方向键左/右：选择不同的控制条（例如音量、麦克风增益等）。
- 方向键上/下：调整选定控制条的音量。
- F4：显示录音控制，可能会有键位冲突。
- F5：显示所有控制，可能会有键位冲突。
- Esc：退出 alsamixer。



图：帮助信息

13.6.2 amixer

amixer 是 Linux 系统中的一个命令行工具，用于配置和控制高级 Linux 声音架构 (ALSA) 的音频设备。它能够显示和修改混音器控件，包括音量、音频通道的选择、静音控制以及高级音频设置。

开发板执行指令：

```
amixer
```

```
[root@ATK-DL2K0300 ~]# amixer
Simple mixer control 'PCM',0
  Capabilities: volume
  Playback channels: Front Left - Front Right
  Capture channels: Front Left - Front Right
  Limits: 0 - 192
  Front Left: 169 [88%] [-11.50dB]
  Front Right: 169 [88%] [-11.50dB]
Simple mixer control 'Mic PGA',0
  Capabilities: volume
  Playback channels: Front Left - Front Right
  Capture channels: Front Left - Front Right
  Limits: 0 - 8
  Front Left: 8 [100%] [24.00dB]
  Front Right: 8 [100%] [24.00dB]
Simple mixer control 'Capture Digital',0
  Capabilities: volume
  Playback channels: Front Left - Front Right
  Capture channels: Front Left - Front Right
  Limits: 0 - 192
  Front Left: 192 [100%] [0.00dB]
  Front Right: 192 [100%] [0.00dB]
Simple mixer control 'Capture Polarity',0
  Capabilities: enum
  Items: 'Normal' 'L Invert' 'R Invert' 'L + R Invert'
  Item0: 'R Invert'
Simple mixer control 'Capture ZC',0
  Capabilities: pswitch pswitch-joined
  Playback channels: Mono
  Mono: Playback [on]
Simple mixer control 'DAC Deemphasis',0
  Capabilities: pswitch pswitch-joined
  Playback channels: Mono
  Mono: Playback [on]
Simple mixer control 'Differential Mux',0
  Capabilities: enum
  Items: 'Line 1' 'Line 2'
  Item0: 'Line 2'
Simple mixer control 'Left Line Mux',0
  Capabilities: enum
  Items: 'Line 1' 'Line 2' 'PGA' 'Differential'
  Item0: 'Line 2'
Simple mixer control 'Left Mixer',0
  Capabilities: pswitch pswitch-joined
  Playback channels: Mono
  Mono: Playback [on]
以下省略
```

图：amixer 输出信息

从 amixer 输出来看，这是一段嵌入式系统中音频设备的混音器配置。具体信息包括音频通道的配置、增益控制、捕捉和播放的音量设置等。这些配置控制了音频系统的不同部分，如立体声、单声道切换，3D 效果模式，以及自动电平控制 (ALC) 等功能。

13.7 音频测试

ATK-DLRK3506B 开发板板载一颗高性能音频编解码芯片 ES8388。板载麦克风可录音，可以接 3.5mm 耳机孔播放声音。

13.7.1 播放音频测试

用系统指令 aplay 可以播放音频文件。插上耳机或者听底板上的喇叭有没有声音。

注意：戴耳机时先靠近耳朵旁听，不要直接戴上，防止耳机可能音量过大，影响体验。

播放出厂系统自带音频测试文件，也可更换音频文件进行测试。按 Ctrl+c 终止指令。

```
gst-play-1.0 /userdata/piano2-CoolEdit.mp3
```

```
root@ATK-DLRK3506:/# gst-play-1.0 /userdata/piano2-CoolEdit.mp3
Press 'k' to see a list of keyboard shortcuts.
Now playing /userdata/piano2-CoolEdit.mp3
Redistribute latency...
Redistribute latency...
0:00:06.3 / 0:00:06.3
Reached end of play list.

root@ATK-DLRK3506:/#
```

图：播放音频文件

13.7.2 录音测试

ATK-DLRK3506B 底板上有一个是麦克风 MIC，可以录制声音。

录音，执行下面的指令。(对准开发板上的麦克风 MIC 喊话)

```
amixer -c 0 cset name='ADC Data Select' 'Right Right'        #声卡的一些配置
amixer -c 0 cset name='ALC Capture Max PGA' '7'
amixer -c 0 cset name='ALC Capture Min PGA' '3'
amixer -c 0 cset name='ALC Capture Function' 'Stereo'

arecord -f S16_LE -c 2 -d 5 record.wav -D plughw:0,0         #开始录音
```

```
root@ATK-DLRK3506:/userdata# amixer -c 0 cset name='ADC Data Select' 'Right Right'
numid=61,iface=MIXER,name='ADC Data Select'
  ; type=ENUMERATED,access=rw------,values=1,items=4
  ; Item #0 'Left Right'
  ; Item #1 'Left Left'
  ; Item #2 'Right Right'
  ; Item #3 'Right Left'
  : values=2
root@ATK-DLRK3506:/userdata# amixer -c 0 cset name='ALC Capture Max PGA' '7'
numid=39,iface=MIXER,name='ALC Capture Max PGA'
  ; type=INTEGER,access=rw------,values=1,min=0,max=7,step=0
  : values=7
root@ATK-DLRK3506:/userdata# amixer -c 0 cset name='ALC Capture Min PGA' '3'
numid=40,iface=MIXER,name='ALC Capture Min PGA'
  ; type=INTEGER,access=rw------,values=1,min=0,max=7,step=0
  : values=3
root@ATK-DLRK3506:/userdata# amixer -c 0 cset name='ALC Capture Function' 'Stereo'
numid=41,iface=MIXER,name='ALC Capture Function'
  ; type=ENUMERATED,access=rw------,values=1,items=4
  ; Item #0 'Off'
  ; Item #1 'Right'
  ; Item #2 'Left'
  ; Item #3 'Stereo'
  : values=3
root@ATK-DLRK3506:/userdata# arecord -f S16_LE -c 2 -d 5 record.wav -D plughw:0,0
Recording WAVE 'record.wav' : Signed 16 bit Little Endian, Rate 8000 Hz, Stereo
root@ATK-DLRK3506:/userdata#
```

图：录音测试

录制的音频保存在当前文件夹下：

```
root@ATK-DLRK3506:/userdata# ls
200frames_count.h264          lib                          record.wav
belle-nuit-testchart-1080p.png  piano2-CoolEdit.mp3        ui
root@ATK-DLRK3506:/userdata#
```

图：录音文件

可以执行 gst-play-1.0 record.wav 播放录音。

第十四章 WiFi 子系统

14.1 WiFi 子系统简介

嵌入式 Linux 的 WiFi 子系统主要负责无线网络的管理和通信。它由多个组件和框架组成，提供驱动程序支持、网络配置管理、以及用户空间和内核空间之间的通信接口

14.2 组成部分

嵌入式 Linux 的 WiFi 子系统由多个关键组件组成，每个组件都在 WiFi 通信和管理过程中扮演着重要角色。以下是这些组成部分的详细介绍：

1、无线驱动程序（Wireless Drivers）

mac80211:这是 Linux 内核中的一个通用的 WiFi 协议栈，支持大多数现代 WiFi 芯片。它处理与无线通信相关的各种任务，例如帧管理、数据传输、以及协议实现。

cfg80211:这是 mac80211 的配置接口层，它为用户空间提供一个标准化的 API，用于配置和管理 WiFi 设备。cfg80211 与 mac80211 协作，处理扫描、连接、断开等操作。

硬件特定驱动程序:这些驱动程序直接与 WiFi 芯片硬件交互，通常由芯片制造商提供，并使用 mac80211/cfg80211 框架来实现硬件功能。

2、内核接口

nl80211:这是一个 netlink 接口，提供用户空间与内核 WiFi 子系统（通过 cfg80211 和 mac80211）的交互。通过 nl80211，用户空间程序可以请求执行诸如网络扫描、连接到网络、设置无线电信道等操作。

3、用户空间工具

wpa_supplicant:这是一个广泛使用的用户空间守护程序，用于处理 WiFi 认证和加密（如 WPA/WPA2）。它通过 nl80211 与内核交互，实现网络连接和安全协议管理。

iw:这是一个命令行工具，使用 nl80211 接口，允许用户直接配置和管理 WiFi 接口，执行扫描、连接和网络状态监控等任务。

4、网络管理框架

NetworkManager:这是一个常见的网络管理工具，自动管理有线和无线网络连接，适合桌面和嵌入式系统。它能够处理复杂的网络配置和连接切换。

systemd-networkd:这是 systemd 的一部分，用于网络配置和管理，支持静态 IP 配置和基本的 WiFi 连接管理，适合资源受限的嵌入式系统。

5、射频开关（RFKILL）

rfkill:这是一个内核子系统，用于管理无线设备的射频功能。它允许系统或用户关闭无线电发射以节省电力或遵守法规。

6、硬件抽象层

固件（Firmware）:WiFi 芯片通常需要加载特定的固件来启用完整功能，这些固件在启动时加载并运行于设备中，管理低级硬件操作。

驱动程序抽象:Linux 通过 mac80211 和 cfg80211 提供一个通用的接口，使得不同制造商的硬件能够在相同的框架下运作，简化了驱动开发和集成。

7、性能与功耗管理

嵌入式 WiFi 设备需要在有限的资源下优化性能和功耗。内核和驱动程序会使用各种技术，如省电模式（Power Save Mode，PSM）和动态频率调整，来减少电力消耗并延长电池寿命。

8、网络配置与管理

DHCP Client/Server:用于动态分配 IP 地址，通常在设备连接到无线网络时使用。

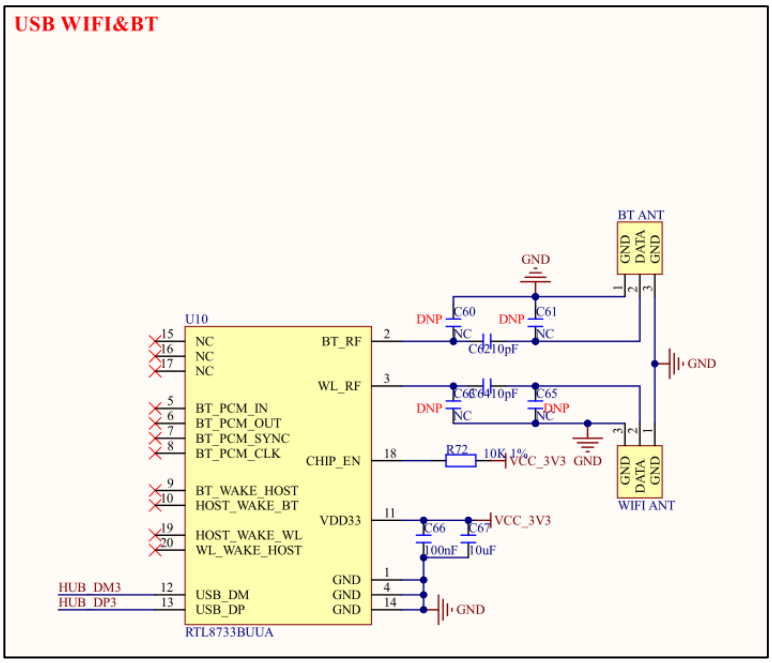
dnsmasq:这是一个轻量级的 DNS、DHCP、TFTP 服务，适用于嵌入式系统，用于提供本地网络服务。

这些组成部分共同协作，确保嵌入式 Linux 系统能够可靠且高效地管理 WiFi 连接，从硬件通信到用户空间的网络配置和管理，涵盖了整个网络栈。

14.3 开发板 WiFi 说明

14.3.1 原理图分析

开发板上有一个 WiFi&BT 芯片。打开开发板上开发板网盘 A 盘-基础资料\02、原理图\01、底板原理图\ ATK-DLRK3506B V1.0.pdf，搜索 WIFI。可以看到相关电路如下。



图：开发板 WIFI 电路

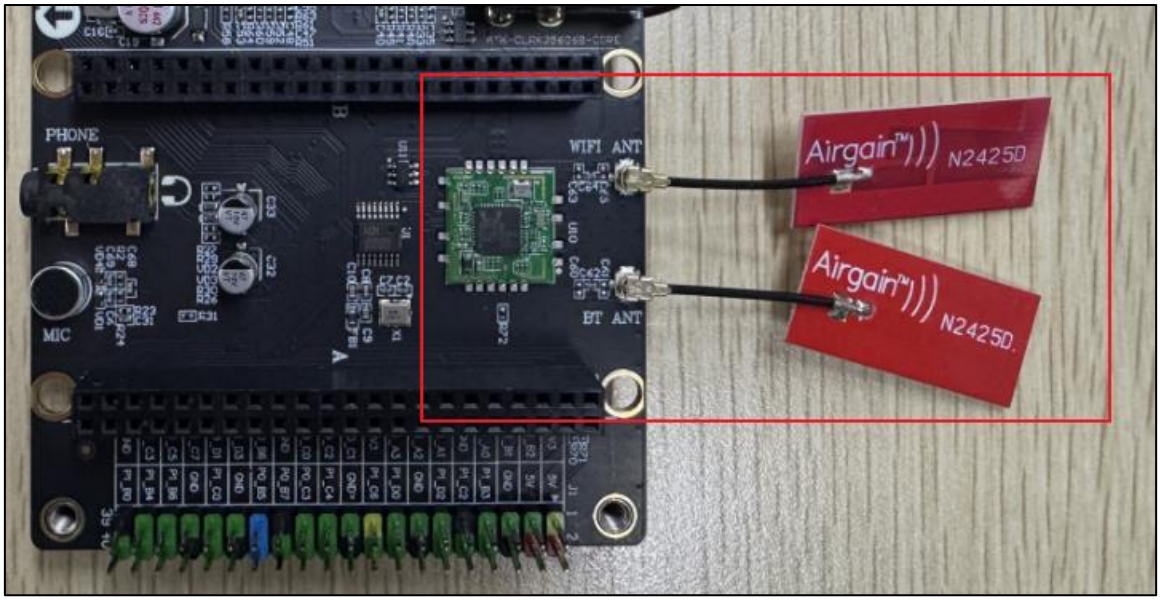
这个原理图展示了 USB WiFi 和蓝牙模块的电路连接，该模块是 Realtek 的 RTL8733BUUA。以下是对该电路的解析：

- 电源部分
  - VCC_3V3: 连接到 P3V3 电源给模块。
  - USB_DM 和 USB_DP: 连接的是由 USB20_OTG1_DP 和 USB20_OTG1_DM 通过 CH334R 扩展出来的 HUB_DM3 和 HUB_DP3 引脚。
- 天线部分
  - WL_RF: 这是 WiFi 天线接口，连接到天线（WIFI_ANT），用于射频信号的传输和接收

这个原理图展示了一个典型的 USB WiFi 和蓝牙模块的硬件设计，通过一系列电源和控制电路实现其功能。这个设计注重模块的低功耗操作和稳定性，适用于嵌入式系统的无线通信应用。

14.3.2 硬件实物

开发板上的 WiFi 相关的电路位置如下图所示：



图：开发板 WiFi 模块

14.4 内核 WiFi 相关使能

内核源码配置文件 kernel/arch/arm/configs/ rk3506-atk-dlrk3506.config 和 rk3506_defconfig 中，WiFi 相关配置如下：

```
CONFIG_RTL8733BU=m                  //编译 RTK8733 的驱动代码成模块
CONFIG_NETDEVICES=y                 //开启网络设备的支持
CONFIG_WLAN_VENDOR_REALTEK=y        //开启对 Realtek 厂商无线网卡的支持
```

对应的驱动源码包位于 kernel/drivers/net/wireless/realtek/rtl8733bu 目录下，8733bu.ko 是 RTL8733BU 对应的驱动模块。驱动模块已经安装在 Builroot 根文件系统 /usr/lib/modules/6.1.99+/kernel/drivers/net/wireless/realtek/rtl8733bu/8733bu.ko 下。

14.5 WiFi 工具

14.5.1 connmanctl

connmanctl 是 ConnMan（Connection Manager）的命令行界面，用于在 Linux 系统上管理网络连接。ConnMan 是一个网络连接管理器，专门为嵌入式设备设计，支持有线网络、Wi-Fi、蓝牙、蜂窝网络等。

开发板执行下面指令：

```
connmanctl
```

此时会进入 connmanctl 命令行终端，在终端中可以执行 connmanctl 相关指令操作，我们可以执行 help 指令查看 connmanctl 命令行终端支持的指令：

```
root@ATK-DLRK3506:/# connmanctl
Error getting VPN connections: The name net.connman.vpn was not provided by any .service filesconnmanc
connmanctl> help
state                                 Shows if the system is online or offline
technologies                          Display technologies
clock                                 Get System Clock Properties
enable          <technology>|offline  Enables given technology or offline mode
disable         <technology>|offline  Disables given technology or offline mode
tether          <technology> on|off
          wifi [on|off] <ssid> <passphrase> [<freq>] Enable, disable tethering, set SSID and passphr
ase for wifi
tethering_clients                      Display tethering clients
services        [<service>]           Display services
   --properties      [<service>]      (obsolete)
peers           [peer]                Display peers
scan            <technology>          Scans for new services for given technology
connect         <service/peer>        Connect a given service or peer
disconnect      <service/peer>        Disconnect a given service or peer
move-before     <service> <target service>      Move <service> before <target service>
move-after      <service> <target service>      Move <service> after <target service>
config          <service>             Set service configuration options
   --nameservers     <dns1> [<dns2>] [<dns3>]
   --timeservers     <ntp1> [<ntp2>] [...]
   --domains         <domain1> [<domain2>] [...]
   --mdns            yes|no
   --ipv6            off|auto [enable|disable|preferred]|
                        manual <address> <prefixlength> <gateway>
   --proxy           direct|auto <URL>|manual <URL1> [<URL2>] [...]
                        [exclude <exclude1> [<exclude2>] [...]]
   --autoconnect     yes|no
   --ipv4            off|dhcp|manual <address> <netmask> <gateway>
   --remove                           Remove service
monitor         [off]                 Monitor signals from interfaces
   --services        [off]            Monitor only services
   --tech            [off]            Monitor only technologies
   --manager         [off]            Monitor only manager interface
   --vpnmanager      [off]            Monitor only VPN manager interface
   --vpnconnection   [off]            Monitor only VPN connections
agent           on|off                Agent mode
```

图：connmanctl 支持的指令

一些关键命令的详细解释：

- state: 显示系统当前的联网状态（在线或离线）。
- technologies: 列出系统中可用的网络技术（如 Wi-Fi、Ethernet 等）。
- enable 和 disable: 启用或禁用指定的网络技术或将系统设置为离线模式。例如：
  - enable wifi 启用 Wi-Fi
  - disable wifi 禁用 Wi-Fi
- tether: 用于启用或禁用网络共享(热点)，以及设置 Wi-Fi 热点的 SSID 和密码。例如：
  - tether wifi on myssid mypassword 启用 Wi-Fi 热点并设置 SSID 为 myssid，密码为 mypassword。
- services: 列出当前可用的网络服务（例如，已扫描到的 Wi-Fi 网络）
- connect 和 disconnect: 连接或断开指定的网络服务。例如：
  - connect wifi_service 连接到指定的 Wi-Fi 服务
  - disconnect wifi_service 断开与指定 Wi-Fi 服务的连接
- config: 配置指定服务的网络参数，如 DNS、NTP 服务器、IPv4 和 IPv6 设置等。例如：
  - config wifi_service --ipv4 manual 192.168.1.10 255.255.255.0 192.168.1.1 手动配置指定 Wi-Fi 服务的 IPv4 地址
- scan: 扫描指定网络技术的新服务。例如
  - scan wifi 扫描可用的 Wi-Fi 网络
- monitor: 监控来自接口的信号，例如网络服务和技术的变化
- vpnconnections: 显示当前的 VPN 连接

WiFi连接示例：进行WiFi测试之前，请确保开发板上安装好天线，否则无法连接WiFi。进入connmanctl命令行后，输入下面命令进行为WiFi 测试。

```
enable wifi
scan wifi
agent on
services
```

```
connmanctl> enable wifi
wifi is already enabled
connmanctl> scan wifi
Scan completed for wifi
connmanctl> agent on
Agent registered
connmanctl> services
    ALIENTEK-linux       wifi_488f4c6fe8c0_414c49454e54454b2d6c696e7578_managed_psk
                         wifi_488f4c6fe8c0_52696368393031_managed_psk
                         wifi_488f4c6fe8c0_414c49454e54454b2d5758_managed_psk
                         wifi_488f4c6fe8c0_hidden_managed_psk
                         wifi_488f4c6fe8c0_41544b2d485541574549_managed_psk
                        N wifi_488f4c6fe8c0_4449524543542d48434445534b544f502d37413838354d566d7
34a
                       77 wifi_488f4c6fe8c0_4445534b544f502d4741534b414c502031313737_managed_psk
                        wifi_488f4c6fe8c0_414c49454e54454b2d37e69cbae688bf_managed_psk
                         wifi_488f4c6fe8c0_526963683930312d3547_managed_psk
                         wifi_488f4c6fe8c0_414c49454e54454b2d58485953_managed_psk
                         wifi_488f4c6fe8c0_414c49454e54454b2d57585f3547_managed_psk
                         wifi_488f4c6fe8c0_4368696e614e65742d59685544_managed_psk
                         wifi_488f4c6fe8c0_54502d4c494e4b5f44323035_managed_psk
                         wifi_488f4c6fe8c0_53594a593530302d3547_managed_psk
```

图：开启 WiFi 并扫描 WiFi

enable wifi

作用: 启用 Wi-Fi 技术。执行此命令后，系统会启动 Wi-Fi 硬件模块，使其能够扫描和连接无线网络。

scan wifi

作用: 扫描可用的 Wi-Fi 网络。此命令会搜索附近所有的 Wi-Fi 网络，并在列表中显示它们的服务标识符（SSID）。

agent on

作用: 启用代理模式。在启用代理后，系统会允许用户处理一些需要交互的操作，例如连接 Wi-Fi 时需要输入密码的情况。

services

作用: 列出当前检测到的服务（如 Wi-Fi 网络）。显示的结果包括每个网络的服务标识符（例如 SSID）和加密类型。

ALIENTEK-linux: 一个名为 ALIENTEK-YY 的 Wi-Fi 网络，使用 PSK 进行管理。

wifi_488f4c6fe8c0_414c49454e54454b2d6c696e7578_managed_psk: 这是一个隐藏的 Wi-Fi 网络，SSID 没有广播，因此显示为隐藏网络。

这些命令的组合用于启用 Wi-Fi，扫描可用的网络，并配置代理以便在连接过程中处理输入（如输入 Wi-Fi 密码）。当你看到可用的网络列表后，可以选择连接到其中的一个网络。

从扫描到的WIFI 热点列表中，使用命令connect XXXX 进行连接，再输入密码按回车确认。比如示例要连接ALIENTEK-linux，运行结果如下：

```
connect wifi_488f4c6fe8c0_414c49454e54454b2d6c696e7578_managed_psk \\此为示例，需修改。按实际为主
有 Passphrase?提示后输入 WiFi 密码
exit
```

```
connmanctl> connect wifi_488f4c6fe8c0_414c49454e54454b2d6c696e7578_managed_psk
Agent RequestInput wifi_488f4c6fe8c0_414c49454e54454b2d6c696e7578_managed_psk
  Passphrase = [ Type=psk, Requirement=mandatory, Alternates=[ WPS ] ]
  WPS = [ Type=wpspin, Requirement=alternate ]
Passphrase? 159
Connected wifi_488f4c6fe8c0_414c49454e54454b2d6c696e7578_managed_psk
connmanctl> exit
root@ATK-DLRK3506:/#
```

图：连接 WiFi

connect wifi_488f4c6fe8c0_414c49454e54454b2d6c696e7578_managed_psk

作用: 连接到指定的 Wi-Fi 网络，该网络的标识符为 wifi_f0a882f7cd09_414c49454e54454b2d6c696e7578_managed_psk。这个网络的 SSID 是 ALIENTEK-linux，并且使用预共享密钥（PSK）进行加密管理。

Agent RequestInput

作用: 代理请求输入连接该 Wi-Fi 网络所需的信息。在这种情况下，系统要求输入 Passphrase（密码短语），这是连接 Wi-Fi 所必需的。

Passphrase? 输入 WiFi 密码

作用: 输入 WiFi 密码短语，用于连接该 Wi-Fi 网络。

Connected wifi_488f4c6fe8c0_414c49454e54454b2d6c696e7578_managed_psk

作用: 表明连接成功，设备已连接到 SSID 为 ALIENTEK-linux 的 Wi-Fi 网络。

exit

作用: 退出 connmanctl 交互界面。

通过这些步骤，系统已经成功启用 Wi-Fi 模块，扫描网络，选择并连接到了指定的 Wi-Fi 网络。

退出 connmanctl 交互界面后，可以执行 ifconfig 指令查看 WiFi 获取到的 IP 地址。

```
root@ATK-DLRK3506:/# ifconfig
eth0      Link encap:Ethernet  HWaddr FA:A3:84:57:8A:01
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:48 Base address:0x4000

eth1      Link encap:Ethernet  HWaddr FE:A3:84:57:8A:01
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:50

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:21272 errors:0 dropped:0 overruns:0 frame:0
          TX packets:21272 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1064368 (1.0 MiB)  TX bytes:1064368 (1.0 MiB)

wlan0     Link encap:Ethernet  HWaddr 48:8F:4C:6F:E8:C0
          inet addr:192.168.3.190  Bcast:192.168.3.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:190 errors:0 dropped:21 overruns:0 frame:0
          TX packets:65 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:27903 (27.2 KiB)  TX bytes:8646 (8.4 KiB)

wlan1     Link encap:Ethernet  HWaddr 4A:8F:4C:6F:E8:C0
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

root@ATK-DLRK3506:/#
```

图：查看获取到的 IP 地址

ping 百度测试连通性，也可以 ping 网关来测试 WiFi 的连通性。注意，如果 ping 不通百度，请不要在开发板接入网线或者其他网络设备，因为系统默认会只让一个网卡连外网。

```
ping www.baidu.com -I wlan0
```

```
root@ATK-DLRK3506:/# ping www.baidu.com -I wlan0
PING www.baidu.com (183.2.172.17): 56 data bytes
64 bytes from 183.2.172.17: seq=0 ttl=51 time=6.221 ms
64 bytes from 183.2.172.17: seq=1 ttl=51 time=10.847 ms
64 bytes from 183.2.172.17: seq=2 ttl=51 time=9.939 ms
64 bytes from 183.2.172.17: seq=3 ttl=51 time=8.576 ms
^Z
[1]+  Stopped(SIGTSTP)        ping www.baidu.com -I wlan0
root@ATK-DLRK3506:/#
```

图：ping 百度测试

如果要删除掉已保存的 WiFi，在 connmanctl 命令行中执行下面操作：

显示已保存的服务（包括 Wi-Fi 网络）：

```
services
```

```
root@ATK-DLRK3506:/# connmanctl
Error getting VPN connections: The name net.connman.vpn was not provided by any .service filesconnmanc
connmanctl> services
*AR ALIENTEK-linux      wifi_488f4c6fe8c0_414c49454e54454b2d6c696e7578_managed_psk
                        wifi_488f4c6fe8c0_hidden_managed_psk
    ALIENTEK-WX         wifi_488f4c6fe8c0_414c49454e54454b2d5758_managed_psk
```

图：已保存的服务

在 connmanctl 中，*AR 是连接状态的缩写符号：

* 表示当前活跃的（正在使用的）连接。

A 表示这个网络已经被自动连接过。

R 表示该网络已被记住，即它是一个已保存的网络配置，设备会自动连接到该网络。

所以，*AR 表示该 Wi-Fi 网络是一个已保存的网络，并且目前正在被使用。

第十五章 Bluetooth 子系统

15.1 Bluetooth 子系统简介

嵌入式 Linux 的 Bluetooth 子系统是用于管理和操作蓝牙设备的一整套软件框架和驱动程序。这个子系统主要由蓝牙协议栈、驱动程序、用户空间工具和 API 组成，用于实现蓝牙通信和设备管理。

15.2 组成部分

1、蓝牙协议栈（Bluetooth Protocol Stack）

- HCI（Host Controller Interface）：
  提供主机与蓝牙控制器之间的通信接口。
  在内核中实现，管理底层硬件与协议栈的交互。
- L2CAP（Logical Link Control and Adaptation Protocol）：
  逻辑链路控制和适配协议，负责数据包的封装和适配。
  支持面向连接和无连接的服务。
- RFCOMM：
  提供串行端口仿真功能，常用于模拟 RS-232 串行通信。
- SCO（Synchronous Connection-Oriented）：
  用于传输同步音频数据，如语音通信。
- GATT（Generic Attribute Profile）：
  在低功耗蓝牙（BLE）中用于描述设备间的数据交换格式，支持发现和操作远程设备的服务。

2、蓝牙驱动程序

- HCI 驱动程序：

负责控制和管理与蓝牙硬件设备（如 USB、UART、SPI 等接口的蓝牙模块）的通信。
提供了对蓝牙控制器的初始化、配置、传输数据等功能。

- 蓝牙控制器驱动（Controller Drivers）：

特定于硬件的驱动程序，用于支持各种蓝牙芯片或模块。
例如，Broadcom、Qualcomm 等芯片的蓝牙驱动程序。

3、内核支持

- 蓝牙核心模块（Bluetooth Core Module）：

包括蓝牙协议栈在内核中的实现，如 HCI、L2CAP 等。
提供基本的蓝牙功能支持，包括设备发现、连接管理、数据传输等。

- 特定协议的内核模块：

支持各种蓝牙协议的内核模块，如 RFCOMM、SCO、A2DP（用于音频传输）等。

4、用户空间工具和 API

- BlueZ：

Linux 系统中的蓝牙协议栈实现，主要运行在用户空间。
提供了与内核蓝牙模块的接口，以及管理和操作蓝牙设备的工具和库。

- bluetoothd:

BlueZ 的守护进程，负责设备管理、服务发现、连接管理等核心功能。

- bluetoothctl:

交互式命令行工具，用于管理蓝牙设备，包括设备发现、配对、连接等。

- hcitool 和 hciconfig:

用于配置和管理 HCI 接口的工具，提供蓝牙设备的低级配置和控制。

- D-Bus API:

提供了蓝牙服务与应用程序之间的通信接口，使得用户空间应用可以通过 D-Bus 与蓝牙设备交互。

5、蓝牙管理和应用层

- 蓝牙设备发现和配对：

通过协议栈和工具实现设备发现、配对和连接管理。

- 服务发现和配置：

使用 GATT 和其他配置文件管理和操作蓝牙设备提供的服务。

6、应用示例

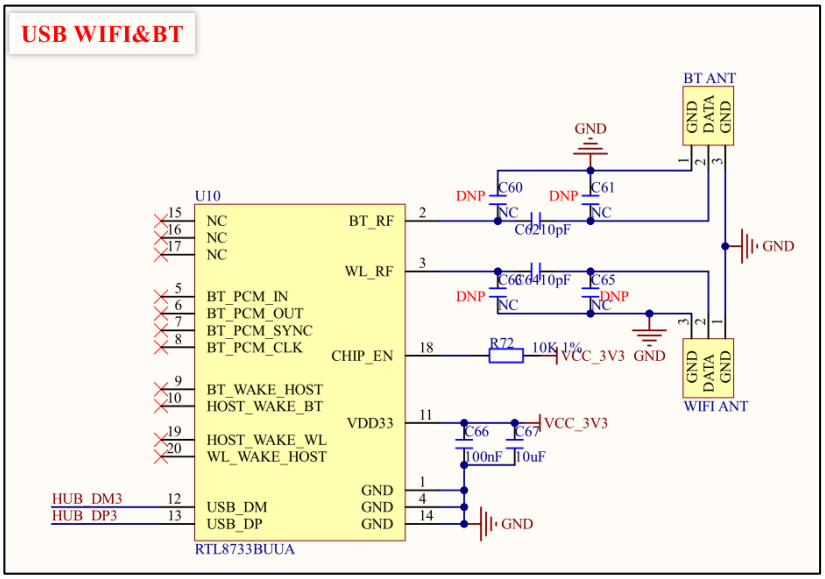
- 音频传输（A2DP）： 通过 A2DP 协议实现音频流的无线传输。
- 串行通信（RFCOMM）： 通过 RFCOMM 实现模拟串行端口通信。
- BLE 传感器连接： 通过 GATT 协议连接低功耗传感器，并读取数据。

嵌入式 Linux 的蓝牙子系统是一个复杂的集成系统，涵盖了从底层硬件驱动到高级应用的各个层次，使得开发者能够灵活实现各种蓝牙应用。

15.3 开发板 Bluetooth 说明

15.3.1 原理图分析

开发板上有一个 WiFi&BT 芯片。打开开发板上开发板网盘 A 盘-基础资料\02、原理图\01、底板原理图\ ATK-DLRK3506B V1.0.pdf，搜索 BT。可以看到相关电路如下。



图：开发板 Bluetooth 电路

这个原理图展示了 USB WiFi 和蓝牙模块的电路连接，该模块是 Realtek 的 RTL8733BUUA。以下是对该电路的解析：

- 电源部分
  - VDD33: 连接到 P3V3 电源给模块。
  - CHIP_EN: 使能引脚，控制芯片的开启或关闭状态。

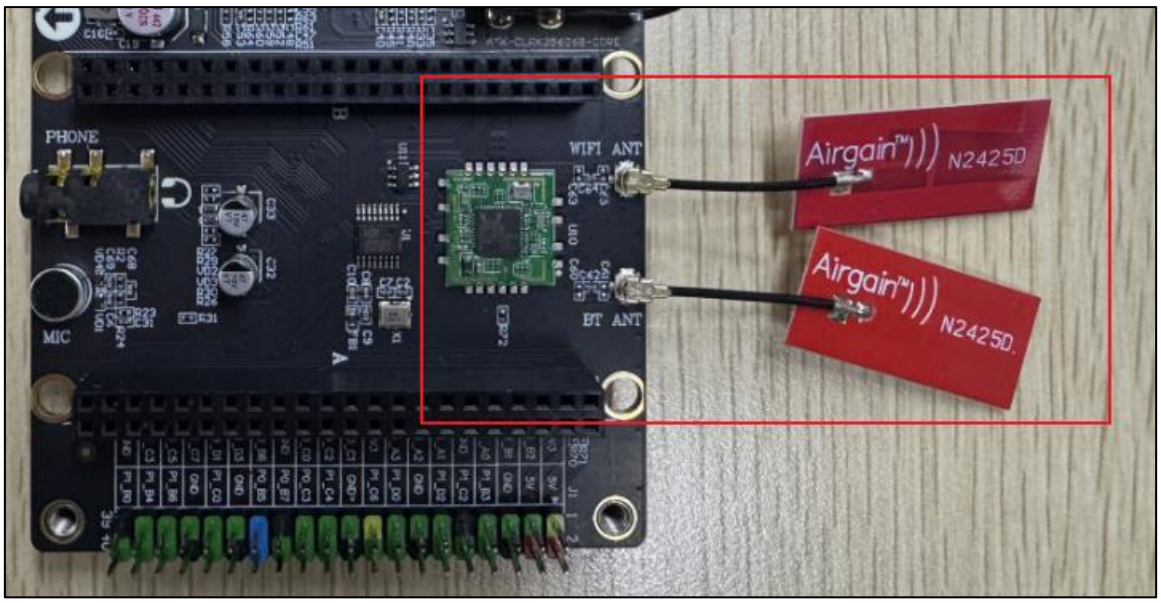
- USB_DM 和 USB_DP: 连接的是由 CPU_USB1_DP 和 CPU_USB1_DM 通过 CH334R 扩展出来的 HUB_DM2 和 HUB_DP2 引脚。

- 天线部分
  - WL_RF: 这是 WiFi 天线接口，连接到天线（WIFI_ANT），用于射频信号的传输和接收

这个原理图展示了一个典型的 USB WiFi 和蓝牙模块的硬件设计，通过一系列电源和控制电路实现其功能。这个设计注重模块的低功耗操作和稳定性，适用于嵌入式系统的无线通信应用。

15.3.2 硬件实物

开发板上的 Bluetooth 相关的电路位置如下图所示：



图：开发板 Bluetooth 电路

15.4 内核 Bluetooth 相关使能

内核源码配置文件 kernel/arch/arm/configs/ rk3506-atk-dlrk3506.config 和 rk3506_defconfig 中，Bluetooth 相关配置如下：

```
CONFIG_BT=y  //启用蓝牙支持，这是蓝牙子系统的核心配置。
CONFIG_BT_RFCOMM=y    //启用 RFCOMM 协议，主要用于仿真串行端口通信。
CONFIG_BT_RFCOMM_TTY=y    /*启用 RFCOMM TTY 支持，
                            用于将 RFCOMM 设备作为 TTY（串口）设备进行访问。*/
CONFIG_BT_BNEP=y /*启用蓝牙网络封装协议（BNEP），
                       主要用于网络连接共享（如蓝牙 PAN）。*/
CONFIG_BT_HIDP=y //启用蓝牙 HID 协议，用于支持蓝牙输入设备（如键盘、鼠标）。
```

15.5 板载蓝牙测试

这里介绍一下 linux 下的蓝牙测试常见的工具：

hciconfig：hciconfig 是 Linux 系统中一个用于配置和管理蓝牙主机控制器接口，设备的命令行工具。它能让用户查看和修改蓝牙设备的各种参数与状态。可以执行

```
hciconfig -h        //查看 hciconfig 工具使用说明
```

```
root@ATK-DLRK3506:/# hciconfig -h
hciconfig - HCI device configuration utility
Usage:
        hciconfig
        hciconfig [-a] hciX [command ...]
Commands:
        up                      Open and initialize HCI device
        down                    Close HCI device
        reset                   Reset HCI device
        rstat                   Reset statistic counters
        auth                    Enable Authentication
        noauth                  Disable Authentication
        encrypt                 Enable Encryption
        noencrypt               Disable Encryption
        piscan                  Enable Page and Inquiry scan
        noscan                  Disable scan
        iscan                   Enable Inquiry scan
        pscan                   Enable Page scan
        ptype       [type]      Get/Set default packet type
        lm          [mode]      Get/Set default link mode
        lp          [policy]    Get/Set default link policy
        name        [name]      Get/Set local name
        class       [class]     Get/Set class of device
        voice       [voice]     Get/Set voice setting
        iac         [iac]       Get/Set inquiry access code
        inqtpl      [level]     Get/Set inquiry transmit power level
        inqmode     [mode]      Get/Set inquiry mode
        inqdata     [data]      Get/Set inquiry data
        inqtype     [type]      Get/Set inquiry scan type
        inqparms    [win:int]   Get/Set inquiry scan window and interval
        pageparms   [win:int]   Get/Set page scan window and interval
        pageto      [to]        Get/Set page timeout
        afhmode     [mode]      Get/Set AFH mode
        sspmode     [mode]      Get/Set Simple Pairing Mode
        aclmtu      <mtu:pkt>   Set ACL MTU and number of packets
        scomtu      <mtu:pkt>   Set SCO MTU and number of packets
        delkey      <bdaddr>    Delete link key from the device
        oobdata                 Get local OOB data
        commands                Display supported commands
        features                Display device features
        version                 Display version information
        revision                Display revision information
        block       <bdaddr>    Add a device to the reject list
        unblock     <bdaddr>    Remove a device from the reject list
        lerandaddr  <bdaddr>    Set LE Random Address
        leadv       [type]      Enable LE advertising
                        0 - Connectable undirected advertising (default)
                        3 - Non connectable undirected advertising
        noleadv                 Disable LE advertising
        lestates                Display the supported LE states
root@ATK-DLRK3506:/#
```

图：hciconfig 帮助指令

图中列出了 hciconfig 的使用指令，比如：

```
hciconfig hci0              //查看蓝牙设备
hciconfig hci0 up           //启用蓝牙设备
hciconfig hci0 down         //禁用蓝牙
hciconfig hci0 piscan       //启用查询扫描
```

```
root@ATK-DLRK3506:/# hciconfig hci0
hci0:   Type: Primary  Bus: USB
        BD Address: 48:8F:4C:6F:E8:C1  ACL MTU: 1021:6  SCO MTU: 255:12
        UP RUNNING PSCAN ISCAN
        RX bytes:836 acl:0 sco:0 events:65 errors:0
        TX bytes:2306 acl:0 sco:0 commands:65 errors:0

root@ATK-DLRK3506:/# hciconfig hci0 up
root@ATK-DLRK3506:/# hciconfig hci0 down
[  966.370561] rtk_btcoex: Close BTCOEX
root@ATK-DLRK3506:/# hciconfig hci0 up
[  969.180790] rtk_btusb: btusb_open hdev->promisc ==0
[  969.180938] rtk_btusb: chip type value: 0x76
[  969.194176] rtk_btusb: chip type value: 0x76
[  969.194246] rtk_btusb: btusb_open set HCI UP RUNNING
[  969.194299] rtk_btcoex: Open BTCOEX
root@ATK-DLRK3506:/# hciconfig hci0 piscan
```

图：hciconfig 使用

hcitool：用于与蓝牙主机控制器接口（HCI）进行交互的命令行工具，它主要用于管理和操作蓝牙设备的底层功能。执行 hcitool –help 可以查看到该工具的使用：

```
root@ATK-DLRK3506:/# hcitool --help
hcitool - HCI Tool ver 5.77
Usage:
        hcitool [options] <command> [command parameters]
Options:
        --help  Display help
        -i dev  HCI device
Commands:
        dev     Display local devices
        inq     Inquire remote devices
        scan    Scan for remote devices
        name    Get name from remote device
        info    Get information from remote device
        spinq   Start periodic inquiry
        epinq   Exit periodic inquiry
        cmd     Submit arbitrary HCI commands
        con     Display active connections
        cc      Create connection to remote device
        dc      Disconnect from remote device
        sr      Switch central/peripheral role
        cpt     Change connection packet type
        rssi    Display connection RSSI
        lq      Display link quality
        tpl     Display transmit power level
        afh     Display AFH channel map
        lp      Set/display link policy settings
        lst     Set/display link supervision timeout
        auth    Request authentication
        enc     Set connection encryption
        key     Change connection link key
        clkoff  Read clock offset
        clock   Read local or remote clock
        lescan  Start LE scan
        leinfo  Get LE remote information
        lealadd Add device to LE Accept List
        lealrm  Remove device from LE Accept List
        lealsz  Read size of LE Accept List
        lealclr Clear LE Accept List
        lewladd Deprecated. Use lealadd instead.
        lewlrm  Deprecated. Use lealrm instead.
        lewlsz  Deprecated. Use lealsz instead.
        lewlclr Deprecated. Use lealclr instead.
        lerladd Add device to LE Resolving List
        lerlrm  Remove device from LE Resolving List
        lerlclr Clear LE Resolving List
        lerlsz  Read size of LE Resolving List
        lerlon  Enable LE Address Resolution
        lerloff Disable LE Address Resolution
        lecc    Create a LE Connection
        ledc    Disconnect a LE Connection
        lecup   LE Connection Update

For more information on the usage of each command use:
        hcitool <command> --help
root@ATK-DLRK3506:/#
```

图：hcitool 帮助指令

上图中列出了 hcitool 的操作指令，比如：

```
hcitool dev        //查看蓝牙设备
hcitool scan       //扫描蓝牙设备
```

```
root@ATK-DLRK3506:/# hcitool dev
Devices:
        hci0    48:8F:4C:6F:E8:C1
root@ATK-DLRK3506:/# hcitool scan
Scanning ...[ 9180.577598] rtk_btcoex: hci (periodic)inq start

[ 9190.830204] rtk_btcoex: inquiry complete
        00:13:EF:A0:04:F1       DX3 Pro
        90:F0:52:BC:8D:7B       MEIZU 20
        6C:06:D6:A3:FC:AB       sunny 明明的nova 7 5G
root@ATK-DLRK3506:/# hcitool scan
Scanning ...[ 9246.515950] rtk_btcoex: hci (periodic)inq start

[ 9256.760818] rtk_btcoex: inquiry complete
        6C:06:D6:
        90:F0:52:
        70:02:CA:
        00:13:EF:
        EC:81:50:75:4F:E6       iPhone
root@ATK-DLRK3506:/#
```

图：hcitool 指令

可以看到我们扫描到几个蓝牙设备，要想测试蓝牙是否正常，可以通过下面的指令去 ping 一下对应的蓝牙设备：

```
l2ping EC:81:50:75:4F:E6        //蓝牙 ping 测试指令，注意 l2ping 中 l（是 L 的小写）
```

```
root@ATK-DLRK3506:/# l2ping EC:81:50:75:4F:E6
[ 9490.960189] rtk_btcoex: hci create connection, start paging
[ 9491.499613] rtk_btcoex: connected, handle 0008, status 0x00
[ 9491.499786] rtk_btcoex: Page success
Ping: EC:81:50:75:4F:E6 from 48:8F:4C:6F:E8:C1 (data size 44) ...
0 bytes from EC:81:50:75:4F:E6 id 0 time 131.86ms
0 bytes from EC:81:50:75:4F:E6 id 1 time 93.68ms
0 bytes from EC:81:50:75:4F:E6 id 2 time 18.16ms
0 bytes from EC:81:50:75:4F:E6 id 3 time 84.83ms
0 bytes from EC:81:50:75:4F:E6 id 4 time 94.76ms
```

图：蓝牙测试